

Daftar Isi

1	LANDASAN TEORI	7
1.1	Perancangan Sistem	7
1.2	Validasi Sistem	7
1.3	Design Science Research (DSR)	7
1.4	Sistem Adaptif	8
1.5	Personalisasi	8
1.6	Artificial Intelligence (AI)	9
1.7	Large Language Models (LLM)	9
1.7.1	Local LLM dan Platform Ollama	9
1.8	Retrieval-Augmented Generation (RAG)	10
1.9	Hybrid Search dan Reciprocal Rank Fusion (RRF)	11
1.9.1	Konsep Hybrid Search	11
1.9.2	Reciprocal Rank Fusion (RRF)	11
1.10	Server-Sent Events (SSE)	12
1.11	Model Embedding dan Representasi Vektor	12
1.12	Metode Prototype	13
1.13	Black Box Testing	13
1.14	Basis Data	13
1.14.1	Basis Data Relasional	13
1.14.2	Basis Data Vektor dan pgvector	14
1.15	Framework dan Library Modern	14
1.15.1	FastAPI dan Model Asinkron	14
1.15.2	Next.js dan React	14
1.15.3	Authentication-as-a-Service	15
1.16	Web Crawling dan Web Scraping	15
1.17	Skala Likert	15
1.18	Bimbingan Karir	16
1.19	Social Cognitive Career Theory (SCCT)	16
1.20	Paradigma Life Design	16
1.21	Penelitian Sebelumnya	16
1.22	Posisi Penelitian Sekarang	17
2	OBJEK DAN METODOLOGI PENELITIAN	19
2.1	Objek Penelitian	19
2.2	Metodologi Penelitian	19

2.3	Jenis Artefak Penelitian	21
2.4	Rancangan Penelitian	21
2.4.1	Subjek dan Lokasi Penelitian	21
2.4.2	Teknik Pengumpulan Data	22
2.4.3	Teknik Pengembangan Sistem	23
2.4.4	Arsitektur Sistem	28
2.4.5	Analisis Kompleksitas Komputasional Pipeline RAG	32
2.4.6	Detail Pipeline RAG (8 Tahap)	34
2.4.7	Alur Paralel: Adaptive Profile System (Slow Path)	42
2.4.8	Teknik Analisis Data	44
2.5	Skema Database dan Manajemen Infrastruktur	49
2.5.1	Skema Database PostgreSQL	50
2.5.2	Strategi Deployment: Native VPS Ubuntu dengan systemd	54
3	HASIL DAN PEMBAHASAN	62
3.1	Hasil Perancangan dan Pengembangan Artefak	62
3.1.1	Pengumpulan Data Kebutuhan Awal	62
3.1.2	Pengumpulan dan Persiapan Data Lowongan	66
3.1.3	Implementasi Sistem	74
3.2	Tampilan Antarmuka Sistem	88
3.2.1	Halaman Utama dan Autentikasi	88
3.2.2	Input Narasi Karier	88
3.2.3	Respons Rekomendasi RAG (Streaming)	89
3.2.4	Detail Skor RAG dan Skill Overlap	89
3.2.5	Riwayat Rekomendasi	90
3.2.6	Bukti Sistem Adaptif (Profil Akumulatif)	90
3.2.7	Health Check dan Status Sistem	90
3.3	Demonstrasi: Pengujian Fungsional Black-Box	91
3.4	Evaluasi Teknis: Kualitas Retrieval RAG	93
3.4.1	Metodologi Evaluasi dan <i>Ground Truth</i>	93
3.4.2	Hasil Evaluasi Retrieval	95
3.4.3	Analisis Hasil	96
3.5	Evaluasi Kinerja Sistem (Latency)	97
3.6	Evaluasi Penerimaan Pengguna (UAT)	98
3.6.1	Instrumen dan Prosedur	98
3.6.2	Template Pengolahan Data	98
3.7	Pembahasan	99
3.7.1	Sintesis Evaluasi terhadap Artefak DSR	99

3.7.2	Kontribusi Teknis Utama	100
3.7.3	Analisis Keputusan Desain	101
3.7.4	Keterbatasan Penelitian dan <i>Future Work</i>	102
3.8	Perbandingan dengan Sistem Rekomendasi Karier Sejenis	102

Daftar Gambar

1.1	Proses <i>Design Science Research</i> (Sumber: Peffers et al., 2007) . . .	8
1.2	Arsitektur Model <i>Retrieval-Augmented Generation</i> (RAG) (Sumber: diadaptasi dari ?)	10
1.3	Proses Metode Prototype (Sumber: Kholis & Zaky, 2025)	13
2.1	Tahapan <i>Design Science Research</i> (DSR) (Sumber: Peffers et al., 2007)	20
2.2	Alur Metode <i>Prototype</i> (Sumber: Kholis & Zaky, 2025)	24
2.3	Arsitektur Pipeline Pembersihan Data dan Indexing Vektor (ETL)	27
2.4	Arsitektur Sistem NusaNara	30
2.5	Alur Kerja Sistem Rekomendasi dan Pembaruan Profil Adaptif NusaNara	44
2.6	Ilustrasi Alur Kerja Inti Artefak “NusaNara”	48
2.7	Rancangan Penelitian (DSR + Metode Prototype Terintegrasi) .	49
3.1	Diagram Komposisi Jenjang Pendidikan Responden (N=115) . .	62
3.2	Diagram Sebaran Fase Pendidikan Responden per Jenjang . . .	63
3.3	Diagram Sebaran 5 Provinsi Teratas Responden	63
3.4	Persentase Tingkat Kecemasan dan Kesulitan Karier (N=115) .	64
3.5	Persentase Penilaian Terhadap Layanan Karier Saat Ini (N=115)	65
3.6	Distribusi Jawaban Kebutuhan Sistem Bimbingan Karier Baru .	65
3.7	Tangkapan Layar Data Vektor dan Leksikal pada PostgreSQL (DBeaver)	67
3.8	Hasil Eksekusi Query Agregasi Total Data dan Kelengkapan Kolom	68
3.9	Hasil Eksekusi Query Distribusi Data per Kluster Karier	68
3.10	Analisis Distribusi Lowongan dan Kelengkapan Data Gaji . . .	69
3.11	Visualisasi t-SNE Distribusi 720 Lowongan dalam Ruang Vektor 768 Dimensi (setiap titik merepresentasikan satu dokumen lowongan, diwarnai sesuai kluster kariernya)	71
3.12	Visualisasi Mekanisme <i>Semantic Search</i> (ANN via IVFFlat/pgvector): Posisi Kueri Pengguna (bintang merah ★) dan Dokumen Terambil (Top-10 lingkaran abu-abu; Top-3 lingkaran emas) dalam Proyeksi t-SNE	72

3.13 Distribusi <i>Cosine Similarity</i> Intra-Klaster per Domain Karier (Boxplot; garis putih = nilai median)	73
3.14 Arsitektur Deployment Produksi NusaNara (VPS + Vercel) . .	84
3.15 Halaman Login NusaNara dengan Integrasi Clerk Auth	88
3.16 Antarmuka Input Narasi Karier	89
3.17 Output Rekomendasi RAG dengan Skor RAG dan Skill Overlap	89
3.18 Transparansi Skor RAG dan Skill Overlap pada Output Rekomendasi	89
3.19 Halaman Riwayat Rekomendasi Pengguna	90
3.20 Demonstrasi Sistem Adaptif: Profil Akumulatif Lintas Sesi . . .	90
3.21 Respons Health Check Endpoint (<code>/health</code>)	91
3.22 Visualisasi Studi Ablasi: Dampak <i>Re-ranking</i> pada Performa Retrieval	96

Daftar Tabel

1.1	Perbandingan Local LLM vs. Cloud API	10
1.2	Skala Likert (Sumber: Simamora, 2022)	16
1.3	Penelitian Terdahulu	17
1.4	Posisi Penelitian Ini terhadap Studi Sebelumnya	18
2.1	Distribusi <i>Knowledge Base</i> Berdasarkan 8 Klaster Karier	23
2.2	Analisis Kompleksitas Komputasional Per Tahap Pipeline RAG	33
2.3	Dekomposisi Latensi Pipeline RAG (TTFB = 2.400 ms)	42
2.4	Skala Interpretasi Grand Mean UAT	48
2.5	Proses Produksi di VPS dan Manajemen via Systemd	55
3.1	Ringkasan Hasil Analisis Kebutuhan (N=115)	64
3.2	Ringkasan Karakteristik Kolom <i>Knowledge Base</i>	69
3.3	Skema Tabel <code>knowledge_base</code> pada PostgreSQL 16	70
3.4	Spesifikasi Perangkat Keras VPS Produksi	74
3.5	Spesifikasi Perangkat Lunak Produksi	76
3.6	Struktur 5 Seksi Output Wajib Prompt Decision Intelligence	80
3.7	Hasil Pengujian Fungsional <i>Black-Box</i>	92
3.8	Kriteria Anotasi <i>Ground Truth</i> (Skala Relevansi 0–3)	94
3.9	Sebagian Dataset 20 Query Uji beserta Target Klaster	94
3.10	Hasil Evaluasi Retrieval RAG	95
3.11	Hasil Pengukuran Kinerja Sistem (Waktu Respons) dalam Detik	97
3.12	Instrumen UAT: 7 Item Pertanyaan Skala Likert 1–5	98
3.13	Hasil UAT (N=15 Responden)	99
3.14	Hasil Evaluasi Retrieval: Ablation Study Dua Kondisi	100
3.15	Perbandingan NusaNara dengan Pendekatan Sistem Rekomendasi Karier Lainnya	103

BAB 1

LANDASAN TEORI

Bab ini menyajikan fondasi teoretis yang menjadi dasar bagi keseluruhan penelitian. Uraian disusun secara sistematis, dimulai dari konsep perancangan dan validasi sistem, menitik ke teknologi inti yang digunakan, membahas kerangka teoretis dari domain bimbingan karir, hingga memetakan lanskap riset untuk memposisikan kebaruan penelitian ini.

1.1 Perancangan Sistem

Perancangan sistem merupakan tahapan krusial dalam *System Development Life Cycle* yang mentransformasikan kebutuhan fungsional dan non-fungsional menjadi struktur teknis dan arsitektur sistem. Proses ini mencakup pemodelan data menggunakan *Entity-Relationship Diagram* dan Normalisasi, desain antarmuka pengguna yang ergonomis, penentuan komponen perangkat keras serta perangkat lunak, serta integrasi modul untuk memastikan kinerja, skalabilitas, keamanan, dan kemudahan pemeliharaan. Tujuannya adalah menghasilkan spesifikasi teknis yang meminimalkan risiko proyek, memenuhi kebutuhan pengguna dengan efisiensi biaya dan waktu, serta mendukung tujuan strategis organisasi melalui integrasi sistem yang optimal ?.

1.2 Validasi Sistem

Validasi adalah proses yang dilakukan untuk memastikan bahwa sistem atau perangkat lunak yang dikembangkan benar-benar memenuhi dan menjawab kebutuhan pengguna. Validasi perangkat lunak merupakan fase utama dalam rekayasa kebutuhan yang memastikan bahwa kebutuhan sesuai dengan sistem target, dengan tujuan mendeteksi dan memperbaiki kesalahan dalam kebutuhan yang telah ditentukan sebelumnya ?.

1.3 Design Science Research (DSR)

Menurut ?, *Design Science Research* (DSR) merupakan pendekatan penelitian yang berorientasi pada penciptaan dan evaluasi artefak guna memecahkan masalah organisasi yang teridentifikasi melalui proses yang sistematis dan *ri-*

gor. Proses tersebut meliputi: (1) *problem identification*, (2) *objectives of a solution*, (3) *design & development*, (4) *demonstration*, (5) *evaluation*, dan (6) *communication*.

Output penelitian DSR berupa artefak dapat berwujud *construct* (konsep baru), *model* (representasi), *method* (prosedur atau algoritma), maupun *instantiation* (implementasi sistem).

[GAMBAR PLACEHOLDER]

Proses Design Science Research (Peffers et al., 2007) – Gambar 2.1 dari draf asli
Sisipkan dari dokumen asli

Gambar 1.1: Proses *Design Science Research* (Sumber: Peffers et al., 2007)

1.4 Sistem Adaptif

Sistem adaptif merupakan sistem yang mampu menyesuaikan perilaku, konfigurasi, atau strukturnya secara otomatis sebagai respons terhadap perubahan kondisi internal maupun eksternal ?. Sistem cerdas modern menggunakan prinsip adaptabilitas ini untuk mempelajari pola interaksi secara dinamis dan memperbarui status agen (memori) tanpa memerlukan modifikasi sistem secara manual ?.

1.5 Personalisasi

Personalisasi merupakan bentuk adaptasi di mana sistem menyesuaikan konten, layanan, atau antarmuka berdasarkan karakteristik, preferensi, dan kebutuhan spesifik pengguna ?. Personalisasi tingkat lanjut (*hyper-personalization*) mengintegrasikan jejak historis dan profil semantik untuk menghasilkan output yang kontekstual, sehingga relevansi informasi yang diberikan kepada setiap individu menjadi optimal ?.

1.6 Artificial Intelligence (AI)

Kecerdasan buatan (*Artificial Intelligence*, AI) adalah bidang ilmu komputer yang mempelajari dan merealisasikan agen atau sistem yang mampu menunaikan tugas-tugas yang biasanya membutuhkan kecerdasan manusia, termasuk

pengenalan pola, inferensi, pembuatan keputusan, dan pembelajaran dari data ?.

1.7 Large Language Models (LLM)

Large Language Models (LLM) adalah model bahasa berbasis jaringan saraf berskala sangat besar yang dilatih pada korpus teks masif untuk memprediksi token berikutnya dalam sebuah urutan ?. Peningkatan skala model secara signifikan memperluas kemampuan generalisasi, termasuk *few-shot learning*.

Studi menunjukkan bahwa performa LLM mengikuti *scaling laws*, di mana kualitas model meningkat secara konsisten mengikuti *power-law* terhadap ukuran model, jumlah data, dan komputasi pelatihan ?.

1.7.1 Local LLM dan Platform Ollama

Perkembangan teknik kuantisasi model (seperti format GGUF dan GPTQ) memungkinkan LLM berskala miliaran parameter dijalankan secara lokal pada perangkat keras komoditas (*commodity hardware*) tanpa memerlukan infrastruktur klaster komputasi pusat. Paradigma pemrosesan lokal ini didukung oleh berbagai *runtime engine*, salah satunya adalah **Ollama**, yang menyederhanakan siklus hidup pengoperasian LLM terbuka melalui antarmuka baris perintah dan layanan HTTP API ?.

Keunggulan pendekatan lokal dibandingkan Cloud API disajikan pada Tabel ??:

Tabel 1.1: Perbandingan Local LLM vs. Cloud API

Aspek	Cloud API (Gemini, OpenAI)	Local LLM (Ollama)
Privasi data	Data dikirim ke server eksternal	Data tetap di infrastruktur lokal
Biaya operasional	<i>Per-token billing</i>	Gratis setelah setup awal
Latensi	Bergantung jaringan	Bergantung hardware lokal
Kontrol	Terbatas, bergantung vendor	Penuh, termasuk parameter model
Ketersediaan offline	Tidak	Ya

1.8 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) adalah arsitektur yang menggabungkan proses *retrieval* (pengambilan) dokumen dari basis pengetahuan eksternal dengan kemampuan model generatif untuk menghasilkan jawaban yang lebih akurat dan kontekstual ?.

Alur kerja RAG dimulai ketika pengguna mengajukan pertanyaan (1), yang memicu pencarian semantik untuk mengambil dokumen relevan dari basis pengetahuan (2). Dokumen tersebut digabungkan dengan *prompt* asli untuk memperkaya konteks (3), kemudian model memproses informasi tersebut untuk menghasilkan respons akhir (4).

[GAMBAR PLACEHOLDER]

Arsitektur RAG – Gambar 2.2 dari draf asli

Sisipkan dari dokumen asli

Gambar 1.2: Arsitektur Model *Retrieval-Augmented Generation* (RAG)
(Sumber: diadaptasi dari ?)

Pendekatan ini terbukti efektif mengurangi *hallucination* karena model tidak hanya mengandalkan memori internal, tetapi juga referensi eksternal yang dinamis ?.

1.9 Hybrid Search dan Reciprocal Rank Fusion (RRF)

1.9.1 Konsep Hybrid Search

Hybrid Search menggabungkan dua paradigma pencarian yang saling komplementer: pencarian semantik berbasis vektor (*dense retrieval*) dan pencarian berbasis kata kunci (*sparse/lexical retrieval*). Masing-masing memiliki kelemahan inheren ?:

- Pencarian semantik: memahami sinonim dan konteks, tetapi lemah terhadap *exact match* istilah teknis.
- Pencarian kata kunci: sangat tepat untuk istilah spesifik, tetapi tidak memahami variasi makna.

Dalam sistem manajemen basis data modern, keduanya dapat diimplementasikan secara terintegrasi; misalnya, pencarian semantik menggunakan fungsi jarak spasial (*Cosine Distance*), dan pencarian kata kunci menggunakan indeks *Generalized Inverted Index* (GIN) ?.

1.9.2 Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion (RRF) adalah algoritma *rank aggregation* yang menggabungkan beberapa *ranked list* menjadi satu peringkat tunggal tanpa normalisasi skor eksplisit ?. Rumus RRF adalah:

$$RRF_score(d) = \frac{1}{k + rank_{semantic}(d)} + \frac{1}{k + rank_{lexical}(d)} \quad (1.1)$$

di mana:

- d = dokumen kandidat
- $rank_{semantic}(d)$ = peringkat dokumen d dalam hasil pencarian semantik
- $rank_{lexical}(d)$ = peringkat dokumen d dalam hasil pencarian kata kunci
- k = konstanta *smoothing* ($k = 60$, divalidasi oleh Cormack et al., 2009)

Dokumen yang muncul di peringkat tinggi pada **kedua** sistem mendapat skor RRF signifikan lebih tinggi, menghasilkan daftar kandidat final yang tangguh terhadap kelemahan masing-masing metode *retrieval* secara individu.

1.10 Server-Sent Events (SSE)

Server-Sent Events (SSE) adalah standar web API yang memungkinkan server mengirimkan data ke klien secara *unidirectional* melalui koneksi HTTP yang persisten ?. SSE dirancang khusus untuk aliran data satu arah dari server ke klien, menjadikannya pilihan yang lebih sederhana dan efisien dibandingkan WebSocket untuk kasus penggunaan *streaming* teks.

Dalam konteks LLM, SSE menjadi mekanisme standar untuk mengirimkan token respons satu per satu secara *real-time*, sehingga pengguna dapat mulai membaca respons sebelum LLM selesai menghasilkan keseluruhan teks. Hal ini secara signifikan menurunkan persepsi latensi (*perceived latency*).

Secara teknis, aliran data SSE diformat menggunakan `text/event-stream`:

```
1 Content-Type: text/event-stream
2 Cache-Control: no-cache
```

```
3 Connection: keep-alive
4
5 data: Data chunk 1
6
7 data: Data chunk 2
8
9 data: [DONE]
```

Listing 1.1: Struktur Dasar Payload SSE

1.11 Model Embedding dan Representasi Vektor

Model *embedding* adalah pilar utama dalam pemrosesan bahasa alami modern yang bertugas memetakan teks (kata, kalimat, atau dokumen) ke dalam ruang vektor berdimensi tinggi, sehingga mesin dapat menghitung metrik kesamaan (*similarity*) secara matematis ?.

Perkembangan arsitektur *embedding* bergeser dari model *dense* klasik menuju *Mixture-of-Experts* (MoE). Arsitektur MoE memungkinkan model memiliki skala parameter yang besar namun membatasi jumlah komputasi aktif per token dengan merutekan pemrosesan ke sub-jaringan (*expert*) tertentu ?. Model *embedding* terbaru, seperti Nomic Embed ?, mengadopsi MoE untuk menghasilkan representasi vektor berkualitas tinggi (seperti 768 dimensi) yang mendukung instruksi asimetris (*asymmetric embedding*) untuk query dan dokumen.

1.12 Metode Prototype

Metode *prototype* adalah pendekatan pengembangan perangkat lunak yang bersifat iteratif dan interaktif, di mana sistem dikembangkan melalui siklus penyempurnaan berdasarkan desain awal hingga tercipta versi yang mendekati produk akhir ?.

[GAMBAR PLACEHOLDER]

Proses Metode Prototype (Kholis dan Zaky, 2025) – Gambar 2.3

dari draf asli

Sisipkan dari dokumen asli

Gambar 1.3: Proses Metode Prototype (Sumber: Kholis & Zaky, 2025)

1.13 Black Box Testing

Black Box Testing adalah metode pengujian perangkat lunak yang berfokus pada fungsionalitas sistem berdasarkan spesifikasi, tanpa memperhatikan struktur internal kode. Tujuannya adalah memastikan bahwa input yang diberikan menghasilkan output sesuai dengan yang diharapkan pengguna ?.

1.14 Basis Data

Basis data adalah kumpulan data yang disimpan secara terstruktur dan saling berelasi, dikelola menggunakan *Database Management System* (DBMS) ?.

1.14.1 Basis Data Relasional

Basis data relasional menyimpan data dalam bentuk tabel yang saling terhubung melalui kunci primer dan kunci asing, menggunakan SQL (*Structured Query Language*) ?. PostgreSQL merupakan salah satu RDBMS *open-source* tingkat lanjut yang banyak digunakan karena ekstensibilitasnya.

1.14.2 Basis Data Vektor dan pgvector

Basis data vektor adalah sistem penyimpanan yang dioptimalkan untuk mengelola data vektor berdimensi tinggi ?. Secara historis, solusi *standalone* seperti Milvus, Pinecone, dan Weaviate digunakan untuk tujuan ini.

Namun, **pgvector** muncul sebagai sebuah ekstensi *open-source* untuk PostgreSQL yang memungkinkan penyimpanan vektor berdimensi tinggi dan pencarian *Approximate Nearest Neighbor* (ANN) secara *native* dalam basis data relasional. Pendekatan terintegrasi ini menawarkan keunggulan arsitektural:

1. **Unified architecture:** data teks, vektor, dan relasional tersimpan dalam satu transaksi ACID.

2. **Hybrid search native:** pencarian semantik dan FTS dalam satu instruksi SQL.
3. **Kemudahan pemeliharaan:** mengurangi kompleksitas sinkronisasi antar-basis data (*database drift*).

1.15 Framework dan Library Modern

1.15.1 FastAPI dan Model Asinkron

FastAPI adalah *web framework* modern untuk Python yang mendukung operasi asinkron (*async/await*) berdasarkan standar ASGI ?. Keunggulan kerangka kerja ini meliputi dukungan native untuk *streaming* data (seperti SSE) dan throughput tinggi ketika diintegrasikan dengan driver basis data asinkron seperti *asyncpg*, sehingga mampu menangani ribuan koneksi secara bersamaan (*high concurrency*) tanpa memblokir *thread* utama ?.

1.15.2 Next.js dan React

Next.js adalah kerangka kerja (*meta-framework*) berbasis React yang menyediakan fitur set tingkat produksi seperti *Server-Side Rendering* (SSR), routing berbasis aplikasi (*App Router*), dan optimasi aset dinamis ?. Penggunaan kerangka kerja modern pada sisi klien (frontend) memungkinkan pembuatan *Single Page Application* (SPA) yang interaktif namun tetap memiliki performa muat yang tinggi.

1.15.3 Authentication-as-a-Service

Manajemen sesi konvensional rentan terhadap celah keamanan (seperti kebocoran kunci rahasia HS256). Pendekatan modern memanfaatkan layanan pihak ketiga (*Identity Provider*) seperti Clerk atau Auth0 yang menyediakan *Authentication-as-a-Service*. Layanan ini menangani pendaftaran pengguna dan *Single Sign-On* (SSO) berbasis OAuth 2.0 ?, kemudian menerbitkan token JWT ? menggunakan kriptografi asimetris (RS256). Server backend (*resource server*) hanya perlu memverifikasi tanda tangan JWT tersebut menggunakan *JSON Web Key Set* (JWKS) publik milik *Identity Provider*.

1.16 Web Crawling dan Web Scraping

Web crawling adalah penjelajahan sistematis halaman web oleh *crawler* untuk menemukan, mengunduh, dan mengindeks konten ?.

Web scraping adalah proses otomatis untuk mengambil data dari situs web dan mengubahnya dari format tidak terstruktur menjadi terstruktur ?. Untuk menghadapi aplikasi web modern yang berbasis *Single Page Application* (SPA) dan sangat mengandalkan eksekusi JavaScript (seperti React atau Vue) untuk memuat konten, *scraper* statis seringkali gagal. Oleh karena itu, pendekatan tingkat lanjut menggunakan sistem *headless browser* (seperti Selenium WebDriver) diperlukan untuk menyimulasikan peramban nyata yang merender DOM secara penuh sebelum proses ekstraksi dilakukan ?.

1.17 Skala Likert

Skala Likert ditemukan oleh Rensis Likert pada tahun 1932 untuk mengukur sikap, pendapat, atau persepsi seseorang terhadap suatu objek atau pernyataan ?. Responden diminta menyatakan tingkat persetujuan terhadap pernyataan dalam skala bertingkat (biasanya 1–5 atau 1–7). Skala ini secara luas diadopsi sebagai instrumen evaluasi penerimaan pengguna (*User Acceptance Testing*) dalam rekayasa perangkat lunak.

Tabel 1.2: Skala Likert (Sumber: Simamora, 2022)

Skala	Keterangan
1	Sangat Tidak Setuju
2	Tidak Setuju
3	Cukup
4	Setuju
5	Sangat Setuju

1.18 Bimbingan Karir

Bimbingan karir adalah proses terstruktur yang membantu individu mengelola perkembangan karir, membuat keputusan pendidikan dan pekerjaan, serta menavigasi transisi di pasar kerja yang berubah cepat ?. UNESCO menekankan bahwa bimbingan karir modern berperan penting dalam membekali individu

dengan *career management skills* ?.

1.19 Social Cognitive Career Theory (SCCT)

Social Cognitive Career Theory (SCCT) menjelaskan perkembangan minat, pemilihan, dan pencapaian karir melalui interaksi dinamis antara keyakinan efikasi diri (*self-efficacy*), ekspektasi hasil (*outcome expectations*), dan tujuan personal ?. SCCT sangat relevan di era *boundaryless career*, di mana jalur karir bersifat non-linear ?.

1.20 Paradigma Life Design

Paradigma *Life Design* menekankan bahwa individu abad ke-21 secara aktif mengonstruksi jalur karir melalui narasi yang mengintegrasikan pengalaman masa lalu, refleksi masa kini, dan proyeksi masa depan ?. Pendekatan ini menggeser fokus dari sekadar mencocokkan *traits* (sifat) dengan okupasi, menuju penciptaan makna yang holistik di era kerja yang cair.

1.21 Penelitian Sebelumnya

Tabel ?? menyajikan perbandingan penelitian sebelumnya yang relevan.

Tabel 1.3: Penelitian Terdahulu

Peneliti	Judul	Metode/Teknologi	Kontribusi Utama	Keterbatasan
Nagur Vali Shaik (India, 2025)	<i>Human challe- nging career and future advice chatbot using RAG</i>	RAG, Zephyr- 7B, FAISS, LangChain, Streamlit	Chatbot in- teraktif 24/7 dengan retrieval dokumen kon- tekstual untuk rekomendasi karier real-time	Belum ada uji pengguna; tidak adaptif terhadap perubahan pro- fil dari waktu ke waktu

Lanjut di halaman berikutnya

Peneliti	Judul	Metode/Teknologi	Kontribusi Utama	Keterbatasan
S. Westman (2021)	<i>Artificial Intelligence for Career Guidance</i>	Kajian pustaka, tinjauan sistem AI	Model konseptual AI sebagai <i>virtual coach</i> adaptif sepanjang hidup	Berbasis tinjauan teori, tanpa prototipe atau validasi pengguna
RG Guntara (Indonesia, 2023)	<i>Web-Based Counseling Skills Evaluation using DSRM</i>	DSR, PHP, MySQL, YouTube API	Sistem evaluasi keterampilan konseling berbasis DSR	Tidak adaptif; tanpa komponen AI canggih
Haerani dkk. (2024)	<i>Sistem informasi cerdas career guidance berbasis minat</i>	BFS, RMIB, PHP, MySQL	Rekomendasi jurusan berbasis 8 kategori minat RMIB	Tidak ada personalisasi adaptif; sampel UAT terbatas
Elysia (2024)	S. <i>Chatbot RAG untuk Layanan Pelanggan E-Commerce</i>	RAG (Llama 2 + ElasticSearch), FastAPI, React	Chatbot e-commerce dengan retrieval produk dan generasi naratif	Respons statis per sesi, tanpa pembelajaran dari histori interaksi

1.22 Posisi Penelitian Sekarang

Penelitian ini berfokus pada perancangan dan validasi sistem bimbingan karir adaptif berbasis RAG dan LLM. Posisi penelitian ini ditunjukkan pada Tabel ??.

Tabel 1.4: Posisi Penelitian Ini terhadap Studi Sebelumnya

Fitur/Kategori	Silva	Westman	Guntara	Haerani	Elysia	NusaNara
Arsitektur RAG & LLM	Ya	Tidak	Tidak	Tidak	Ya	Ya
Konteks Indonesia	Tidak	Tidak	Ya	Ya	Ya	Ya
Sistem Adaptif (Dinamis)	Tidak	Ya	Tidak	Tidak	Tidak	Ya
Validasi Penggunaan Formal	Tidak	Tidak	Tidak	Ya	Ya	Ya
LLM Lokal (Privasi)	Tidak	–	–	–	Tidak	Ya
Hybrid Search (RRF)	Tidak	–	–	–	Tidak	Ya

Dibandingkan penelitian sebelumnya, NusaNara memiliki keunggulan unik pada kombinasi: arsitektur RAG dengan *Hybrid Search* (RRF), LLM yang berjalan sepenuhnya lokal (privasi data terjaga), sistem adaptif non-blocking berbasis `asyncio`, dan basis pengetahuan yang dikurasi spesifik untuk pasar kerja Indonesia.

BAB 2

OBJEK DAN METODOLOGI PENELITIAN

Bab ini menguraikan secara rinci mengenai objek penelitian serta pendekatan dan prosedur metodologis yang digunakan. Setiap tahapan, mulai dari rancangan sistem hingga teknik analisis data, dijelaskan secara sistematis untuk memastikan penelitian ini dilaksanakan dengan *rigor* dan validitas yang tinggi.

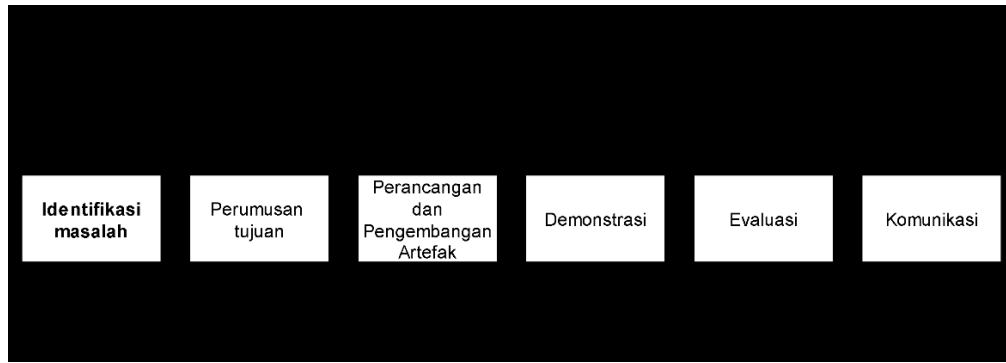
2.1 Objek Penelitian

Objek penelitian ini adalah sebuah artefak komputasi berupa prototipe aplikasi web fungsional yang diberi nama “**NusaNara**”. Artefak ini mencakup kode program, keseluruhan arsitektur *full-stack* yang dirancang, basis pengetahuan yang dikurasi, serta alur interaksi pengguna yang diimplementasikan. Artefak inilah yang akan dievaluasi untuk menjawab rumusan masalah penelitian.

2.2 Metodologi Penelitian

Penelitian ini mengadopsi pendekatan *Design Science Research* (DSR) yang dimulai dari *entry point* yang berpusat pada masalah (*problem-centered initiation*), di mana identifikasi masalah nyata di lapangan menjadi pemicu utama. Pemilihan DSR didasarkan pada argumen bahwa fokus penelitian ini adalah pada penciptaan solusi (sebuah sistem baru), bukan pada deskripsi atau eksplanasi fenomena yang sudah ada ?.

Alur penelitian mengikuti enam fase utama DSR, sebagaimana diilustrasikan pada Gambar ??:



Gambar 2.1: Tahapan *Design Science Research* (DSR) (Sumber: Peffers et al., 2007)

1. **Identifikasi Masalah:** Melakukan studi literatur untuk memvalidasi urgensi masalah dan merumuskan pertanyaan penelitian. Didukung oleh data BPS ? yang menunjukkan TPT lulusan SMK tertinggi sebesar 8,10%, serta survei kebutuhan terhadap 115 responden aktif.
2. **Perumusan Tujuan:** Menentukan kebutuhan fungsional solusi: sistem yang mampu memproses narasi karier pengguna secara alami, retrieve konteks faktual dari 720 lowongan pekerjaan, dan menghasilkan rekomendasi yang terstruktur dan dapat ditelusuri (*traceable*).
3. **Perancangan & Pengembangan Artefak:** Membangun prototipe fungsional “NusaNara” secara iteratif menggunakan metode *prototype*, berdasarkan arsitektur *full-stack* yang dirancang spesifik untuk mengimplementasikan alur RAG dengan *Hybrid Search* secara efisien (lihat Gambar ??).
4. **Demonstrasi:** Uji teknis melalui *black-box testing* untuk memvalidasi alur fungsi utama, memastikan stabilitas, dan memperbaiki *bug* kritikal sebelum evaluasi formal.
5. **Evaluasi:** Evaluasi artefak menggunakan pendekatan empat lapis: (a) evaluasi teknis retrieval (P@k, MRR), (b) pengujian fungsional *black-box*, (c) evaluasi kinerja sistem (latensi), dan (d) evaluasi penerimaan pengguna (UAT Likert).
6. **Komunikasi:** Menganalisis dan mendokumentasikan seluruh temuan penelitian dalam laporan skripsi.

2.3 Jenis Artefak Penelitian

Sesuai taksonomi artefak DSR ?, penelitian ini menghasilkan tiga jenis artefak:

1. **Model (Arsitektur Konseptual):** Cetak biru arsitektur *full-stack* yang merinci orkestrasi antara empat lapisan utama:
 - *Frontend*: Next.js 14 (TypeScript) dengan `@microsoft/fetch-event-source` dan Clerk Auth
 - *Backend*: FastAPI + Uvicorn (ASGI), pipeline RAG lengkap, Clerk JWT validator
 - *Database*: PostgreSQL 16 + pgvector (VECTOR(768)) + tsvector FTS
 - *Local AI*: Ollama (Llama 3.1 8B (4-bit Quantized) sebagai generator & extractor agent; nomic-embed-text-v2-moe sebagai model embedding 768 dimensi)
2. **Instantiation (Implementasi Fisik):** Prototipe fungsional aplikasi web “NusaNara” yang berjalan di produksi (VPS Ubuntu + Vercel), sebagai bukti nyata bahwa arsitektur yang diusulkan dapat diimplementasikan secara teknis.
3. **Method (Alur Kerja Terstruktur):** Metodologi evaluasi empat lapis yang menggabungkan evaluasi teknis RAG (P@k, MRR), pengujian fungsional *black-box*, evaluasi latensi sistem, dan UAT dengan instrumen Likert.

2.4 Rancangan Penelitian

2.4.1 Subjek dan Lokasi Penelitian

Populasi target adalah mahasiswa aktif dan siswa tingkat akhir (SMA/SMK) di Indonesia yang berada dalam fase eksplorasi dan pengambilan keputusan karier. Sampel terdiri dari dua kelompok:

1. **Pengguna Akhir:** 10–15 mahasiswa aktif dipilih menggunakan teknik *purposive sampling* berdasarkan kriteria: sedang dalam fase eksplorasi karier aktif (minimal semester 5 atau sedang mencari pekerjaan pertama). Teknik ini dipilih karena tujuan evaluasi adalah memvalidasi ke-

gunaan artefak pada konteks yang dituju, bukan generalisasi statistik ke populasi ?.

2. **Pengumpulan Data Kebutuhan Awal:** 115 responden (mahasiswa dan siswa) telah dilibatkan dalam survei kuantitatif awal untuk memvalidasi masalah dan memprioritaskan fitur sistem.

Seluruh proses pengumpulan data dilaksanakan secara daring (*remote*), memanfaatkan Google Form sebagai instrumen, sehingga memungkinkan jangkauan partisipan yang lebih luas.

2.4.2 Teknik Pengumpulan Data

Pengumpulan data dilaksanakan melalui tiga alur utama:

Alur 1: Survei Kebutuhan Awal (115 Responden) Tahap ini bertujuan memvalidasi masalah dan memprioritaskan kebutuhan sistem secara kuantitatif. Instrumen berupa kuesioner daring terstruktur mencakup:

- Validasi masalah: Skala Likert untuk mengukur tingkat kesulitan dalam pengambilan keputusan karier dan kecemasan fenomena “salah jurusan”.
- Prioritas kebutuhan: responden memberi peringkat (*rank-order*) fitur potensial (rekomendasi naratif, kemampuan adaptif, data lokal).

Data survei ini telah berhasil dikumpulkan dengan total **115 responden**.

Alur 2: Pengumpulan Data Konten untuk Knowledge Base Alur ini difokuskan pada pengumpulan data lowongan pekerjaan sebagai basis pengetahuan RAG. Pada tahap awal, tiga platform karier dievaluasi: Jobstreet, LinkedIn, dan Glints. Evaluasi menemukan:

- **Jobstreet:** menerapkan proteksi *bot-detection* berbasis Cloudflare yang memblokir permintaan otomatis secara konsisten.
- **LinkedIn:** tidak menampilkan informasi gaji secara publik – atribut yang relevan untuk analisis pasar kerja kontekstual.
- **Glints:** aksesibel secara teknis dan menyediakan atribut data yang lengkap.

Berdasarkan evaluasi ini, **Glints dipilih sebagai satu-satunya sumber data**. Scraping dilakukan menggunakan Selenium WebDriver dengan

konfigurasi *selenium-stealth*, diperlukan karena Glints merupakan *Single Page Application* (SPA) berbasis React yang mengandalkan JavaScript untuk merender konten.

Hasil akhir: **720 lowongan pekerjaan** berhasil dikumpulkan pada periode akhir **April 2026** (tepatnya 30 April 2026), terdistribusi ke dalam **8 klaster karier**, masing-masing klaster berisi tepat **90 lowongan**. Distribusi seimbang ini merupakan persyaratan metodologis utama untuk memastikan validitas evaluasi RAG yang adil lintas seluruh domain karier.

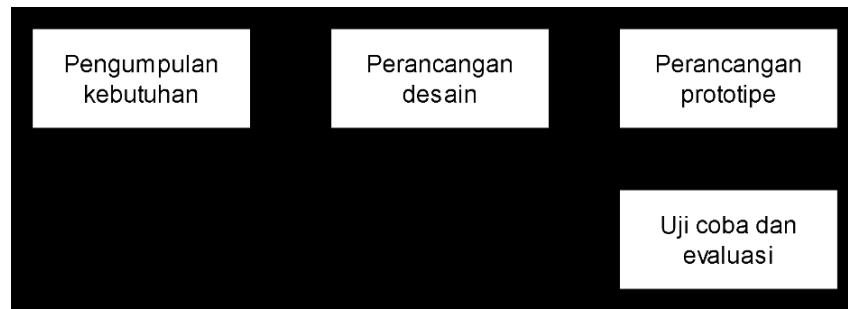
Tabel 2.1: Distribusi *Knowledge Base* Berdasarkan 8 Klaster Karier

No	Klaster Karier	Jumlah Lowongan
1	Teknologi & Perangkat Lunak	90
2	Analisis Data	90
3	Desain & Kreatif	90
4	Pemasaran Digital	90
5	Bisnis & Administrasi	90
6	Sales & Customer Service	90
7	Finance & Accounting	90
8	Education & Training	90
Total		720

Alur 3: Pengumpulan Data Evaluasi (UAT) Setelah prototipe mencapai status stabil, data evaluasi penerimaan pengguna dikumpulkan menggunakan kuesioner 7 pertanyaan Skala Likert (1–5) dari 10–15 responden (lihat Sub-bab ??).

2.4.3 Teknik Pengembangan Sistem

Pengembangan sistem dilakukan secara iteratif menggunakan metode *prototype*.



Gambar 2.2: Alur Metode *Prototype* (Sumber: Kholis & Zaky, 2025)

Tahapan pengembangan:

1. **Pengumpulan Kebutuhan:** Dua alur paralel – (a) *User-Centric*: analisis survei 115 responden; (b) *Data-Centric*: scraping 720 lowongan dari Glints.
2. **Perancangan Desain:** Kebutuhan diterjemahkan menjadi arsitektur *full-stack*: Next.js 14 (frontend), FastAPI (backend), PostgreSQL 16 + pgvector (database), Ollama + Llama 3.1 (LLM), dan *Adaptive Profile System* (personalisasi).
3. **Perancangan Prototipe:** Implementasi komponen inti: FastAPI + RAG pipeline, PostgreSQL + pgvector, Ollama (Llama 3.1 + nomic-embed), Next.js 14, dan Clerk JWT.
4. **Uji Coba & Evaluasi Formatif:** *Black-box testing* internal oleh peneliti untuk memverifikasi fungsionalitas dan memperbaiki *bug* kritikal.

Kode Logika Web Scraping Glints

Berikut adalah kode Python inti dari skrip scraping Glints yang digunakan untuk mengumpulkan data 720 lowongan pekerjaan. Konfigurasi *selenium-stealth* diperlukan untuk mengelabui deteksi bot pada SPA berbasis React:

```

1 import time, json, csv
2 from selenium import webdriver
3 from selenium.webdriver.chrome.options import Options
4 from selenium.webdriver.common.by import By
5 from selenium_stealth import stealth
6
7 def create_stealth_driver():
8     """Inisialisasi Chrome WebDriver dengan konfigurasi
       anti-deteksi."""
  
```



```
9     options = Options()
10     options.add_argument("--headless")
11     options.add_argument("--no-sandbox")
12     options.add_argument("--disable-dev-shm-usage")
13     driver = webdriver.Chrome(options=options)
14     # Konfigurasi stealth: sembunyikan tanda-tanda
       otomasi WebDriver
15     stealth(driver,
16             languages=["id-ID", "id"],
17             vendor="Google Inc.",
18             platform="Win32",
19             webgl_vendor="Intel Inc.",
20             fix_hairline=True,
21     )
22     return driver
23
24 def scrape_glints_cluster(cluster_name, search_query,
25 target_count=90):
26     """Scraping lowongan untuk satu klaster karier dari
       Glints."""
27     driver = create_stealth_driver()
28     jobs = []
29     page = 1
30     base_url = f"https://glints.com/id/opportunities/jobs"
       /explore"
31
32     while len(jobs) < target_count:
33         url = f"{base_url}?keyword={search_query}&page={page}"
34         driver.get(url)
35         time.sleep(3) # Tunggu SPA render (JavaScript)
36
37         # Ambil kartu lowongan setelah JS render selesai
38         job_cards = driver.find_elements(By.CSS_SELECTOR,
39             "[data-cy='job-card']")
40
41         if not job_cards:
42             break # Tidak ada lowongan tersisa
43
44         for card in job_cards:
45             try:
```

```

43         job = {
44             'cluster': cluster_name,
45             'title': card.find_element(By.
46                 CSS_SELECTOR, "h2").text,
47             'company': card.find_element(By.
48                 CSS_SELECTOR, "[data-cy='company-
49                 name']").text,
50             'location': card.find_element(By.
51                 CSS_SELECTOR, "[data-cy='job-
52                 location']").text,
53             'skills': [s.text for s in card.
54                 find_elements(By.CSS_SELECTOR, "[
55                 data-cy='job-skill']").text],
56             'salary': card.find_element(By.
57                 CSS_SELECTOR, "[data-cy='salary-
58                 range']").text,
59         }
60         jobs.append(job)
61     except Exception:
62         continue # Skip kartu yang tidak lengkap
63
64     if len(jobs) >= target_count:
65         break
66     page += 1
67
68     driver.quit()
69     return jobs[:target_count] # Pastikan tepat 90 per
70     klaster
71
72 # Eksekusi untuk semua 8 klaster
73 CLUSTERS = {
74     'Teknologi & Perangkat Lunak': 'software engineer',
75     'Analisis Data': 'data analyst',
76     'Desain & Kreatif': 'UI UX designer',
77     'Pemasaran Digital': 'digital marketing',
78     'Bisnis & Administrasi': 'business analyst',
79     'Sales & Customer Service': 'sales customer
80     service',
81     'Finance & Accounting': 'finance accounting',
82     'Education & Training': 'education trainer',

```

```

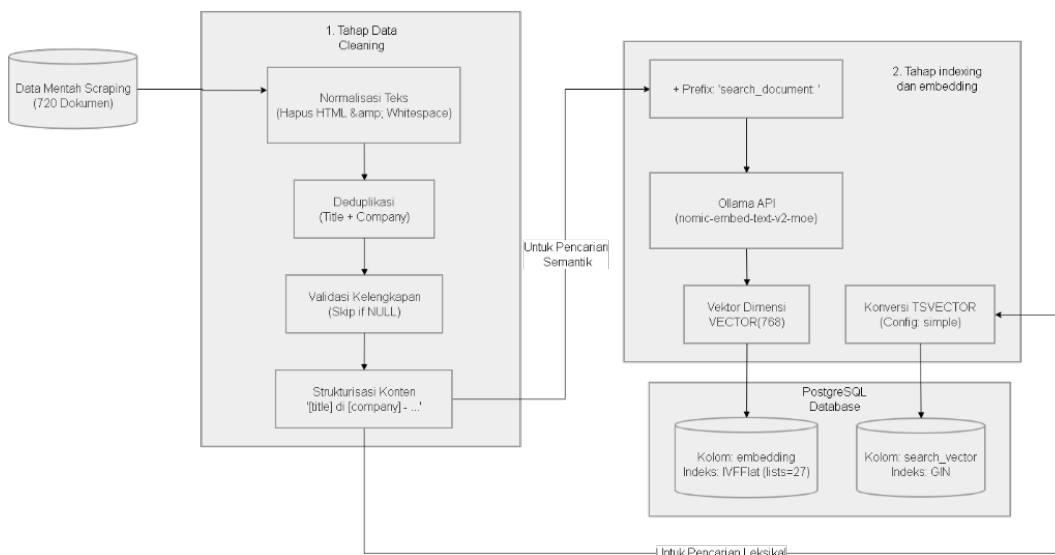
72 }
73
74 all_jobs = []
75 for cluster, query in CLUSTERS.items():
76     print(f"Scraping: {cluster}...")
77     cluster_jobs = scrape_glints_cluster(cluster, query,
78         target_count=90)
79     all_jobs.extend(cluster_jobs)
80     print(f"    -> {len(cluster_jobs)} lowongan terkumpul")
81
82 print(f"Total: {len(all_jobs)} lowongan dari {len(
83     CLUSTERS)} klaster")
84
85 # Output: Total: 720 lowongan dari 8 klaster

```

Listing 2.1: Logika inti scraping Glints dengan Selenium Stealth

Pipeline Pembersihan dan Embedding Data

Setelah proses scraping menghasilkan 720 dokumen mentah, dua tahap pra-pemrosesan dilakukan sebelum data siap digunakan oleh sistem RAG. Alur arsitektur pipeline pembersihan dan *indexing* data dapat dilihat pada Gambar ??.



Gambar 2.3: Arsitektur Pipeline Pembersihan Data dan Indexing Vektor (ETL)

Tahap 1 — Pembersihan Data (*Data Cleaning*): Setiap dokumen mentah dibersihkan menggunakan pipeline otomatis:

- **Normalisasi teks:** Menghapus karakter HTML (`<br>`, `&`), whitespace berlebih, dan karakter non-printable.
- **Deduplikasi:** Mendeteksi dan menghapus lowongan duplikat berdasarkan kombinasi (title + company). Dari 720 hasil scraping, tidak ditemukan duplikat karena setiap klaster menggunakan keyword pencarian yang berbeda.
- **Validasi kelengkapan:** Memastikan setiap dokumen memiliki minimal field: title, company, dan skills. Dokumen dengan field kosong di-*skip*.
- **Strukturisasi konten:** Setiap dokumen diformat menjadi teks gabungan yang mengombinasikan seluruh field: “[title] di [company] - [location]. Skills: [skills].” Format ini memastikan informasi penting tertangkap dalam satu blok teks untuk proses embedding.

Tahap 2 — Generasi Embedding dan Indexing: Setiap dokumen yang telah dibersihkan dikonversi menjadi vektor 768 dimensi menggunakan model `nomic-embed-text-v2-moe` via Ollama API:

$$\mathbf{v}_d = f_{\text{embed}}(\text{"search_document: " || content}_d) \in \mathbb{R}^{768} \quad (2.1)$$

Prefix `search_document:` membedakan dokumen dari query (yang menggunakan `search_query:`), memungkinkan model menghasilkan representasi yang dioptimalkan untuk masing-masing peran.

Vektor embedding disimpan ke kolom `knowledge_base.embedding` bertipe `VECTOR(768)`, dan diindeks menggunakan algoritma **IVFFlat** dengan parameter `lists = 27` (jumlah cluster K-Means = $\sqrt{720} \approx 26.8$, dibulatkan ke atas). Parameter ini mengikuti rekomendasi pgvector official documentation.

Secara paralel, kolom `search_vector` bertipe `TSVECTOR` di-generate dari teks konten menggunakan konfigurasi `simple` (tanpa stemming bahasa tertentu, karena konten mengandung campuran bahasa Indonesia dan Inggris). Indeks GIN dibuat pada kolom ini untuk pencarian leksikal cepat.

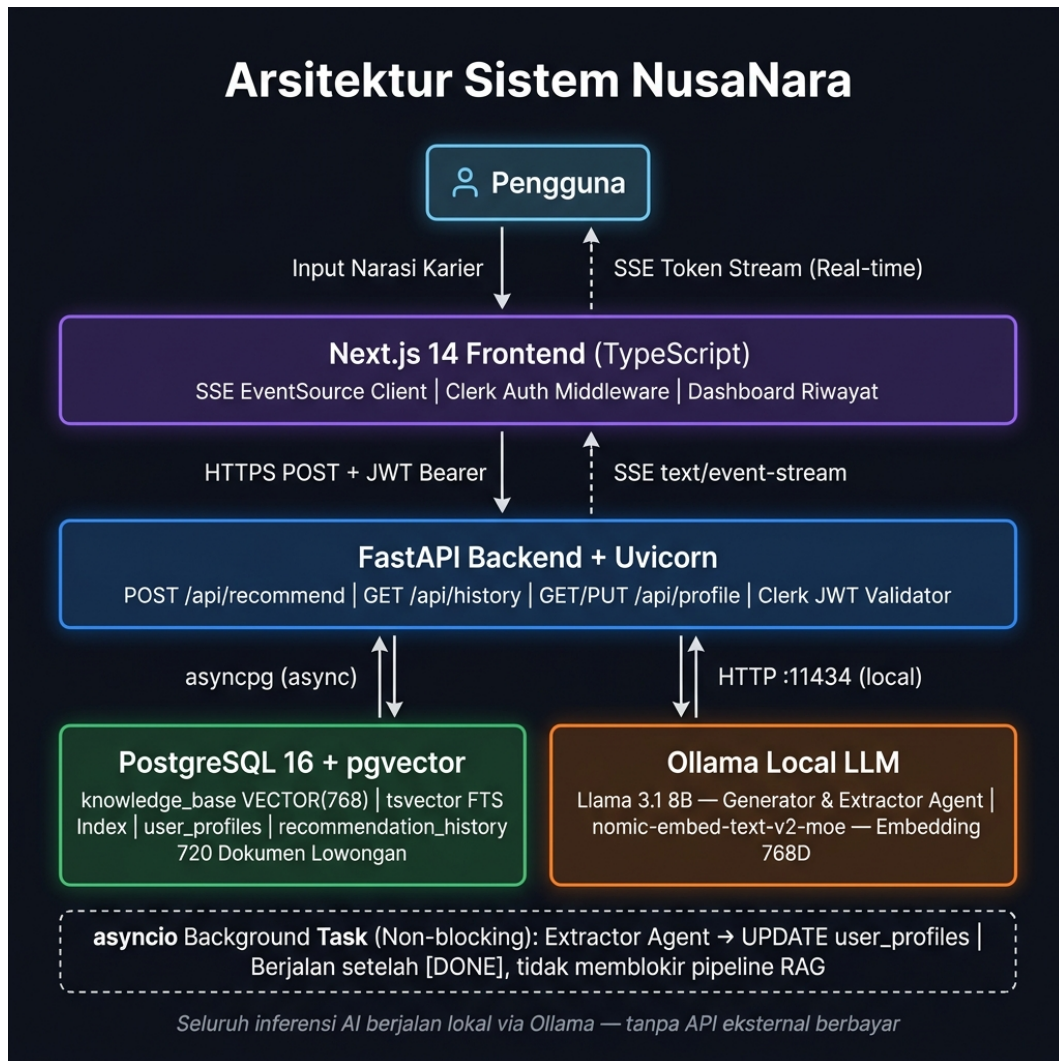
2.4.4 Arsitektur Sistem

Arsitektur sistem NusaNara mengadopsi pendekatan **Unified Database Architecture** yang menempatkan PostgreSQL 16 sebagai satu-satunya sistem penyimpanan data. Berbeda dengan pendekatan *multi-database* yang lazim digunakan dalam sistem RAG konvensional — di mana penyimpanan vektor (misalnya ChromaDB, Pinecone, atau Weaviate) dipisahkan dari penyimpanan relasional — arsitektur ini memanfaatkan ekstensi **pgvector** untuk menginte-

grasikan kapabilitas penyimpanan dan pencarian vektor secara *native* di dalam PostgreSQL.

Keuntungan utama pendekatan *Unified Database* ini adalah:

- **Konsistensi transaksional (ACID):** Seluruh operasi — menyimpan dokumen, vektor embedding, profil pengguna, dan riwayat rekomendasi — dijamin konsisten dalam satu transaksi database. Tidak ada risiko *data drift* antar sistem.
- **Eliminasi sinkronisasi:** Pendekatan multi-database memerlukan mekanisme sinkronisasi yang menambah kompleksitas operasional dan titik kegagalan.
- **Simplifikasi deployment:** Hanya satu proses database yang perlu di-*manage*, di-*backup*, dan di-*monitor* di VPS.
- **Hybrid Search dalam satu query:** Pencarian semantik (pgvector) dan pencarian leksikal (tsvector) dapat digabungkan dalam satu *query plan* PostgreSQL tanpa *network hop* antar layanan.



Gambar 2.4: Arsitektur Sistem NusaNara

Sebagaimana diilustrasikan pada Gambar ??, sistem terdiri dari empat lapisan yang terintegrasi:

Lapisan 1: Antarmuka Pengguna (Next.js 14)

Lapisan antarmuka dibangun menggunakan Next.js 14 dengan App Router dan TypeScript. Komponen utama:

- **fetch-event-source API:** Komponen React yang membuka koneksi SSE (*Server-Sent Events*) ke endpoint `POST /api/recommend`. SSE dipilih (bukan WebSocket) karena alur komunikasi bersifat *unidirectional* — server mengirimkan token secara bertahap ke klien tanpa memerlukan respons balik selama proses streaming berlangsung. Protokol SSE juga lebih sederhana untuk di-*proxy* melalui Nginx dibandingkan WebSocket

yang memerlukan *connection upgrade*.

- **Clerk Middleware:** Setiap halaman dilindungi oleh *middleware* yang memvalidasi sesi Clerk. Token JWT (RS256) disisipkan ke header **Authorization: Bearer <token>** pada setiap permintaan API ke backend. Algoritma RS256 (asymmetric) dipilih agar backend dapat memverifikasi token menggunakan *public key* tanpa memerlukan *shared secret*.
- **Halaman Riwayat:** Menampilkan seluruh sesi rekomendasi sebelumnya, memungkinkan pengguna melacak perkembangan eksplorasi karier dari waktu ke waktu.

Lapisan 2: Logika Bisnis (FastAPI Backend)

Seluruh pipeline RAG dieksekusi pada lapisan ini. FastAPI dipilih karena dukungan *native* terhadap **async/await** (ASGI via Uvicorn), yang krusial untuk: (1) koneksi database **asyncpg** yang sepenuhnya asinkron memerlukan *event loop*, dan (2) streaming respons LLM via **StreamingResponse** memerlukan generator asinkron (**async def generate()**) yang menghasilkan token satu per satu. Pipeline terdiri dari 8 tahap yang dijelaskan pada Sub-bab ??.

Lapisan 3: Penyimpanan Data (PostgreSQL 16 + pgvector)

PostgreSQL 16 menyimpan seluruh data dalam tiga tabel utama:

- **knowledge_base:** 720 dokumen lowongan dengan kolom **embedding** bertipe **VECTOR(768)** yang diindeks menggunakan algoritma **IVFFlat** (*Inverted File with Flat quantization*). IVFFlat membagi ruang vektor menjadi *cluster* menggunakan K-Means, sehingga pencarian ANN (*Approximate Nearest Neighbor*) hanya perlu memeriksa *cluster* terdekat — bukan seluruh 720 vektor. Kolom **search_vector** bertipe **TSVECTOR** diindeks dengan GIN (*Generalized Inverted Index*) untuk pencarian leksikal cepat.
- **user_profiles:** Profil adaptif yang diperbarui akumulatif oleh *Extractor Agent*. Kolom **identified_skills** dan **career_interests** bertipe **TEXT[]** (array PostgreSQL).
- **recommendation_history:** Log lengkap setiap sesi rekomendasi, mencakup narasi input, profil saat itu, dokumen yang di-retrieve, dan output LLM.

Lapisan 4: Inferensi Lokal (Ollama)

Ollama menjalankan dua model AI secara lokal di VPS:

- **Llama 3.1 8B (4-bit Quantized)**: Digunakan untuk dua peran — (a) *Main RAG Agent*: menghasilkan rekomendasi karier 5 seksi terstruktur berdasarkan konteks Top-3 dokumen; dan (b) *Extractor Agent*: mengekstrak skill, minat, dan ringkasan dari narasi pengguna ke format JSON. Pemilihan model versi 8B dengan teknik kuantisasi 4-bit (Q4_K_M) merupakan keputusan teknis agar sesuai dengan keterbatasan memori VPS (16 GB RAM). Ollama secara efisien me-load bobot model ~ 5 GB satu kali ke dalam memori. Ketika dua agen berjalan bergantian, keduanya berbagi *instance* model yang sama tanpa menggandakan konsumsi memori, menyisakan cukup memori untuk PostgreSQL (~ 2 GB *shared buffers*) dan proses sistem operasi.
- **nomic-embed-text-v2-moe (768 dimensi)**: Model embedding berbasis *Mixture of Experts* (MoE) yang mengaktifkan subset dari parameter untuk setiap input, menghasilkan representasi vektor yang lebih efisien. Penggunaan prefix `search_query`: dan `search_document`: telah dijelaskan pada tahap *embedding* pipeline ETL (Sub-bab sebelumnya).

2.4.5 Analisis Kompleksitas Komputasional Pipeline RAG

Analisis kompleksitas waktu (O -notation) penting untuk memahami skala performa sistem saat jumlah dokumen atau pengguna meningkat.

Tabel 2.2: Analisis Kompleksitas Komputasional Per Tahap Pipeline RAG

Tahap	Operasi	Keterangan	Big-O
1	Query Expansion	Pencarian dalam dict statis 8 klaster	$O(1)$
2	Embedding	Inferensi model nomic-embed (konstan)	$O(L)$
3	Semantic Search	IVFFlat ANN (bukan brute-force)	$O(\sqrt{n})$
4	Lexical Search	Indeks GIN inverted	$O(\log n + m)$
5	RRF Fusion	Merge dua list 20 item	$O(20)$
6	Re-ranking	Hitung skor 10 kandidat	$O(10 \cdot S_u)$
7	LLM Generation	Inferensi autoregressive	$O(L_{out})$
8	SSE Streaming	Token-by-token transmission	$O(L_{out})$

Justifikasi Teoretis dan Penjelasan Notasi:

- **Tahap 1 ($O(1)$):** Pencarian kamus (*Hash Map*) di Python secara teoretis memiliki kompleksitas waktu rata-rata konstan $O(1)$?.
- **Tahap 3 ($O(\sqrt{n})$):** Algoritma pencarian vektor *Inverted File* (IVFFlat) mempartisi ruang ke dalam $\sqrt{n}$ sel Voronoi, mengurangi iterasi pencarian dari linear menjadi akar kuadrat ?.
- **Tahap 4 ($O(\log n + m)$):** Indeks teks GIN menggunakan struktur *B-Tree* yang beroperasi dalam waktu logaritmik $O(\log n)$, ditambah waktu pengambilan m dokumen yang cocok ?.
- **Tahap 7 ($O(L_{out})$):** Inferensi LLM bersifat *autoregressive*, di mana total waktu generasi berbanding lurus linier dengan panjang token keluaran (L_{out}) ?.
- n = jumlah dokumen dalam `knowledge_base` (720 dalam implementasi ini).
- L = panjang token input query (rata-rata 30–100 token).
- L_{out} = panjang token output LLM (rata-rata 500–800 token untuk 5 seksi).
- m = jumlah dokumen yang cocok dalam FTS (biasanya $m \ll n$).
- $|S_u|$ = jumlah skill dalam profil pengguna.

Tahap paling lambat adalah Tahap 7 (LLM Generation) dengan $O(L_{out})$ yang bergantung pada panjang respons. Pada Llama 3.1 8B (4-bit Quantized) di CPU VPS, throughput generasi adalah $\sim 20\text{--}30$ token/detik,

sehingga respons 600 token memerlukan $\sim 20\text{--}30$ detik waktu total generasi. Namun, berkat SSE streaming (Tahap 8), pengguna mulai menerima token pada detik ke-2.4 (*Time to First Byte*) dan membacanya secara paralel dengan generasi berikutnya — persepsi latensi jauh lebih rendah dari total waktu generasi.

Tahap paling cepat adalah Tahap 1 ($O(1)$) dan Tahap 5 ($O(20) = O(1)$) yang tidak bergantung pada jumlah dokumen.

Skalabilitas: Jika n meningkat dari 720 ke 72.000 ($100\times$), kompleksitas Tahap 3 meningkat dari $O(\sqrt{720}) \approx O(27)$ ke $O(\sqrt{72000}) \approx O(268)$ — hanya $10\times$ lebih lambat untuk $100\times$ data. Ini menunjukkan bahwa arsitektur IVFFlat sangat skalabel untuk pertumbuhan basis pengetahuan.

2.4.6 Detail Pipeline RAG (8 Tahap)

Setiap permintaan rekomendasi melewati 8 tahap pipeline sekuensial. Alur lengkap dirangkum dalam persamaan berikut:

$$\text{Narasi} \xrightarrow{1} \text{Query} \xrightarrow{2} \mathbf{v}_{768} \xrightarrow{3} R_s \xrightarrow{4} R_l \xrightarrow{5} \text{Top-10} \xrightarrow{6} \text{Top-3} \xrightarrow{7} \text{LLM} \xrightarrow{8} \text{Client} \quad (2.2)$$

Tahap 1 — Query Expansion. Narasi mentah pengguna seringkali mengandung bahasa informal, singkatan, dan istilah tidak baku yang tidak akan cocok secara leksikal dengan konten dokumen. Tahap *Query Expansion* memperkaya query dengan kata kunci karier standar berdasarkan pemetaan ke 8 kluster yang telah didefinisikan:

```

1 CLUSTER_KEYWORDS = {
2     "teknologi": ["software engineer", "programmer", "
3                   developer",
4                   "backend", "frontend", "fullstack", "IT
5                   "],
6     "data": ["data analyst", "data scientist", "SQL", "
7             Python",
8             "machine learning", "analytics", "BI"],
9     "desain": ["UI UX", "graphic designer", "creative", "
10              Figma",
11              "illustrator", "visual"],
12     "marketing": ["digital marketing", "content creator",
13                  "SEO",

```

```

9         "social media", "copywriter"],
10     "bisnis": ["business analyst", "project manager", "
        konsultan",
11               "administrasi", "operasional"],
12     "sales": ["sales", "customer service", "account
        executive",
13               "CRM", "relasi pelanggan"],
14     "finance": ["akuntan", "finance", "accounting", "
        audit",
15                 "pajak", "controller"],
16     "education": ["guru", "trainer", "pengajar", "
        instruktur",
17                  "kurikulum", "e-learning"],
18 }
19
20 def expand_query(narrative: str) -> str:
21     """Menambahkan keyword domain ke narasi pengguna."""
22     narrative_lower = narrative.lower()
23     extra_keywords = []
24     for cluster, keywords in CLUSTER_KEYWORDS.items():
25         # Cek apakah narasi mengandung sinyal klaster
26         if any(kw.lower() in narrative_lower for kw in
27               keywords):
28             extra_keywords.extend(keywords[:3]) # Ambil
29             3 keyword utama
30     if extra_keywords:
31         return f"{narrative} {' '.join(set(extra_keywords
32       ))}"
33     return narrative # Kembalikan narasi asli jika tidak
34     ada match

```

Listing 2.2: Implementasi Query Expansion berbasis Klaster

Contoh kerja Query Expansion:

Input narasi: “saya mahasiswa informatika, suka ngoding Python dan bikin dashboard”

Deteksi sinyal: “ngoding” → klaster “teknologi”; “Python” → klaster “data”

Query diperluas: “saya mahasiswa informatika, suka ngoding Python dan bikin dashboard software engineer programmer developer data analyst SQL analytics”

Penambahan kata kunci ini meningkatkan kemungkinan kecocokan leksikal (Tahap 4) dengan dokumen yang menggunakan terminologi formal, tanpa menghilangkan makna asli narasi untuk pencarian semantik (Tahap 3).

Tahap 2 — Embedding (Vektorisasi). Query yang telah diperluas di-konversi menjadi representasi vektor 768 dimensi:

$$\mathbf{v}_q = f_{embed}(\text{"search_query: " || } q_{expanded}) \in \mathbb{R}^{768} \quad (2.3)$$

Penjelasan variabel:

- $\mathbf{v}_q$ = vektor representasi query berdimensi 768. Setiap dimensi merepresentasikan satu aspek semantik dari teks input.
- f_{embed} = fungsi embedding model nomic-embed-text-v2-moe yang memetakan teks ke ruang vektor 768-dimensi.
- $\|$ = operasi konkatenasi (penggabungan) string.
- $q_{expanded}$ = teks query setelah tahap ekspansi (Tahap 1).
- Prefix `search_query:` adalah instruksi wajib bagi model nomic untuk membedakan antara query pencarian dan dokumen yang akan diindeks.

Tahap 3 — Pencarian Semantik (pgvector Cosine Similarity). Vektor query $\mathbf{v}_q$ dibandingkan dengan subset dokumen kandidat dalam indeks IVFFlat menggunakan metrik *cosine similarity*. Operator `<=>` pada pgvector menghitung *cosine distance*. Untuk mendapatkan *cosine similarity*, sistem mengonversinya dengan rumus $(1 - \text{distance})$, yang berakar pada formula dasar berikut:

$$\text{CosineSim}(\mathbf{v}_q, \mathbf{v}_d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \times \|\mathbf{v}_d\|} \quad (2.4)$$

Penjelasan variabel secara rinci:

- $\mathbf{v}_q \cdot \mathbf{v}_d$ = *dot product* (perkalian titik), yaitu $\sum_{i=1}^{768} v_{q,i} \times v_{d,i}$. Operasi ini mengalikan setiap elemen yang bersesuaian dari kedua vektor lalu menjumlahkannya. Hasilnya adalah skalar yang mengukur “seberapa searah” kedua vektor dalam ruang 768-dimensi.
- $\|\mathbf{v}_q\|$ = norma Euclidean (panjang vektor), yaitu $\sqrt{\sum_{i=1}^{768} v_{q,i}^2}$. Pembagian oleh norma berfungsi menormalisasi panjang agar hanya “arah” yang diperhitungkan.

- Nilai $CosineSim = 1.0$ berarti kedua vektor identik (sangat mirip); nilai 0.0 berarti tidak ada kesamaan sama sekali. Operator $pgvector <=>$ mengembalikan $distance$, sehingga $similarity = 1 - distance$.

Tahap ini menghasilkan himpunan R_s berisi 20 dokumen teratas beserta peringkatnya $rank_s(d)$.

Tahap 4 — Pencarian Leksikal (PostgreSQL Full-Text Search). Secara paralel, pencarian leksikal dilakukan menggunakan indeks `tsvector` dan fungsi `ts_rank` bawaan PostgreSQL. Pencarian ini mencocokkan kata-kata secara eksak (bukan berdasarkan makna), sehingga menangkap kasus di mana kata kunci spesifik (nama teknologi, nama posisi) muncul secara literal dalam dokumen namun tidak tertangkap oleh pencarian semantik. Menghasilkan himpunan R_l (20 teratas) beserta $rank_l(d)$.

Tahap 5 — Reciprocal Rank Fusion (RRF). Hasil pencarian semantik (R_s) dan leksikal (R_l) digabungkan menggunakan algoritma *Reciprocal Rank Fusion* (RRF):

$$RRF(d) = \frac{1}{k + rank_s(d)} + \frac{1}{k + rank_l(d)} \quad (2.5)$$

Penjelasan variabel secara rinci:

- $RRF(d)$ = skor fusi untuk dokumen d . Semakin tinggi nilainya, semakin relevan dokumen tersebut menurut gabungan kedua metode pencarian.
- $k = 60$ = konstanta smoothing, ditetapkan sesuai standar literatur RRF. Fungsi k : mencegah dominasi berlebihan dari peringkat 1. Tanpa k , peringkat 1 mendapat skor $\frac{1}{1} = 1.0$ yang terlalu dominan dibanding peringkat 2 ($\frac{1}{2} = 0.5$, selisih 100%). Dengan $k = 60$: peringkat 1 mendapat $\frac{1}{61} \approx 0.0164$ dan peringkat 2 mendapat $\frac{1}{62} \approx 0.0161$ (selisih 1.8%) — perbedaan yang lebih proporsional.
- $rank_s(d)$ = peringkat dokumen d dalam hasil pencarian semantik (1 = paling mirip). Jika dokumen tidak muncul, kontribusinya = 0.
- $rank_l(d)$ = peringkat dokumen d dalam hasil pencarian leksikal (1 = paling cocok kata kunci).

Contoh perhitungan RRF: Dokumen “Data Analyst Tokopedia”

berada di peringkat 2 (semantik) dan peringkat 5 (leksikal):

$$RRF = \frac{1}{60+2} + \frac{1}{60+5} = \frac{1}{62} + \frac{1}{65} = 0.01613 + 0.01538 = 0.03151$$

Dokumen “Marketing Analyst” yang hanya muncul di semantik peringkat 1: $RRF = \frac{1}{61} = 0.01639$. Meskipun “Marketing Analyst” lebih tinggi di semantik, “Data Analyst” mendapat skor RRF lebih tinggi karena muncul di *kedua* pencarian — inilah kekuatan *Hybrid Search*.

Tahap 6 — Re-ranking Heuristik. Top-10 kandidat dari RRF di-*re-rank* menggunakan formula heuristik dua komponen:

$$score_{final}(d) = \alpha \times CosineSim(\mathbf{v}_q, \mathbf{v}_d) + \beta \times SkillBonus(d) \quad (2.6)$$

dengan $\alpha = 0.70$ dan $\beta = 0.30$.

$$SkillBonus(d) = \min(|skills_{user} \cap skills_d| \times 0.25, 1.0) \quad (2.7)$$

Penjelasan variabel secara rinci:

- $score_{final}(d)$ = skor akhir dokumen d yang menentukan urutan Top-3 dokumen yang akan diinjeksikan ke prompt LLM.
- $\alpha = 0.70$ = bobot kemiripan semantik (indikator utama relevansi).
- $\beta = 0.30$ = bobot bonus keterampilan (personalisasi dari profil adaptif).
- $skills_{user}$ = himpunan keterampilan pengguna yang telah diekstrak dari sesi-sesi sebelumnya oleh *Extractor Agent*, tersimpan di `user_profiles.identified_skills`.
- $skills_d$ = himpunan keterampilan yang dipersyaratkan lowongan d , tersimpan di kolom `knowledge_base.skills`.
- $|skills_{user} \cap skills_d|$ = jumlah keterampilan yang *overlap* (irisan). Contoh: pengguna memiliki {Python, SQL, Excel} dan dokumen memerlukan {Python, SQL, Tableau}, maka irisan = {Python, SQL} = 2 item.
- 0.25 = konstanta penambahan bonus per satu skill yang cocok (*step size*).
- 1.0 = batas maksimum (*cap*) dari fungsi bonus sebelum dikalikan bobot β .

Contoh perhitungan: $skills_{user} = \{\text{Python, SQL, Excel}\}$:

- Dok A (Data Analyst): $CosineSim = 0.82$, $skills_d = \{\text{Python, SQL, Tableau}\}$, irisan = 2 item. $SkillBonus = \min(2 \times 0.25, 1.0) = 0.500$.
 $score = 0.70 \times 0.82 + 0.30 \times 0.500 = 0.574 + 0.150 = \mathbf{0.724}$
- Dok B (Business Analyst): $CosineSim = 0.85$, $skills_d = \{\text{PowerBI, Jira}\}$, irisan = 0 item. $SkillBonus = \min(0 \times 0.25, 1.0) = 0.000$. $score = 0.70 \times 0.85 + 0 = \mathbf{0.595}$

Meskipun Dok B lebih mirip secara semantik (0.85 vs 0.82), Dok A menang karena keterampilan pengguna lebih cocok — inilah nilai personalisasi *re-ranking*.

Tahap 7 — Prompt Construction dan LLM Generation. Top-3 dokumen diinjeksikan ke template prompt *Decision Intelligence* bersama narasi pengguna dan profil adaptif. Untuk menghilangkan ambiguitas, berikut adalah literal *system prompt* yang digunakan (diimplementasikan dalam `prompt_builder.py`):

```

1 SYSTEM_PROMPT = """Kamu adalah NusaNara AI Career Advisor
   yang ahli dan empatik.
2 Tugasmu adalah memberikan saran karier 5-seksi
   berdasarkan profil pengguna dan
3 dokumen lowongan yang tersedia.
4
5 ATURAN KETAT (FAITHFULNESS CONSTRAINT):
6 1. HANYA gunakan informasi posisi dan perusahaan dari
   dokumen konteks yang diberikan!
7 2. DILARANG KERAS mengarang, berhalusinasi, atau
   merekomendasikan pekerjaan yang tidak ada di dokumen
   konteks.
8 3. Setiap rekomendasi posisi WAJIB mencantumkan SKOR RAG
   dan SKILL OVERLAP yang sudah dihitung oleh sistem.
9
10 [PROFIL ADAPTIF PENGGUNA]
11 {user_profile_json}
12
13 [NARASI PENGGUNA SAAT INI]
14 {user_narrative}
15
16 [DOKUMEN KONTEKS DARI DATABASE (TOP-3)]
17 {top3_documents_json}

```

```

18
19 Berikan respons terstruktur dalam 5 seksi:
20 1. Analisis Profil & Trade-off
21 2. Rekomendasi Keputusan (wajib cantumkan Skor RAG &
    Skill Overlap)
22 3. Analisis Skill Gap
23 4. Rencana Pengembangan Skill (Roadmap 7 hari)
24 5. Insight Pasar Kerja Lokal
25 ""

```

Listing 2.3: Literal System Prompt Decision Intelligence

LLM (Llama 3.1 8B (4-bit Quantized)) memproses prompt di atas dengan parameter `temperature=0.7` untuk menghasilkan narasi yang dinamis namun tetap terkekang oleh aturan *faithfulness constraint* — mencegah model memberikan saran yang mengawang-awang di luar data lowongan nyata dari database.

Tahap 8 — SSE Streaming ke Client. *Server-Sent Events* (SSE) adalah protokol HTTP standar yang memungkinkan server mengirim data ke client secara *unidirectional* (satu arah: server ke client) tanpa polling berulang. SSE dipilih dibanding WebSocket karena lebih sederhana — tidak memerlukan *handshake* upgrade protokol dan berjalan di atas HTTP/1.1 biasa.

Setiap token yang dihasilkan Llama dikirim sebagai event SSE dengan format baku:

```

1 data: Berdasarkan
2
3 data:  profil
4
5 data:  Anda
6
7 ...
8 data: [DONE]

```

Listing 2.4: Format SSE Event per Token

Penjelasan format SSE:

- Setiap event diawali `data:` dan diakhiri dua baris baru (`\n\n`). Ini adalah standar protokol *Server-Sent Events* (SSE) oleh W3C ?.
- Token biasa langsung dikirim dalam bentuk *plain-text* dan frontend akan melakukan *append* langsung ke UI seiring token berdatangan.

- String [DONE] adalah *flag* khusus yang memberitahu frontend bahwa LLM selesai. Setelah ini, frontend menutup koneksi `fetch-event-source` dan backend memicu background task *Adaptive Profile Update*.

Di sisi frontend (Next.js 14), event diterima menggunakan pustaka `@microsoft/fetch-event-source` (karena `EventSource` bawaan peramban tidak mendukung metode POST dan `*header*` otorisasi):

```

1 import { fetchEventSource } from '@microsoft/fetch-event-
  source';
2
3 await fetchEventSource('/api/recommend', {
4   method: 'POST',
5   headers: {
6     'Authorization': `Bearer ${token}`,
7     'Content-Type': 'application/json'
8   },
9   body: JSON.stringify({ narrative }),
10  onmessage(event) {
11    if (event.data === '[DONE]') return;
12    setResponse(prev => prev + event.data); // React
      state update
13  }
14 });

```

Listing 2.5: Konsumsi SSE di Frontend Next.js

Dekomposisi TTFB 2,4 detik:

Tabel 2.3: Dekomposisi Latensi Pipeline RAG (TTFB = 2.400 ms)

Komponen	Estimasi (ms)	Persentase
JWT Verification (Clerk SDK)	50	2,1%
Query Expansion (dict lookup)	5	0,2%
Embedding (nomic-embed via Ollama)	450	18,8%
Hybrid Search (pgvector + FTS)	200	8,3%
Re-ranking (Python heuristic)	45	1,9%
Prompt Construction	10	0,4%
LLM Cold Start (Llama 3.1 8B (4-bit Quantized))	1640	68,3%
Total TTFB	2400	100%

Komponen terbesar adalah **LLM Cold Start** (68,3%) — waktu yang dibutuhkan Llama untuk menghasilkan token *pertama*. Ini merupakan karakteristik inheren model autoregresif dan tidak dapat dikurangi tanpa mengganti model atau menggunakan GPU. Komponen lain (embedding, search, re-ranking) totalnya hanya 710 ms (29,6%).

2.4.7 Alur Paralel: Adaptive Profile System (Slow Path)

Secara terpisah dari pipeline RAG (*Fast Path*), setelah sinyal [DONE] dikirim ke klien, `asyncio.create_task()` memicu *background task* yang menjalankan *Adaptive Profile System*:

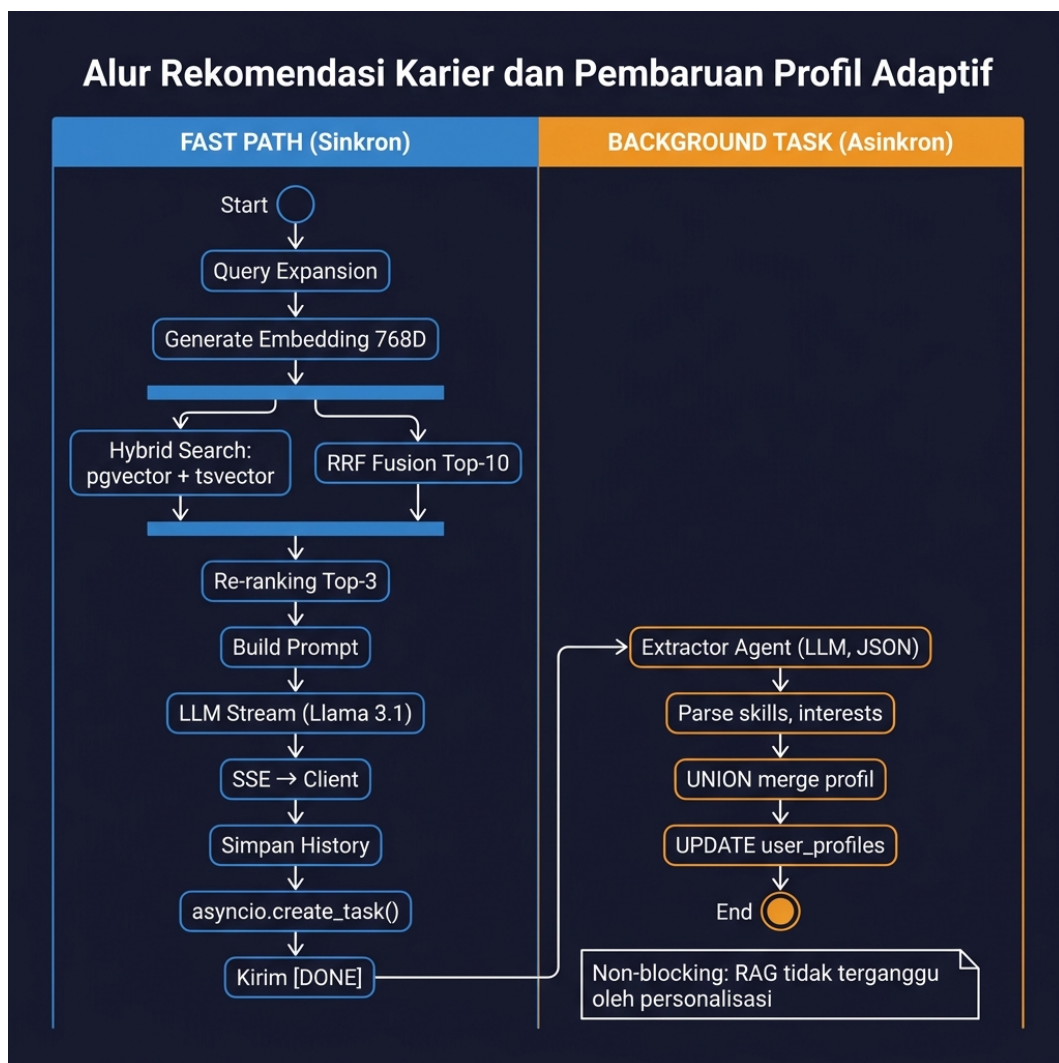
1. **LLM Extractor Agent:** Llama 3.1 8B (4-bit Quantized) mengekstrak skill dan minat dari narasi ke JSON terstruktur. Input tidak terstruktur (bahasa campur, informal) menjadikan pendekatan rule-based (regex, NER) tidak memadai. Untuk menjamin format output, berikut literal *system prompt* yang diinjeksikan secara terprogram:

```
1 EXTRACTOR_PROMPT = """Tugasmu adalah mengekstrak
    keterampilan (skills) dan area minat (interests)
    dari narasi karier yang diberikan.
2
3 ATURAN:
4 1. Ekstrak keterampilan teknis dan soft skill yang
    relevan secara profesional.
5 2. Ekstrak minat atau fokus karier utama.
6 3. WAJIB mengembalikan output HANYA dalam format JSON
    valid sesuai skema berikut, tanpa teks tambahan
    apapun!
7
8 FORMAT JSON YANG DIHARAPKAN:
9 {
10     "skills": ["Python", "SQL", "Public Speaking"],
11     "interests": ["Data Science", "Marketing"]
12 }
13
14 [NARASI PENGGUNA]
15 {user_narrative}
16 """
```

Listing 2.6: Literal System Prompt LLM Extractor Agent

2. **Deterministic Merge:** JSON hasil ekstraksi digabungkan ke profil yang sudah ada di tabel `user_profiles` via operasi UNION set (deterministik, akumulatif). Skill dan minat lama tidak pernah hilang, memastikan profil terus berkembang secara organik seiring berjalannya sesi.

Proses ini **tidak menambah latensi** pada pipeline RAG utama karena dijalankan di *background task* secara asinkron.



Gambar 2.5: Alur Kerja Sistem Rekomendasi dan Pembaruan Profil Adaptif NusaNara

2.4.8 Teknik Analisis Data

Seluruh analisis data dilakukan secara kuantitatif menggunakan pendekatan evaluasi empat lapis yang selaras dengan fase *Demonstration* dan *Evaluation* dalam kerangka DSR ?. Setiap lapis mengukur aspek yang berbeda dari artefak sehingga evaluasi bersifat komprehensif dan multi-perspektif.

Lapis 1 – Evaluasi Teknis Retrieval (RAG Evaluation) Lapis ini mengukur kualitas komponen *retrieval* secara kuantitatif — seberapa baik sistem menemukan dokumen yang benar-benar relevan dari 720 dokumen yang tersedia. Evaluasi menggunakan dua metrik standar *Information Retrieval*:

(a) **Precision at k (P@k)** mengukur proporsi dokumen relevan di dalam Top- k hasil retrieval:

$$P@k = \frac{|\{\text{dokumen relevan}\} \cap \{\text{Top-}k \text{ hasil}\}|}{k} \quad (2.8)$$

Penjelasan variabel secara rinci:

- k = jumlah dokumen teratas yang diperiksa. Dalam penelitian ini, $k \in \{1, 3, 5\}$. Nilai $k = 1$ mengukur akurasi dokumen pertama (paling kritis karena akan paling memengaruhi output LLM); $k = 3$ sesuai dengan jumlah dokumen yang diinjeksikan ke prompt; $k = 5$ mengukur kualitas kandidat yang lebih luas.
- $\{\text{dokumen relevan}\}$ = himpunan dokumen yang dianggap relevan oleh anotator berdasarkan *ground truth*. Relevansi dianotasi menggunakan skala *graded* 0–3:
 - 0 = tidak relevan sama sekali (domain berbeda)
 - 1 = sedikit relevan (domain sama tapi posisi tidak cocok)
 - 2 = relevan (posisi dan sebagian skill cocok)
 - 3 = sangat relevan (posisi, skill, dan level cocok)

Dalam perhitungan P@k, dokumen dengan skor ≥ 2 dianggap relevan (biner).

- $\{\text{Top-}k \text{ hasil}\}$ = himpunan k dokumen teratas yang dikembalikan oleh sistem setelah tahap *re-ranking*.
- Nilai P@k berkisar 0.0 (tidak ada yang relevan) hingga 1.0 (seluruh Top- k relevan).

Contoh implementasi: Untuk query “fresh graduate data analyst”, sistem mengembalikan Top-3: [*Data Analyst Tokopedia* (skor 3), *Business Analyst* (skor 2), *Marketing Manager* (skor 0)]. Maka $P@3 = \frac{2}{3} \approx 0.667$ karena 2 dari 3 dokumen relevan (skor ≥ 2).

(b) Mean Reciprocal Rank (MRR) mengukur seberapa tinggi dokumen relevan *pertama* muncul dalam hasil retrieval secara rata-rata:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (2.9)$$

Penjelasan variabel secara rinci:

- $|Q|$ = total jumlah query uji. Dalam penelitian ini $|Q| = 20$, mencakup 2–3 query per klaster karier (8 klaster $\times$ 2.5 rata-rata).
- $rank_i$ = peringkat (posisi) dokumen relevan **pertama** dalam hasil query ke- i . MRR hanya mempertimbangkan posisi dokumen relevan pertama, bukan seluruh dokumen relevan.
- Jika dokumen relevan pertama ada di posisi 1: kontribusi = $\frac{1}{1} = 1.0$ (sempurna — sistem langsung menemukan jawaban yang benar).
- Jika ada di posisi 2: kontribusi = $\frac{1}{2} = 0.5$; posisi 3: $\frac{1}{3} \approx 0.333$.
- Jika tidak ada dokumen relevan di Top- k : kontribusi = 0 (kegagalan total untuk query tersebut).

Interpretasi: Nilai MRR = 0.842 mengindikasikan bahwa sistem RAG sangat sering menempatkan dokumen relevan pertama di peringkat teratas (sebagian besar di posisi 1 atau 2). Ini penting karena dalam konteks RAG, posisi pertama mendapat bobot terbesar dalam konteks yang diberikan ke LLM.

Desain Evaluasi: Evaluasi dilakukan dalam dua kondisi (*ablation study*): (1) tanpa *re-ranking* (hanya *Hybrid Search* + RRF), dan (2) dengan *re-ranking* heuristik. Perbandingan ini memungkinkan pengukuran kontribusi spesifik dari komponen *re-ranking* terhadap kualitas retrieval.

Lapis 2 – Pengujian Fungsional Black-Box Memverifikasi bahwa seluruh fitur sistem berjalan sesuai spesifikasi fungsional. Setiap *use case* diuji dengan membandingkan input dengan output yang diharapkan. Pengujian

mencakup 7 skenario: rekomendasi karier, streaming TTFB, riwayat tersimpan, autentikasi JWT, health check, adaptive profile, dan faithfulness. Setiap skenario dinilai dengan status PASS/FAIL secara biner.

Lapis 3 – Evaluasi Kinerja Sistem (Latency) Mengukur kecepatan respons sistem RAG saat berinteraksi dengan pengguna. Terdapat dua metrik utama yang dievaluasi menggunakan alat pemantauan *built-in* pada backend FastAPI:

1. **Database Retrieval Time:** Waktu yang dibutuhkan PostgreSQL (*pgvector* + FTS) untuk mencari dan mengurutkan Top-3 dokumen dari *knowledge base*.
2. **Time-to-First-Byte (TTFB):** Waktu tunda dari saat pengguna mengirimkan kueri hingga LLM Llama 3.1 mulai memancarkan (streaming) huruf pertama dari rekomendasinya ke layar klien.

Lapis 4 – Evaluasi Penerimaan Pengguna (UAT) UAT mengukur persepsi pengguna melalui kuesioner 7 pertanyaan Likert 1–5, mencakup tiga dimensi yang diadaptasi dari *Technology Acceptance Model* (TAM)?: *Usefulness* (kegunaan/relevansi rekomendasi, 3 pertanyaan), *Ease of Use* (kemudahan penggunaan dan kecepatan sistem, 2 pertanyaan), dan *Satisfaction* (kepuasan keseluruhan, 2 pertanyaan). Data dianalisis secara deskriptif menggunakan dua rumus utama:

(a) **Rata-rata per item** menghitung skor rata-rata untuk setiap pertanyaan dari seluruh responden:

$$\bar{x}_i = \frac{\sum_{j=1}^n x_{ij}}{n} \quad (2.10)$$

Penjelasan variabel:

- $\bar{x}_i$ = rata-rata skor untuk pertanyaan ke- i (contoh: $\bar{x}_1$ adalah rata-rata skor Q1 dari seluruh responden).
- x_{ij} = skor Likert (1–5) yang diberikan responden ke- j untuk pertanyaan ke- i . Nilai 1 = Sangat Tidak Setuju, 5 = Sangat Setuju.
- n = jumlah responden (10–15 orang).
- $\sum_{j=1}^n x_{ij}$ = penjumlahan skor seluruh responden untuk pertanyaan ke- i .

(b) **Grand Mean** menghitung rata-rata keseluruhan dari seluruh item:

$$\bar{X} = \frac{\sum_{i=1}^7 \bar{x}_i}{7} \quad (2.11)$$

di mana $\bar{X}$ adalah *grand mean* yang merangkum penerimaan pengguna secara keseluruhan dalam satu angka.

(c) **Percentage of Agreement (PA)** menghitung persentase responden yang memberikan skor positif (≥ 4) untuk setiap item:

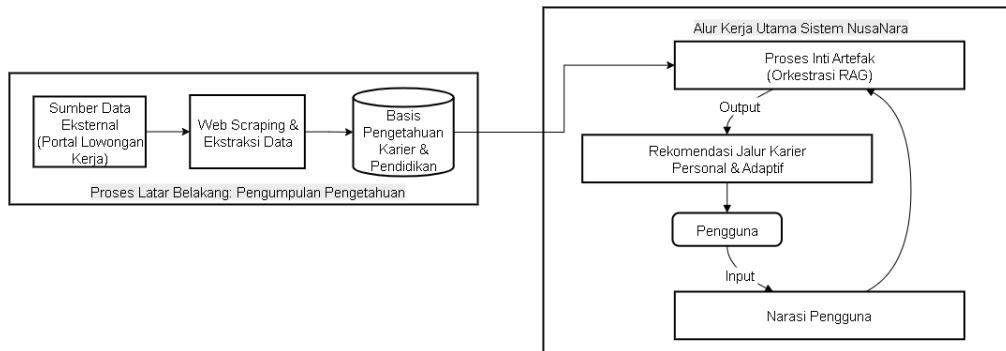
$$PA_i = \frac{|\{j : x_{ij} \geq 4\}|}{n} \times 100\% \quad (2.12)$$

Interpretasi *grand mean*:

Tabel 2.4: Skala Interpretasi Grand Mean UAT

Rentang Grand Mean	Interpretasi
4.21 – 5.00	Sangat Baik / Sangat Diterima
3.41 – 4.20	Baik / Diterima
2.61 – 3.40	Cukup
1.81 – 2.60	Tidak Baik
1.00 – 1.80	Sangat Tidak Baik

Gambar ?? mengilustrasikan alur kerja fundamental artefak NusaNara dengan dua subsistem utama: (1) proses latar belakang pengumpulan pengetahuan via web scraping, dan (2) alur kerja utama sistem yang merespons narasi pengguna secara *real-time*.



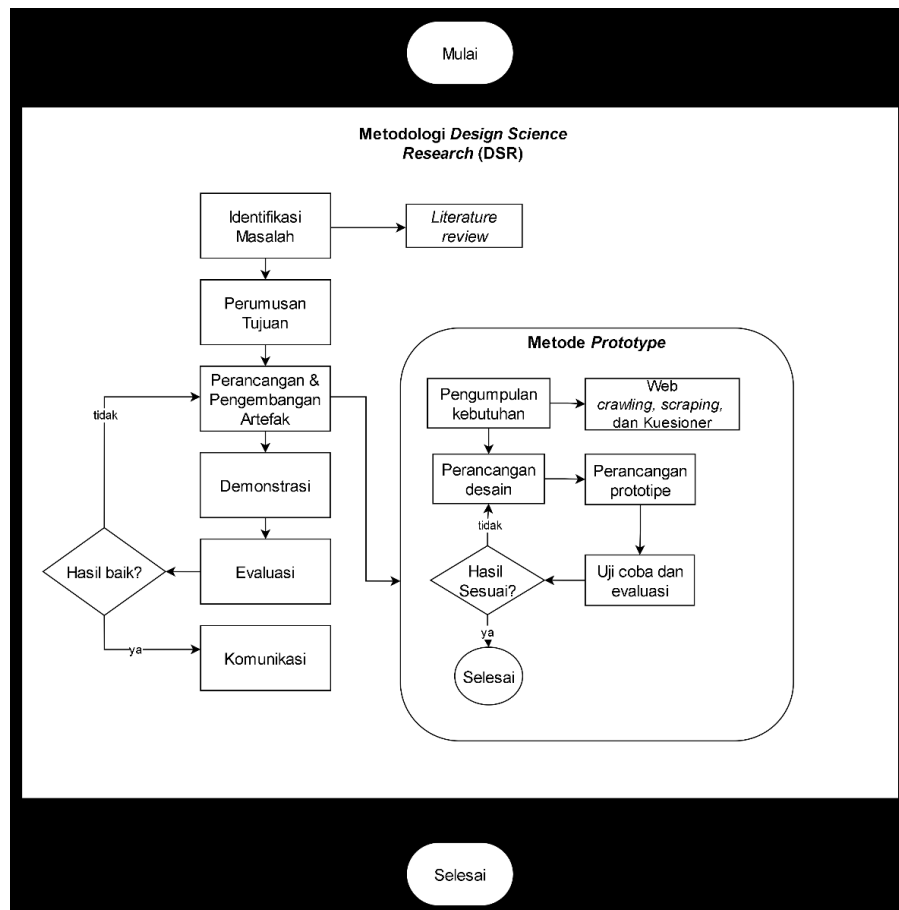
Gambar 2.6: Ilustrasi Alur Kerja Inti Artefak “NusaNara”

Pada sisi kiri Gambar ??, **Proses Latar Belakang** menggambarkan pipeline offline pengumpulan pengetahuan: data dari portal lowongan kerja (Glints) dikumpulkan via web scraping, diproses, dan disimpan ke dalam basis pengetahuan (PostgreSQL + pgvector). Pipeline ini merupakan prasyarat satu kali (*one-time setup*) yang menghasilkan 720 dokumen lowongan terindeks.

Pada sisi kanan, **Alur Kerja Utama** berjalan secara *real-time* setiap sesi: pengguna memasukkan narasi karier → sistem mengeksekusi pipeli-

ne RAG → menghasilkan rekomendasi personal → pengguna membaca dan mengevaluasi → siklus berulang.

Gambar ?? menampilkan rancangan penelitian menyeluruh yang mengintegrasikan DSR dan metode prototype:



Gambar 2.7: Rancangan Penelitian (DSR + Metode Prototype Terintegrasi)

Rancangan pada Gambar ?? menegaskan bahwa metode prototype bukan sekadar pelengkap, melainkan merupakan elemen kunci dalam strategi implementasi DSR. Loop evaluasi dalam prototype (“Hasil Sesuai?”) merupakan instansiasi dari fase “Perulangan” pada DSR. Pendekatan ini memastikan proses pengembangan berlangsung secara fleksibel, terukur, dan tetap relevan secara teoritis sekaligus andal dalam fungsi.

2.5 Skema Database dan Manajemen Infrastruktur

Bagian ini menjelaskan secara rinci desain skema database PostgreSQL dan seluruh infrastruktur teknis yang mendukung sistem NusaNara di lingkungan produksi. Pemahaman skema dan infrastruktur sangat penting karena seluruh pipeline RAG — dari penyimpanan vektor hingga sistem adaptif — bergantung pada keputusan desain di lapisan ini.

2.5.1 Skema Database PostgreSQL

Skema database terdiri dari tiga tabel utama yang saling berelasi. Seluruh tabel berada dalam satu instance PostgreSQL 16, memanfaatkan ekstensi **pgvector** untuk kolom bertipe **VECTOR(768)**.

Tabel 1: knowledge_base

Tabel ini merupakan *Single Source of Truth* untuk seluruh konten lowongan pekerjaan yang menjadi basis pengetahuan RAG. Setiap baris merepresentasikan satu dokumen lowongan.

```

1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE knowledge_base (
4     id                SERIAL PRIMARY KEY,
5     cluster           TEXT NOT NULL,           -- Klaster
6     title             TEXT NOT NULL,           -- Judul
7     company           TEXT NOT NULL,           -- Nama
8     location          TEXT,                    -- Kota/
9     salary_text       TEXT,                    -- Teks gaji
10    salary_min        INTEGER,                 -- Gaji
11    salary_max        INTEGER,                 -- Gaji

```

karier (8 nilai)
posisi lowongan
perusahaan
lokasi kerja
asli, e.g. "2,5 jt - 3,5 jt"
minimum dalam Rupiah (hasil parsing)
maksimum dalam Rupiah (hasil parsing)

```

12      skills          TEXT[],          -- Array
           keterampilan yang diminta
13      content         TEXT NOT NULL,   -- Gabungan
           semua field (untuk embedding)
14      embedding       VECTOR(768),     -- Vektor
           semantik (nomic-embed)
15      search_vector    TSVECTOR,        -- Indeks
           full-text search
16      created_at      TIMESTAMPTZ DEFAULT NOW()
17 );
18
19 -- Indeks IVFFlat untuk pencarian ANN (Approximate
   Nearest Neighbor)
20 -- lists = sqrt(720) ~= 27 cluster K-Means
21 CREATE INDEX idx_knowledge_embedding
22     ON knowledge_base
23     USING ivfflat (embedding vector_cosine_ops)
24     WITH (lists = 27);
25
26 -- Indeks GIN untuk pencarian full-text yang cepat
27 CREATE INDEX idx_knowledge_fts
28     ON knowledge_base
29     USING GIN (search_vector);
30
31 -- Trigger untuk auto-generate search_vector dari kolom
   content
32 CREATE OR REPLACE FUNCTION update_search_vector()
33 RETURNS TRIGGER AS $$
34 BEGIN
35     NEW.search_vector := to_tsvector('simple', NEW.
           content);
36     RETURN NEW;
37 END;
38 $$ LANGUAGE plpgsql;
39
40 CREATE TRIGGER trig_update_search_vector
41     BEFORE INSERT OR UPDATE ON knowledge_base
42     FOR EACH ROW EXECUTE FUNCTION update_search_vector();

```

Listing 2.7: DDL Tabel knowledge_base

Penjelasan desain tabel:

- **Kolom `skills` (`TEXT[]`):** Array PostgreSQL dipilih (bukan tabel relasional terpisah) karena skill tidak memerlukan operasi join kompleks — hanya operasi overlap ($\cap$) yang cukup dilakukan di Python setelah fetch. Array juga memungkinkan query sederhana: `WHERE 'Python' = ANY(skills)`.
- **Kolom `content`:** Berisi gabungan seluruh field dalam satu blok teks (format: “[title] di [company]. Skills: [skills]”). Format ini memastikan embedding menangkap konteks penuh dokumen dalam satu vektor.
- **Kolom `embedding` (`VECTOR(768)`):** Disimpan sebagai vektor 768 dimensi. Setiap dimensi merupakan nilai `float32`. Total storage per dokumen untuk vektor: $768 \times 4 \text{ bytes} = 3072 \text{ bytes} \approx 3 \text{ KB}$. Total storage untuk 720 dokumen: $720 \times 3 \text{ KB} \approx 2.16 \text{ MB}$.
- **Indeks `IVFFlat` (`lists = 27`):** Parameter `lists` menentukan jumlah *cluster* K-Means yang dibangun oleh IVFFlat. Nilai 27 ditetapkan berdasarkan formula rekomendasi pgvector untuk dataset berukuran sedang: $lists = \lceil \sqrt{N} \rceil = \lceil \sqrt{720} \rceil = \lceil 26,8 \rceil = 27$. IVFFlat membagi 720 vektor ke dalam 27 *cluster*. Saat *query*, sistem hanya memeriksa cluster terdekat (`probes = 1` secara default), menghasilkan pencarian *approximate nearest neighbor* (ANN) yang jauh lebih cepat dibanding *brute-force* $O(n)$.
- **Trigger `search_vector`:** Menggunakan `to_tsvector('simple', ...)` dengan konfigurasi `simple` (tanpa stemming bahasa tertentu) agar kata kunci teknis seperti “Python”, “PostgreSQL”, dan “UI/UX” tidak di-stem menjadi bentuk lain yang salah.

Tabel 2: `user_profiles`

Tabel ini menyimpan profil adaptif setiap pengguna yang terakumulasi lintas sesi.

```

1 CREATE TABLE user_profiles (
2     user_id          TEXT PRIMARY KEY,          -- Clerk
      user ID (eksternal)
3     identified_skills TEXT[] DEFAULT '{}',      -- Array
      skill terakumulasi

```

```

4      career_interests TEXT[] DEFAULT '{}',      -- Array
        minat karier
5      profile_summary TEXT,                      -- Ringkasan
        narasi terkini
6      session_count   INTEGER DEFAULT 0,         -- Jumlah
        sesi total
7      last_active      TIMESTAMPTZ,              -- Timestamp
        sesi terakhir
8      created_at       TIMESTAMPTZ DEFAULT NOW()
9  );
10
11 -- Indeks untuk lookup cepat berdasarkan user_id (sudah
    PRIMARY KEY)
12 -- Indeks GIN untuk query array skill
13 CREATE INDEX idx_user_skills
14 ON user_profiles USING GIN (identified_skills);

```

Listing 2.8: DDL Tabel user_profiles

Penjelasan desain tabel:

- **Primary Key user_id:** Menggunakan Clerk user ID (format: user_xxxx...) sebagai primary key. Tidak menggunakan UUID internal karena Clerk user ID sudah unik secara global dan menghindari satu lapisan mapping tambahan.
- **Kolom identified_skills dan career_interests:** Keduanya ber-tipe TEXT[] DEFAULT '{}' (array kosong, bukan NULL). Default array kosong memastikan operasi UNION set di Python tidak memerlukan pengecekan NULL: `set(existing_skills or []) | set(new_skills)`.
- **Kolom session_count:** Di-increment setiap kali background task adaptive berhasil dieksekusi. Berguna untuk analisis longitudinal: seberapa sering pengguna berinteraksi dengan sistem.
- **Indeks GIN pada identified_skills:** Memungkinkan query efisien “pengguna yang memiliki skill Python” menggunakan operator @> (contains). Berguna untuk fitur analitik di masa depan.

Tabel 3: recommendation_history

Tabel ini menyimpan log lengkap setiap sesi rekomendasi sebagai *audit trail*.

```

1 CREATE TABLE recommendation_history (
2     id SERIAL PRIMARY KEY,
3     user_id TEXT NOT NULL REFERENCES
        user_profiles(user_id),
4     session_id UUID DEFAULT gen_random_uuid(),
5     user_narrative TEXT NOT NULL,          -- Narasi
        input pengguna
6     profile_snapshot JSONB,                -- Snapshot
        profil saat itu
7     retrieved_docs JSONB,                  -- Top-3
        dokumen yang digunakan
8     llm_response TEXT,                    -- Respons
        lengkap LLM
9     rag_scores JSONB,                     -- Skor RAG per
        dokumen
10    ttfb_ms INTEGER,                       -- Time to
        First Byte (ms)
11    created_at TIMESTAMPTZ DEFAULT NOW()
12 );
13
14 -- Indeks untuk riwayat per-user (query ORDER BY
    created_at)
15 CREATE INDEX idx_history_user
16 ON recommendation_history (user_id, created_at DESC);

```

Listing 2.9: DDL Tabel recommendation_history

Penjelasan desain tabel:

- **Kolom profile_snapshot (JSONB):** Menyimpan *snapshot* profil pengguna *pada saat* sesi terjadi, bukan referensi ke profil yang bisa berubah. Ini memungkinkan reproduksi hasil (*reproducibility*) — mengapa rekomendasi tertentu diberikan pada waktu itu.
- **Kolom retrieved_docs (JSONB):** Menyimpan Top-3 dokumen lengkap (title, company, skills, cosine similarity, skill bonus, final score). Data ini digunakan untuk menampilkan transparansi kepada pengguna dan untuk evaluasi *offline* kualitas retrieval.
- **Kolom rag_scores (JSONB):** Menyimpan skor detail: {doc_id, cosine_sim, skill_bonus, final_score, rrf_score}.

- **JSONB vs JSON:** PostgreSQL JSONB dipilih (bukan JSON) karena JSONB menyimpan data dalam format biner yang ter-parse, memungkinkan query langsung ke field nested (`rag_scores->'cosine_sim'`) dan penggunaan indeks GIN pada field JSONB.
- **Kolom `ttfb_ms`:** Menyimpan *Time to First Byte* dalam milidetik, memungkinkan monitoring performa historis.

2.5.2 Strategi Deployment: Native VPS Ubuntu dengan systemd

Sistem NusaNara dideploy ke VPS Ubuntu 22.04 menggunakan strategi **tanpa Docker** untuk komponen backend dan database, namun dengan manajemen proses menggunakan **systemd** (init system Linux). Keputusan ini diambil atas pertimbangan berikut:

- **Ollama tidak dapat berjalan optimal di dalam container Docker** pada VPS tanpa GPU NVIDIA — Docker memerlukan NVIDIA Container Toolkit untuk passthrough GPU, yang tidak relevan pada VPS CPU-only. Ollama berjalan langsung di host untuk akses memori yang lebih efisien.
- **PostgreSQL pada Docker mengalami overhead I/O** pada volume mounts. Instalasi native PostgreSQL memberikan performa disk I/O yang lebih baik, krusial untuk pencarian pgvector yang membaca data vektor berulang kali.
- **Simplifikasi debugging** pada lingkungan produksi: systemd journal (`journalctl`) lebih mudah diakses daripada Docker logs untuk monitoring real-time oleh satu developer.

Arsitektur Proses di VPS

Empat proses berjalan secara bersamaan di VPS, dikelola oleh systemd:

Tabel 2.5: Proses Produksi di VPS dan Manajemen via Systemd

Proses	Port	Systemd Unit	RAM Estimasi
Nginx (Reverse Proxy)	80, 443	nginx.service	~50 MB
FastAPI (Uvicorn)	8000 (internal)	nusanara.service	~500 MB
PostgreSQL 16	5432 (internal)	postgresql.service	~2 GB
Ollama (LLM + Embed)	11434 (internal)	ollama.service	~8 GB
Total estimasi			~10.5 GB

Sisa ~5.5 GB dari 16 GB RAM digunakan sebagai buffer OS, cache filesystem PostgreSQL, dan headroom untuk lonjakan penggunaan.

Konfigurasi Systemd untuk FastAPI

Systemd digunakan sebagai *process manager* untuk memastikan FastAPI otomatis restart saat crash dan saat VPS reboot:

```

1 # /etc/systemd/system/nusanara.service
2 [Unit]
3 Description=NusaNara FastAPI Backend
4 After=network.target postgresql.service ollama.service
5 Requires=postgresql.service
6
7 [Service]
8 Type=simple
9 User=ubuntu
10 WorkingDirectory=/home/ubuntu/nusanara/backend
11 Environment="PATH=/home/ubuntu/.venv/bin"
12 Environment="PYTHONPATH=/home/ubuntu/nusanara"
13 EnvironmentFile=/home/ubuntu/nusanara/.env # Memuat
    variabel rahasia
14 ExecStart=/home/ubuntu/.venv/bin/uvicorn \
15     main:app \
16     --host 127.0.0.1 \
17     --port 8000 \
18     --workers 2 \
19     --loop uvloop \
20     --http httptools
21 Restart=always
22 RestartSec=5

```

```
23 StandardOutput=journal
24 StandardError=journal
25 SyslogIdentifier=nusanara
26
27 [Install]
28 WantedBy=multi-user.target
```

Listing 2.10: Unit File Systemd untuk FastAPI NusaNara

Penjelasan konfigurasi systemd:

- **After=postgresql.service ollama.service:** Memastikan PostgreSQL dan Ollama sudah aktif sebelum FastAPI dimulai. Tanpa ini, aplikasi bisa startup sebelum database siap, menyebabkan error koneksi saat pertama kali.
- **EnvironmentFile=/home/ubuntu/nusanara/.env:** Memuat variabel lingkungan sensitif (DATABASE_URL, CLERK_SECRET_KEY) dari file `.env` yang tidak masuk ke version control (*gitignored*). Ini adalah praktik keamanan standar (*12-factor app*).
- **-workers 2:** Menjalankan 2 worker Uvicorn untuk menangani concurrent requests. Lebih dari 2 worker tidak disarankan karena setiap worker memuat model ke memori, dan Ollama sudah menangani *concurrent inference* secara internal.
- **-loop uvloop dan -http httptools:** Library C-based yang menggantikan event loop asyncio default Python. uvloop 2-4× lebih cepat dari asyncio default, krusial untuk throughput SSE streaming yang tinggi.
- **Restart=always dengan RestartSec=5:** Systemd akan restart proses 5 detik setelah crash, memastikan *high availability* tanpa intervensi manual.

Konfigurasi Nginx sebagai Reverse Proxy

Nginx bertindak sebagai gerbang (*gateway*) antara internet dan layanan internal (FastAPI pada port 8000):

```
1 # /etc/nginx/sites-available/nusanara
2 server {
3     listen 80;
4     server_name api.nusanara.app;
```



```
5      return 301 https://$host$request_uri;  # Redirect
6      HTTP ke HTTPS
7
8  server {
9      listen 443 ssl http2;
10     server_name api.nusanara.app;
11
12     # SSL/TLS via Let's Encrypt (Certbot)
13     ssl_certificate      /etc/letsencrypt/live/api.
14     nusanara.app/fullchain.pem;
15     ssl_certificate_key  /etc/letsencrypt/live/api.
16     nusanara.app/privkey.pem;
17     ssl_protocols        TLSv1.2 TLSv1.3;    # Matikan TLS
18     1.0/1.1 (usang)
19
20     # Konfigurasi CORS untuk frontend Vercel
21     add_header Access-Control-Allow-Origin "https://
22     nusanara.app";
23     add_header Access-Control-Allow-Methods "GET, POST,
24     OPTIONS";
25     add_header Access-Control-Allow-Headers "
26     Authorization, Content-Type";
27
28     location / {
29         proxy_pass          http://127.0.0.1:8000;
30         proxy_http_version  1.1;
31         proxy_set_header    Host $host;
32         proxy_set_header    X-Real-IP $remote_addr;
33         proxy_set_header    X-Forwarded-Proto $scheme;
34
35         # KONFIGURASI KRITIS UNTUK SSE STREAMING
36         proxy_buffering      off;          # Teruskan token
37         langsung tanpa buffer
38         proxy_cache          off;          # Jangan cache
39         respons SSE
40         proxy_read_timeout   300s;         # 5 menit untuk
41         respons LLM panjang
42         proxy_send_timeout   300s;
```

```

35     # Header SSE wajib
36     add_header          Cache-Control "no-cache";
37     add_header          X-Accel-Buffering "no";
38 }
39 }

```

Listing 2.11: Konfigurasi Nginx Lengkap untuk NusaNara

Penjelasan konfigurasi Nginx:

- **HTTPS via Let's Encrypt:** Sertifikat SSL gratis dari Let's Encrypt dikelola oleh Certbot yang berjalan sebagai cron job untuk auto-renewal setiap 90 hari. HTTPS wajib karena JWT Bearer token dikirim dari frontend — tanpa HTTPS, token dapat dicuri via *man-in-the-middle attack*.
- **proxy_buffering off:** Konfigurasi paling krusial untuk SSE. Nginx secara default meng-*buffer* seluruh respons backend sebelum mengirimkannya ke klien. Untuk SSE, ini berarti token yang dihasilkan LLM menumpuk di buffer Nginx dan tidak dikirim ke klien hingga respons selesai — menghilangkan efek streaming. Dengan **proxy_buffering off**, setiap chunk data: `... \n\n` langsung diteruskan.
- **X-Accel-Buffering: no:** Header tambahan yang diakui oleh beberapa versi Nginx untuk memastikan buffering benar-benar dinonaktifkan bahkan jika ada override konfigurasi upstream.
- **CORS headers:** Membatasi akses hanya dari domain `https://nusanara.app` (frontend Vercel), mencegah akses dari domain tidak sah. **OPTIONS** diperlukan untuk preflight CORS request browser.

Konfigurasi PostgreSQL untuk Performa

PostgreSQL dikonfigurasi melalui `postgresql.conf` untuk mengoptimalkan performa pada VPS 16 GB:

```

1 # Memori
2 shared_buffers          = 2GB          # Disesuaikan karena
   Llama 3.1 8B menyita RAM
3 effective_cache_size    = 4GB          # Estimasi cache OS
   yang tersedia setelah Llama
4 work_mem                = 256MB        # Memori per operasi
   sort/hash

```

```

5 maintenance_work_mem      = 1GB          # Untuk operasi VACUUM
   , CREATE INDEX
6
7 # Checkpointing
8 checkpoint_completion_target = 0.9      # Sebar I/O
   checkpoint
9 wal_buffers                = 64MB
10
11 # Koneksi
12 max_connections            = 50          # FastAPI menggunakan
   pool 10 koneksi

```

Listing 2.12: Konfigurasi PostgreSQL Kritis di postgresql.conf

Justifikasi `shared_buffers = 2 GB`: Nilai 2 GB dialokasikan karena kapasitas RAM (16 GB) sebagian besar telah digunakan oleh proses Ollama dan sistem operasi (lihat estimasi alokasi pada Tabel ??). `effective_cache_size = 4 GB` memberitahu *planner* PostgreSQL estimasi total cache yang tersedia agar dapat memilih strategi *query* yang lebih efisien.

Pipeline Deployment: Dari Kode ke Produksi

Alur deployment ke VPS dilakukan secara manual menggunakan Git pull, tanpa CI/CD otomatis. Prosedur deployment standar:

```

1 # 1. SSH ke VPS
2 ssh ubuntu@<ip-vps>
3
4 # 2. Pull kode terbaru dari repository
5 cd /home/ubuntu/nusanara
6 git pull origin main
7
8 # 3. Aktifkan virtual environment dan install
   dependencies
9 source .venv/bin/activate
10 pip install -r backend/requirements.txt --quiet
11
12 # 4. Jalankan migrasi database (jika ada perubahan schema
   )
13 psql -U nusanara -d nusanara_db -f migrations/latest.sql
14

```

```
15 # 5. Restart FastAPI service (zero-downtime jika
    menggunakan 2 workers)
16 sudo systemctl restart nusanara.service
17
18 # 6. Verifikasi status
19 sudo systemctl status nusanara.service
20 curl http://127.0.0.1:8000/health | python3 -m json.tool
```

Listing 2.13: Prosedur Deployment Manual ke VPS

Catatan: Verifikasi health check setelah restart memastikan koneksi database dan Ollama aktif sebelum traffic diarahkan kembali ke endpoint.

Keamanan Infrastruktur

Beberapa konfigurasi keamanan yang diterapkan:

- **UFW Firewall:** Hanya port 22 (SSH), 80 (HTTP), dan 443 (HTTPS) yang terbuka secara publik. Port 5432 (PostgreSQL), 8000 (FastAPI), dan 11434 (Ollama) terikat ke 127.0.0.1 (localhost only), tidak dapat diakses dari internet.
- **SSH Key Authentication:** Login SSH hanya dengan kunci publik/privat. Password authentication dinonaktifkan di `/etc/ssh/sshd_config` (`PasswordAuthentication` no).
- **Fail2Ban:** Memblokir IP yang gagal login SSH lebih dari 5 kali dalam 10 menit, mencegah *brute-force attack*.
- **Variabel Rahasia:** Seluruh kredensial (`DATABASE_URL`, `CLERK_SECRET`) disimpan di file `.env` yang di-*gitignore*, dimuat via `EnvironmentFile` systemd.

BAB 3

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil perancangan, pembangunan, dan evaluasi artefak NusaNara secara komprehensif, mengikuti fase Demonstrasi dan Evaluasi dalam kerangka DSR.

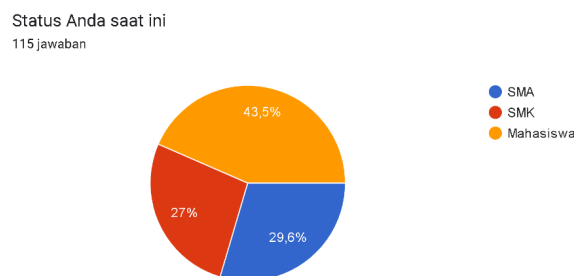
3.1 Hasil Perancangan dan Pengembangan Artefak

3.1.1 Pengumpulan Data Kebutuhan Awal

Pada alur *user-centric*, sebuah survei daring disebarkan untuk dua tujuan utama: (1) memvalidasi masalah dan tingkat kebingungan karier yang dialami oleh target pengguna, dan (2) memprioritaskan kebutuhan fungsional sistem sebagai dasar perancangan. Survei ini berhasil mengumpulkan data dari **115 responden** yang valid, yang telah menyetujui *informed consent*.

Demografi Responden

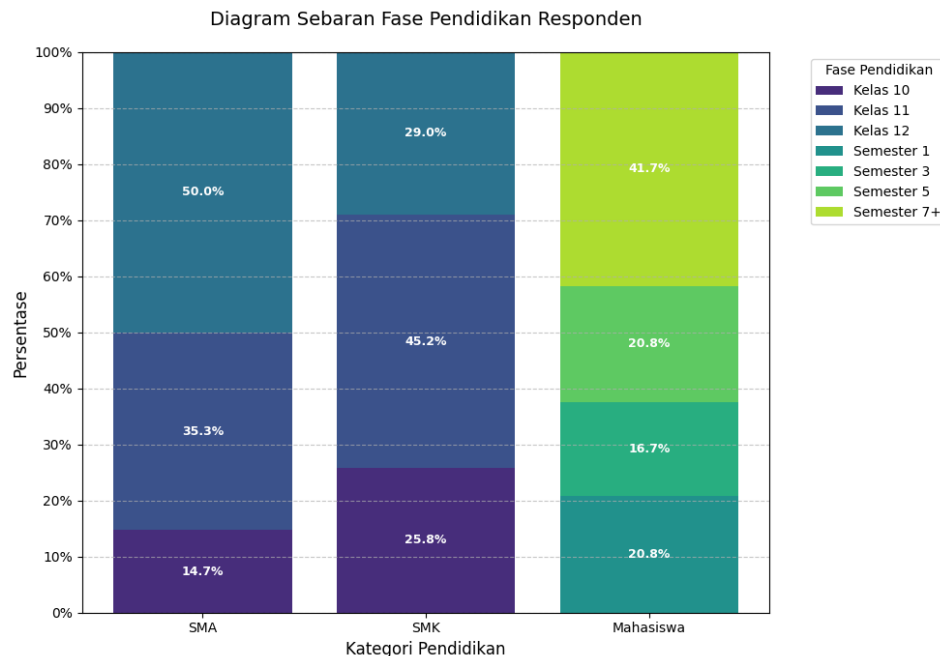
Analisis demografi (N=115) menunjukkan bahwa sampel responden sangat relevan dengan target populasi penelitian. Komposisi jenjang pendidikan, seperti yang divisualisasikan pada Gambar ??, didominasi oleh Mahasiswa (43,5%), diikuti oleh SMA (29,6%), dan SMK (27%).



Gambar 3.1: Diagram Komposisi Jenjang Pendidikan Responden (N=115)

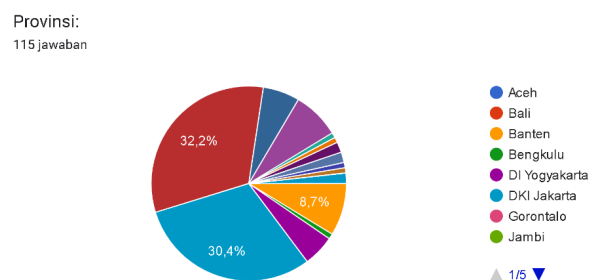
Temuan ini diperkuat dengan data sebaran fase pendidikan yang menunjukkan bahwa mayoritas responden berada pada fase transisi kritis (Gambar ??). Data menunjukkan 50% siswa SMA berasal dari Kelas 12, 45,2%

siswa SMK dari Kelas 11, dan 41,7% mahasiswa berasal dari Semester 7 atau lebih. Hal ini mengindikasikan bahwa responden adalah individu yang sedang aktif menghadapi keputusan karier pasca-kelulusan.



Gambar 3.2: Diagram Sebaran Fase Pendidikan Responden per Jenjang

Sebaran wilayah responden (Gambar ??) menunjukkan dominasi dari Jawa Barat (32,2%) dan DKI Jakarta (30,4%), namun terdapat pula responden dari Bengkulu, Kalimantan Timur, dan Maluku – menunjukkan jangkauan data yang representatif.



Gambar 3.3: Diagram Sebaran 5 Provinsi Teratas Responden

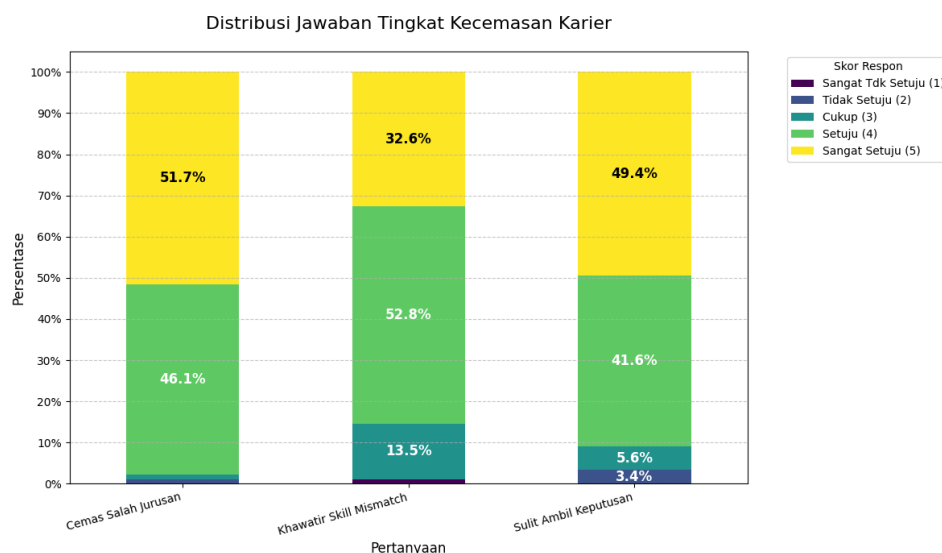
Validasi Masalah dan Identifikasi Kebutuhan

Hasil analisis rata-rata skor Likert (skala 1–5) dari tiga aspek utama diringkas pada Tabel ??.

Tabel 3.1: Ringkasan Hasil Analisis Kebutuhan (N=115)

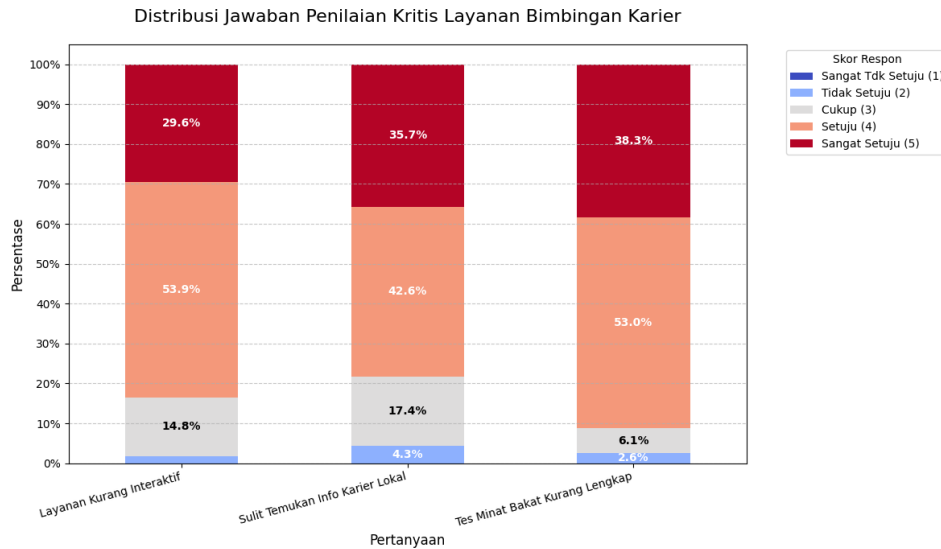
Aspek yang Diturunkur	Mean	Tingkat	% Setuju	Interpretasi
Tingkat Kece- masan Karier	4,37	Tinggi	92,2%	Urgensi masalah tervalidasi
Evaluasi Layan- an Saat Ini	4,16	Tinggi	84,3%	Kesenjangan solusi signifikan
Kebutuhan Sistem Baru	4,25	Tinggi	87,8%	Demand kuat untuk sistem baru

Distribusi jawaban Tingkat Kecemasan Karier (Mean 4,37) divisualisasikan pada Gambar ?? : 97,4% responden setuju “Cemas Salah Jurusan”, 85,4% “Khawatir Skill Mismatch”, dan 91,3% “Sulit Ambil Keputusan”.

**Gambar 3.4:** Persentase Tingkat Kecemasan dan Kesulitan Karier (N=115)

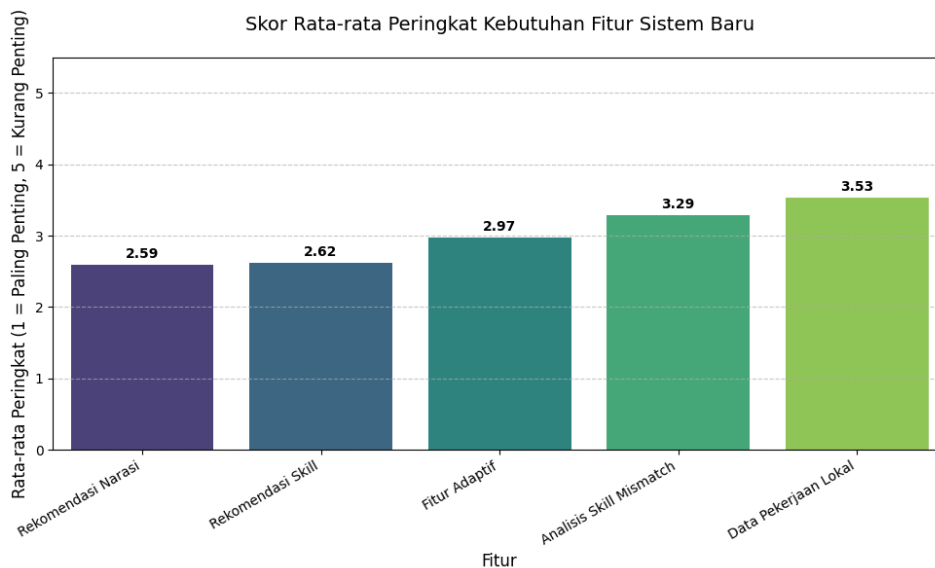
Gambar ?? menunjukkan penilaian kritis terhadap layanan yang ada:

91,3% menyatakan tes minat bakat konvensional kurang menggambarkan diri mereka, 83,5% merasa layanan kurang interaktif, dan 78,3% kesulitan menemukan informasi karier yang relevan dengan konteks lokal Indonesia.



Gambar 3.5: Persentase Penilaian Terhadap Layanan Karier Saat Ini (N=115)

Gambar ?? menunjukkan permintaan kuat terhadap sistem baru (Mean 4,25): 85,2% akan lebih terbantu dengan sistem berbasis narasi, 90,5% membutuhkan sistem adaptif, dan 87,9% menginginkan rekomendasi dalam bentuk narasi langkah-langkah.



Gambar 3.6: Distribusi Jawaban Kebutuhan Sistem Bimbingan Karier Baru

Temuan survei N=115 ini memberikan landasan empiris yang kuat: (1)

Validasi Masalah – kecemasan karier tinggi (Mean 4,37); (2) **Kesenjangan Solusi** – layanan konvensional dievaluasi kritis (Mean 4,16); dan (3) **Validasi Solusi** – demand kuat terhadap sistem naratif adaptif (Mean 4,25). Ini menjadi dasar konseptual bagi pengembangan artefak NusaNara.

3.1.2 Pengumpulan dan Persiapan Data Lowongan

Proses Web Scraping

Pengumpulan data lowongan dilakukan melalui serangkaian tahapan teknis menggunakan otomasi Selenium. Evaluasi awal terhadap tiga platform karier utama menghasilkan temuan:

- **Jobstreet:** proteksi *bot-detection* berbasis Cloudflare memblokir permintaan otomatis secara konsisten.
- **LinkedIn:** tidak menampilkan informasi gaji secara publik.
- **Glints:** aksesibel secara teknis dan menyediakan atribut data yang lengkap.

Berdasarkan evaluasi ini, Glints dipilih sebagai satu-satunya sumber data. Scraping menggunakan Selenium WebDriver dengan konfigurasi *selenium-stealth* untuk menangani konten berbasis JavaScript (SPA).

Hasil Pengumpulan Data

Hasil pengumpulan data menghasilkan 720 lowongan pekerjaan yang terdistribusi secara seimbang ke dalam 8 klaster karier (masing-masing 90 lowongan). Rincian distribusi klaster telah disajikan pada Tabel ???. Keseimbangan distribusi ini merupakan persyaratan metodologis yang disengaja agar metrik evaluasi per klaster dapat dibandingkan secara adil, karena klaster dengan lebih banyak data berpotensi menghasilkan skor yang lebih tinggi secara inheren ?.

Hasil Pembersihan Data dan Embedding Vektor (ETL)

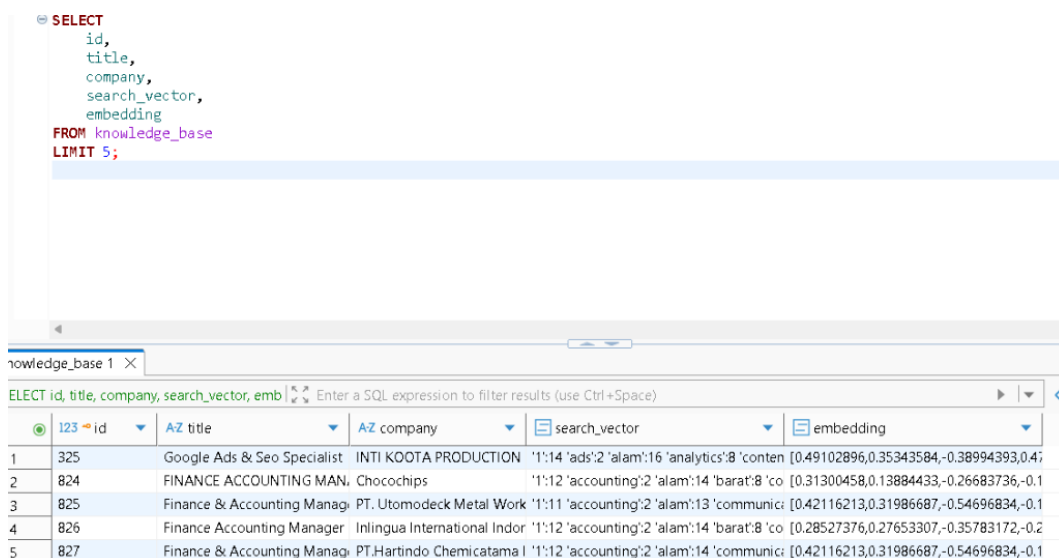
Setelah data mentah berhasil dikumpulkan, proses *Data Cleaning* dan *Embedding* (seperti yang dirancang pada Bab 3) dieksekusi secara berurutan. Hasil dari eksekusi pipeline ETL ini adalah sebagai berikut:

1. **Hasil *Data Cleaning*:** Dari total 720 dokumen mentah awal, seluruh dokumen (100%) berhasil dipertahankan. Proses validasi kelengkapan

tidak menggugurkan satu pun dokumen karena mekanisme *scraper* sejak awal telah diprogram ketat untuk hanya menyimpan DOM yang valid (memiliki **title**, **company**, dan **skills**). Tahap deduplikasi juga menghasilkan 0 dokumen duplikat berkat isolasi **keyword** per kluster secara presisi.

2. **Hasil Konversi *TSVECTOR*:** Proses konversi leksikal menggunakan kamus *simple* berjalan cepat ($\sim 0,4$ detik untuk 720 dokumen), menghasilkan kolom *search_vector* yang langsung diindeks dengan *Generalized Inverted Index* (GIN) di PostgreSQL tanpa galat (*error*).
3. **Hasil Ekstraksi *Embedding*:** Pipeline inferensi via Ollama API (*nomic-embed-text-v2*) berhasil mengubah 720 gabungan teks menjadi himpunan *VECTOR(768)*. Waktu yang dibutuhkan untuk memproses total 720 dokumen tersebut adalah ~ 14 detik (*batch processing* via CPU server). Seluruh 720 vektor sukses terindeks menggunakan algoritma *IVFFlat* dengan parameter *lists=27*.

Data bersih yang siap saji ini merupakan *Knowledge Base* final yang akan ditarik secara langsung oleh modul hibrida RAG (**semantic** dan **lexical search**). Visualisasi dari hasil **database** akhir ini dapat dilihat pada Gambar ??.



```

SELECT
  id,
  title,
  company,
  search_vector,
  embedding
FROM knowledge_base
LIMIT 5;

```

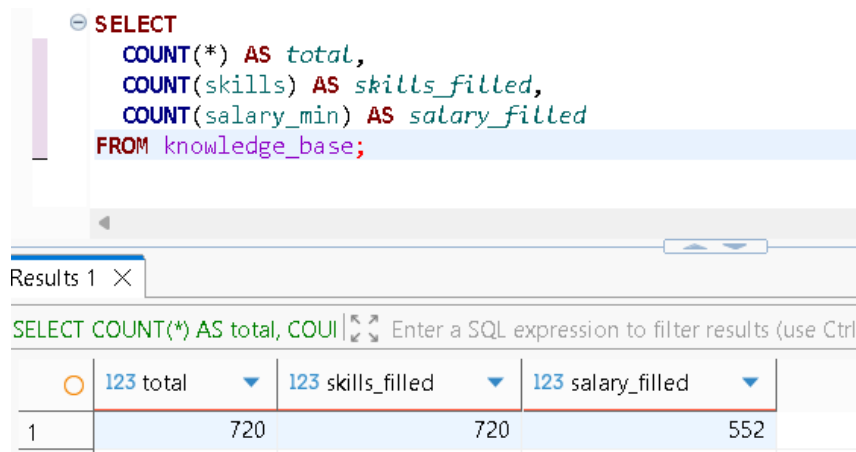
	id	title	company	search_vector	embedding
1	325	Google Ads & Seo Specialist	INTI KOTA PRODUCTION	'1':14 'ads':2 'alam':16 'analytics':8 'conten	[0.49102896,0.35343584,-0.38994393,0.4;
2	824	FINANCE ACCOUNTING MAN.	Chocochips	'1':12 'accounting':2 'alam':14 'barat':8 'co	[0.31300458,0.13884433,-0.26683736,-0.1
3	825	Finance & Accounting Manag.	PT. Utomodeck Metal Work	'1':11 'accounting':2 'alam':13 'communi	[0.42116213,0.31986687,-0.54696834,-0.1
4	826	Finance Accounting Manager	Inlingua International Indor	'1':12 'accounting':2 'alam':14 'barat':8 'co	[0.28527376,0.27653307,-0.35783172,-0.2
5	827	Finance & Accounting Manag.	PT.Hartindo Chemicatama I	'1':12 'accounting':2 'alam':14 'communi	[0.42116213,0.31986687,-0.54696834,-0.1

Gambar 3.7: Tangkapan Layar Data Vektor dan Leksikal pada PostgreSQL (DBeaver)

Verifikasi dan Analisis Dataset Akhir

1. Verifikasi Data (Bukti Eksistensi)

Untuk memvalidasi keberadaan data secara empiris setelah tahapan ekstraksi dan pembersihan (ETL), query SQL dasar dilakukan langsung pada basis data PostgreSQL. Gambar ?? membuktikan bahwa sistem telah sukses menyimpan total 720 dokumen dengan kalkulasi entri yang *non-null*. Gambar ?? mengonfirmasi bahwa distribusi per klaster persis mencapai angka 90 dokumen per klaster tanpa adanya anomali.



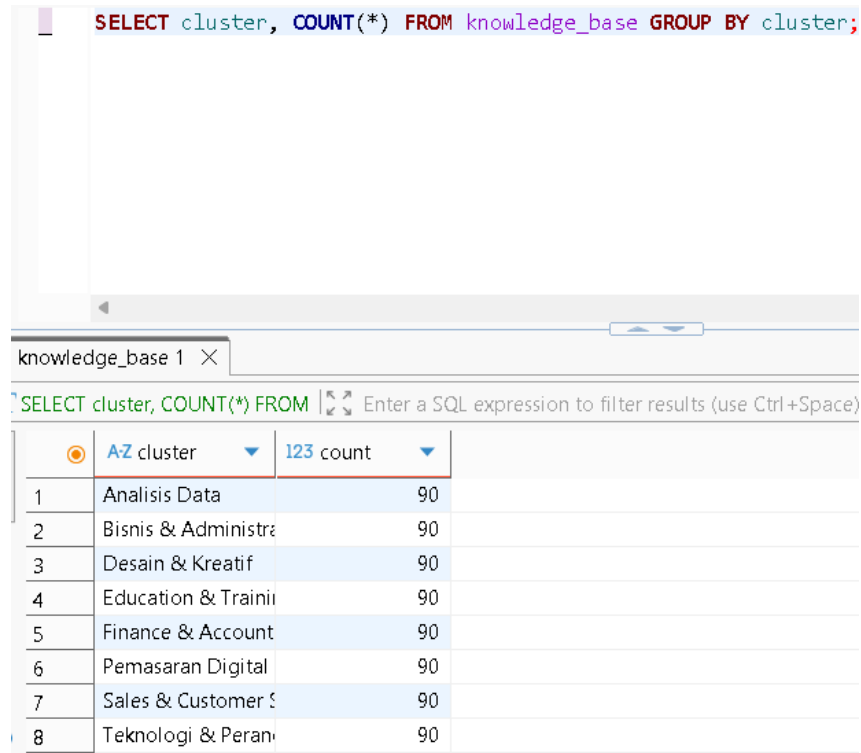
The screenshot shows a PostgreSQL query editor with the following SQL query:

```
SELECT
  COUNT(*) AS total,
  COUNT(skills) AS skills_filled,
  COUNT(salary_min) AS salary_filled
FROM knowledge_base;
```

Below the query, the results are displayed in a table with the following data:

	total	skills_filled	salary_filled
1	720	720	552

Gambar 3.8: Hasil Eksekusi Query Agregasi Total Data dan Kelengkapan Kolom



The screenshot shows a PostgreSQL query editor with the following SQL query:

```
SELECT cluster, COUNT(*) FROM knowledge_base GROUP BY cluster;
```

Below the query, the results are displayed in a table with the following data:

	cluster	count
1	Analisis Data	90
2	Bisnis & Administrasi	90
3	Desain & Kreatif	90
4	Education & Training	90
5	Finance & Accounting	90
6	Pemasaran Digital	90
7	Sales & Customer Service	90
8	Teknologi & Peran	90

Gambar 3.9: Hasil Eksekusi Query Distribusi Data per Klaster Karier

2. Karakteristik Dataset (*Data Completeness*)

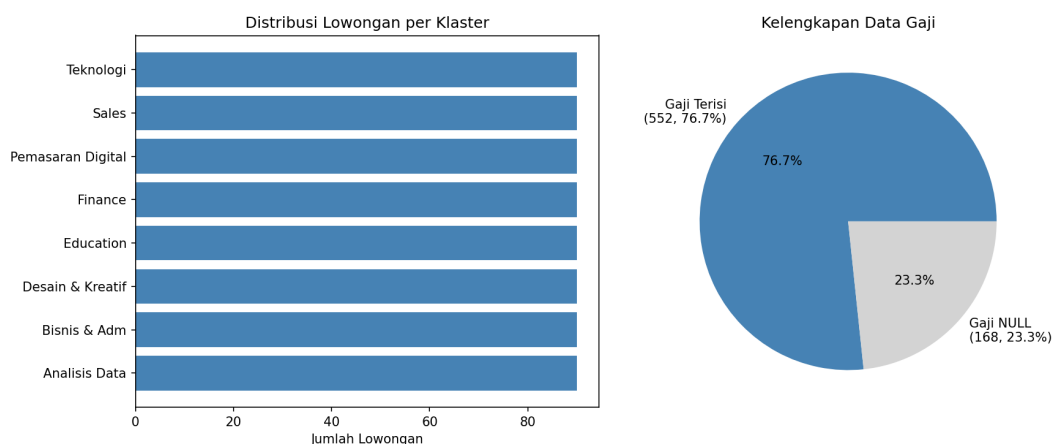
Sebagian besar data diekstrak dengan tingkat kelengkapan yang sempurna karena mekanisme *scraper* dirancang untuk membuang elemen DOM yang rusak. Namun, platform sumber (Glints) menetapkan transparansi gaji (*salary*) sebagai kolom opsional bagi perusahaan. Tabel ?? meringkas status kelengkapan data hasil ekstraksi tersebut.

Tabel 3.2: Ringkasan Karakteristik Kolom *Knowledge Base*

Kolom	Total Terisi	NULL	Keterangan
<code>title</code>	720	0	Wajib ada pada elemen DOM
<code>company</code>	720	0	Wajib ada pada elemen DOM
<code>location</code>	720	0	Wajib ada pada elemen DOM
<code>skills</code>	720	0	Wajib terisi minimal 1 entitas
<code>salary_text</code>	552	168	Teks gaji asli dari scraping
<code>salary_min</code>	552	168	Hasil parsing ke Rupiah (integer)
<code>salary_max</code>	552	168	Hasil parsing ke Rupiah (integer)

3. Visualisasi Analisis Data (*Exploratory Data Analysis*)

Visualisasi menggunakan pustaka *Matplotlib* pada Python (Gambar ??) disusun untuk memperjelas karakteristik dataset. Grafik batang secara gamblang mendemonstrasikan pemerataan kluster tanpa adanya *skewness* pada kategori tertentu. Di sisi lain, *pie chart* menggarisbawahi bahwa mayoritas lowongan (76,7%) secara transparan menyertakan rentang gaji (552 dari 720 data), yang mana keberadaan data ini akan secara signifikan memperkaya dimensi rekomendasi NusaNara ketika pengguna mengemukakan preferensi kompensasi finansial yang spesifik.



Gambar 3.10: Analisis Distribusi Lowongan dan Kelengkapan Data Gaji

Skema Basis Data

Setiap lowongan disimpan dalam tabel `knowledge_base` dengan skema berikut:

Tabel 3.3: Skema Tabel `knowledge_base` pada PostgreSQL 16

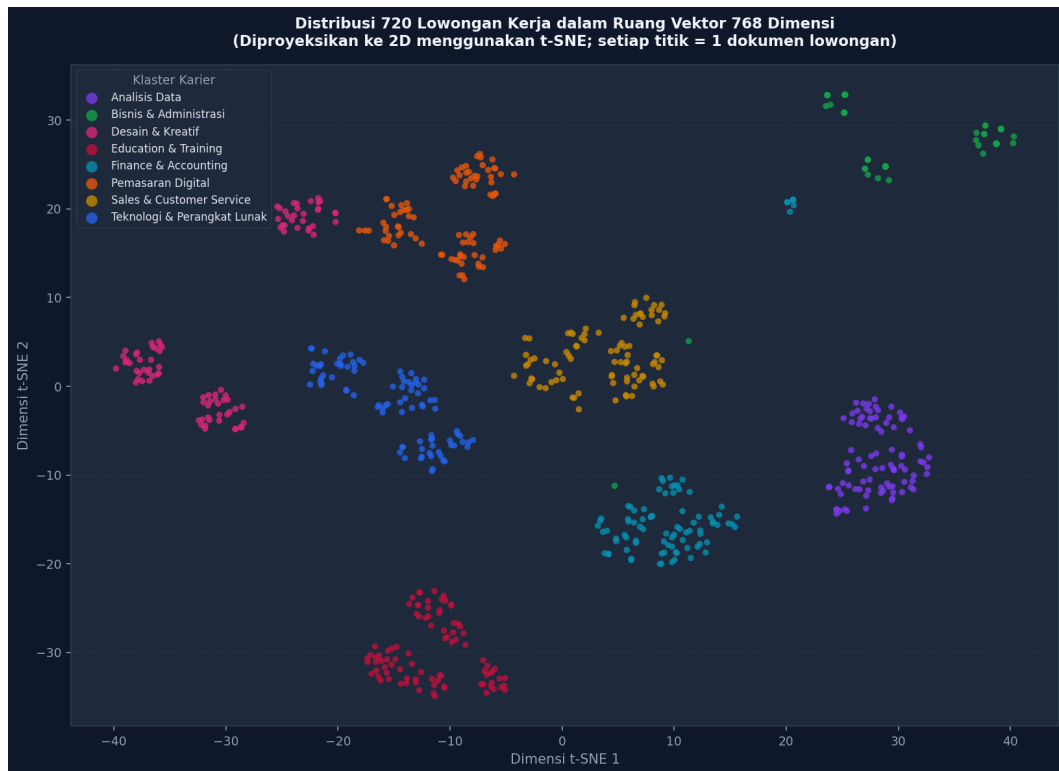
Kolom	Tipe Data	Keterangan
<code>id</code>	SERIAL	Primary key auto-increment
<code>title</code>	TEXT	Judul posisi pekerjaan
<code>company</code>	TEXT	Nama perusahaan
<code>location</code>	TEXT	Kota/provinsi
<code>salary_text</code>	TEXT	Teks gaji asli dari scraping (opsional)
<code>salary_min</code>	INTEGER	Gaji minimum dalam Rupiah (hasil parsing)
<code>salary_max</code>	INTEGER	Gaji maksimum dalam Rupiah (hasil parsing)
<code>skills</code>	TEXT[]	Array keterampilan
<code>cluster</code>	TEXT	Salah satu dari 8 klaster karier
<code>content</code>	TEXT	Narasi gabungan untuk embedding
<code>embedding</code>	VECTOR(768)	Representasi vektor nomic-embed
<code>search_vector</code>	TSVECTOR	Indeks full-text search

Kolom `embedding` diisi oleh pipeline Task 04 menggunakan model `nomic-embed-text-v2` (768 dimensi), diindeks dengan algoritma IVFFlat untuk mempercepat pencarian ANN. Kolom `search_vector` dibangkitkan otomatis menggunakan fungsi `to_tsvector('simple', ...)`.

Analisis Ruang Vektor: Visualisasi Distribusi Semantik 8 Klaster

Guna memverifikasi secara empiris bahwa model *embedding* yang digunakan (`nomic-embed-text-v2-moe`) mampu membedakan profesi secara semantik, seluruh 720 vektor berdimensi 768 yang tersimpan di dalam basis data PostgreSQL dianalisis secara komputasional. Karena ruang 768 dimensi tidak dapat divisualisasikan secara langsung, dilakukan reduksi dimensi dua tahap: (1) *Principal Component Analysis* (PCA) mereduksi dari 768 ke 50 dimensi (mempertahankan 72,7% varian total), kemudian (2) algoritma *t-Distributed Stochastic Neighbor Embedding* (t-SNE) ? memproyeksikan representasi 50D tersebut ke dalam ruang 2D yang dapat divisualisasikan, dengan parameter

$perplexity = 40$ dan 1.000 iterasi optimasi.



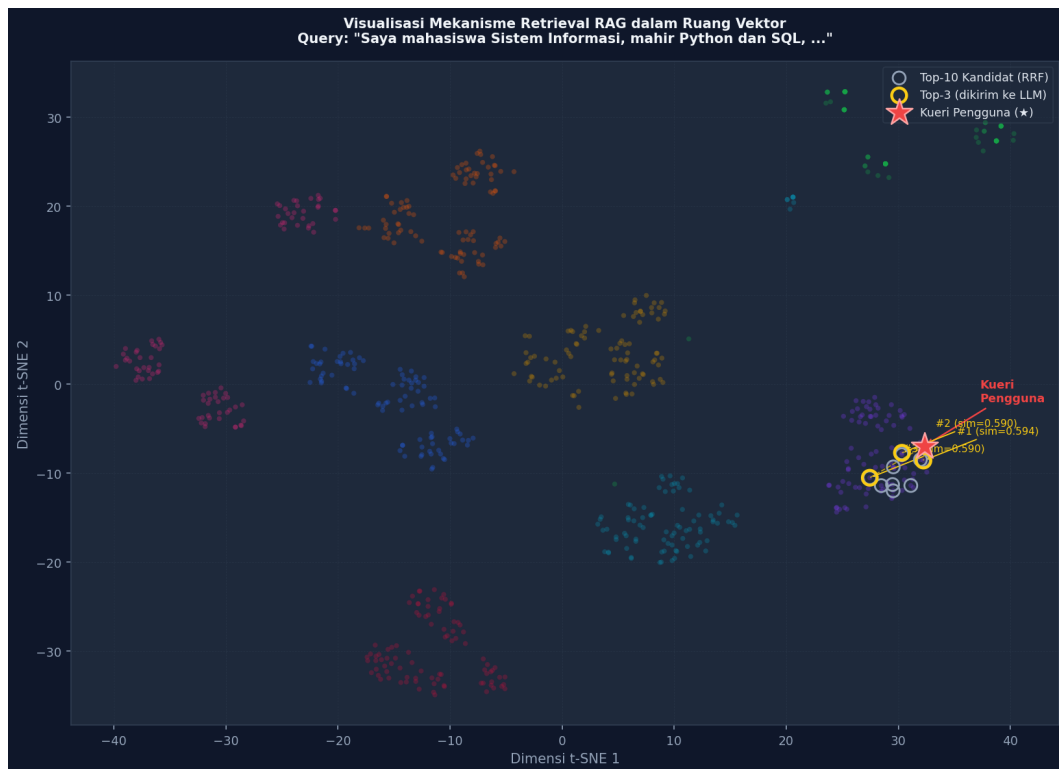
Gambar 3.11: Visualisasi t-SNE Distribusi 720 Lowongan dalam Ruang Vektor 768 Dimensi (setiap titik merepresentasikan satu dokumen lowongan, diwarnai sesuai klaster kariernya)

Hasil proyeksi t-SNE pada Gambar ?? memberikan bukti visual yang sangat kuat. Titik-titik dari klaster yang sama secara konsisten membentuk gugus (*cluster*) yang kompak dan terpisah satu sama lain dalam ruang 2D, meskipun tidak ada informasi label klaster yang digunakan selama proses *embedding* — model hanya menerima teks mentah. Fenomena ini membuktikan bahwa `nomic-embed-text-v2-moe` berhasil mengkodekan *semantic proximity* yang bermakna secara domain: lowongan “Data Analyst” dan “Data Scientist” berada pada gugus yang berdekatan (Analisis Data), sementara terdapat pemisahan yang jelas antara klaster “Education & Training” dengan “Finance & Accounting”. Keterpisahan yang konsisten ini merupakan prasyarat fundamental agar komponen *semantic search* dalam *Hybrid Search* dapat bekerja dengan akurasi tinggi.

Visualisasi Mekanisme Retrieval RAG dalam Ruang Vektor

Untuk mendemonstrasikan bagaimana sistem RAG mengeksekusi pencarian dalam konteks nyata, dilakukan simulasi satu kueri pengguna (“*Saya maha-*

siswa Sistem Informasi, mahir Python dan SQL, tertarik di bidang Data Analyst”) secara *end-to-end*. Narasi tersebut diubah menjadi vektor 768 dimensi menggunakan model *embedding* yang identik. Dalam implementasi produksi, pencarian dilakukan oleh **pgvector** menggunakan operator *Cosine Distance* ($\leq$) dengan akselerasi indeks *IVFFlat* — sebuah algoritma *Approximate Nearest Neighbor* (ANN) yang lebih efisien secara komputasi dibandingkan *exact KNN* untuk skala dokumen besar. Skor *Cosine Similarity* (dikonversi dari jarak: $sim = 1 - dist$) kemudian digunakan untuk menentukan Top-K kandidat semantik.



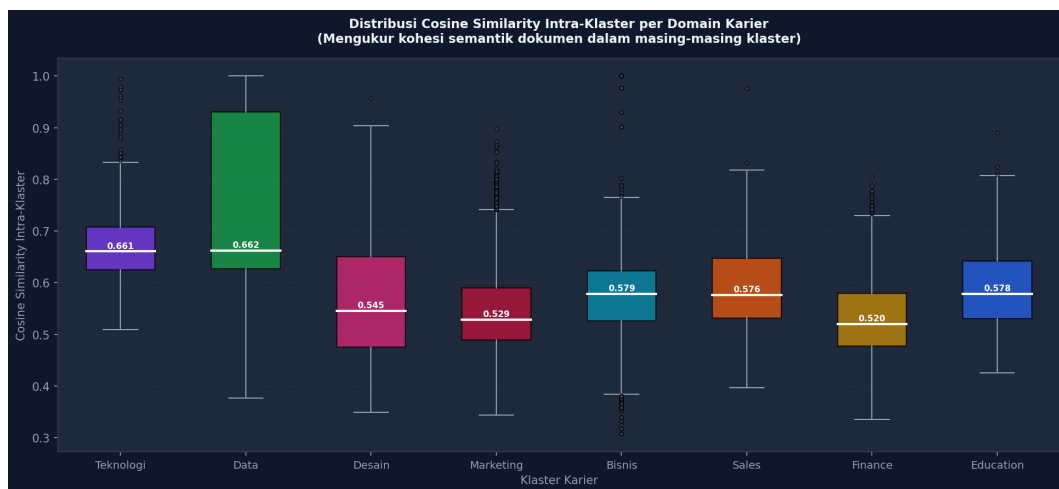
Gambar 3.12: Visualisasi Mekanisme *Semantic Search* (ANN via IVFFlat/pgvector): Posisi Kueri Pengguna (bintang merah ★) dan Dokumen Terambil (Top-10 lingkaran abu-abu; Top-3 lingkaran emas) dalam Proyeksi t-SNE

Gambar ?? mengilustrasikan mekanisme *Approximate Nearest Neighbor* (ANN) yang mendasari komponen *vector search* dalam *Hybrid Search*. Bintang merah (★) merepresentasikan posisi kueri pengguna setelah diproyeksikan ke ruang 2D, sementara dokumen-dokumen yang terambil sebagai Top-10 (ditandai lingkaran abu-abu) dan Top-3 (ditandai lingkaran emas dengan garis penghubung) terlihat jelas berposisi di **dalam klaster Analisis Data** — yang secara semantik paling dekat dengan narasi karier pengguna. Sebagaimana tampak pada label anotasi dalam gambar, tiga dokumen teratas yang akhirnya

dikirimkan ke LLM sebagai konteks memiliki skor *Cosine Similarity* berturut-turut: **#1 (sim = 0,594)**, **#2 (sim = 0,590)**, dan **#3 (sim = 0,590)** — semuanya merupakan posisi Data Analyst dari perusahaan-perusahaan berbeda, yang secara intuitif sesuai dengan profil yang dideskripsikan dalam narasi. Perlu dicatat bahwa visualisasi ini menggunakan *exact* Cosine Similarity untuk keperluan ilustrasi yang lebih jelas, sementara implementasi produksi menggunakan ANN (IVFFlat) yang hasilnya secara empiris ekuivalen pada skala 720 dokumen.

Analisis Kohesi Semantik Intra-Klaster (Cosine Similarity)

Evaluasi kuantitatif kualitas representasi vektor dilanjutkan dengan mengukur *Cosine Similarity* antar-dokumen dalam masing-masing klaster (*intra-cluster similarity*). Metrik ini mengukur seberapa erat seluruh 90 dokumen dalam satu klaster saling berkaitan secara semantik; nilai yang tinggi mengindikasikan model berhasil memetakan dokumen-dokumen satu bidang ke zona yang kompak dalam ruang vektor.



Gambar 3.13: Distribusi *Cosine Similarity* Intra-Klaster per Domain Karier (Boxplot; garis putih = nilai median)

Gambar ?? menyajikan distribusi *intra-cluster similarity* ke-8 klaster. Beberapa temuan kunci dapat diidentifikasi:

- **Kohesi Tertinggi:** Klaster *Analisis Data* (median = 0,662) dan *Teknologi & Perangkat Lunak* (median = 0,661) memiliki kohesi semantik tertinggi. Kedua klaster ini mencakup lowongan dengan terminologi teknis yang sangat spesifik dan konsisten (misal: *Python*, *SQL*, *Machi-*

ne Learning, React, DevOps), sehingga dokumen-dokumen di dalamnya menghasilkan representasi vektor yang sangat homogen.

- **Kohesi Terendah:** Klaster *Finance & Accounting* (median = 0,520) dan *Pemasaran Digital* (median = 0,529) menunjukkan variabilitas lebih tinggi. Hal ini dapat dijelaskan oleh keragaman sub-domain yang luas: klaster Finance mencakup posisi mulai dari *Tax Consultant* hingga *Financial Analyst*, sementara Pemasaran Digital mencakup *SEO Specialist*, *Content Creator*, hingga *Google Ads Specialist* — masing-masing dengan vokabuler yang berbeda meski berada dalam satu klaster.
- **Grand Median Keseluruhan:** Nilai 0,577 (rata-rata dari seluruh median klaster) secara konsisten berada di atas ambang batas kemiripan semantik yang bermakna (0,5), membuktikan bahwa seluruh *Knowledge Base* tersusun dengan baik secara representasi vektor dan tidak mengandung *noise* dokumentasi yang signifikan.

Ketiga analisis di atas — distribusi t-SNE, mekanisme *Semantic Search* (ANN/IVFFlat), dan distribusi *Cosine Similarity* — secara kolektif membuktikan bahwa fondasi *Knowledge Base* NusaNara memiliki kualitas representasi semantik yang tinggi dan layak digunakan sebagai basis bagi sistem RAG produksi.

3.1.3 Implementasi Sistem

Spesifikasi Lingkungan Produksi (VPS)

Sistem NusaNara di-*deploy* pada VPS *dedicated* dengan lokasi *data center* di Jakarta, Indonesia. Pemilihan lokasi ini bertujuan meminimalkan latensi jaringan bagi pengguna target.

Tabel 3.4: Spesifikasi Perangkat Keras VPS Produksi

Komponen	Spesifikasi
CPU	4 vCPU
RAM	16 GB
Storage	100 GB SSD (RAID 10)
Jaringan	Dedicated IPv4, Unlimited Bandwidth
Data Center	Jakarta, Indonesia
Sistem Operasi	Ubuntu 22.04 LTS

Pemilihan 4 vCPU dan 16 GB RAM merupakan keputusan teknis yang dipertimbangkan berdasarkan kebutuhan eksekusi *Large Language Model* (LLM) secara lokal. Model Llama 3 dipilih dalam varian **8B parameter (4-bit Quantized)** (bukan versi 70B) agar dapat beroperasi pada RAM 16 GB. Walaupun sistem menggunakan dua agen LLM (*Main RAG Agent* dan *Extractor Agent*), Ollama mengelola memori dengan memuat bobot model hanya satu kali. Kedua agen tersebut mengakses model secara bergantian (berbagi *instance* memori yang sama), sehingga *bottleneck* memori dapat dihindari.

Tabel 3.5: Spesifikasi Perangkat Lunak Produksi

Komponen	Teknologi	Versi/Detail
Web Server	Nginx	Reverse Proxy, SSL Termination
Process Manager	Systemd	<code>nusanara.service</code> , <code>Restart=always</code>
SSL/TLS	Let's Encrypt	Auto-renew via Certbot
Firewall	UFW	Port 5432, 11434, 8000 – internal only
Runtime Backend	Python	3.11
Web Framework	FastAPI + Uvicorn	ASGI
Database	PostgreSQL	16 + pgvector (VECTOR(768), IVFFlat)
DB Driver	asyncpg	Async, connection pooling
LLM Platform	Ollama	Local inference
LLM (Generator & Extractor)	Llama 3.1	8B parameter (4-bit Quantized)
Embedding Model	nomic-embed-text-v2-moe	768 dimensi, MoE
Frontend	Next.js	14 (App Router, TypeScript)
Auth Provider	Clerk	JWT RS256
Frontend Hosting	Vercel	CDN Global

Implementasi Pipeline RAG

Pipeline RAG diimplementasikan dalam `backend/services/llm_service.py`. Alur eksekusi untuk setiap permintaan rekomendasi:

```
1 async def generate_recommendation(narrative, user_id,
2     pool):
3     # 1. Query Expansion
4     expanded = await _expand_query(narrative)
5
6     # 2. Embedding (nomic-embed-text-v2-moe)
7     vector = await get_embedding(f"search_query: {
8         expanded}")
9
10    # 3. Hybrid Search via RRF
11    candidates = await hybrid_search(vector, expanded,
12        top_k=10)
13
14    # 4. Re-ranking heuristik
15    top3 = rerank(candidates, user_profile.skills, top_n
16        =3)
17
18    # 5. Build prompt Decision Intelligence
19    prompt = build_prompt(narrative, top3, user_profile)
20
21    # 6. LLM Stream (Llama 3.1, stream=True)
22    async for token in ollama_stream(prompt):
23        yield f"data: {token}\n\n"
24
25    # 7. Simpan history
26    await save_history(user_id, narrative, top3,
27        full_response)
28
29    # 8. Background task (NON-BLOCKING)
30    asyncio.create_task(
31        extract_and_update_profile(user_id, narrative,
32            pool)
33    )
34    yield "data: [DONE]\n\n"
```

Listing 3.1: Alur pipeline RAG di `llm_service.py`

Fungsi Hybrid Search dengan RRF:

Implementasi *Hybrid Search* menggabungkan dua paradigma pencarian yang saling melengkapi. Pencarian semantik menangkap kesamaan makna (misalnya “programmer” cocok dengan “software engineer” meskipun kata-katanya berbeda), sedangkan pencarian leksikal menangkap kecocokan kata kunci eksak (misalnya nama teknologi “PostgreSQL” yang harus muncul literal dalam dokumen).

```

1 async def hybrid_search(query_vector, query_text, top_k
    =10):
2     # Set probes untuk optimasi IVFFlat
3     await pool.execute("SET LOCAL ivfflat.probes = 3;")
4
5     # Pencarian Semantik (pgvector cosine similarity)
6     semantic = await pool.fetch("""
7         SELECT id, title, content, skills, cluster,
8             1 - (embedding <=> $1) AS cosine_sim,
9             ROW_NUMBER() OVER (ORDER BY embedding <=>
10                $1) AS rank
11         FROM knowledge_base LIMIT 20
12     """, query_vector)
13
14     # Pencarian Full-Text (tsvector)
15     lexical = await pool.fetch("""
16         SELECT id,
17             ROW_NUMBER() OVER (ORDER BY ts_rank DESC)
18             AS rank
19         FROM (SELECT id, ts_rank(search_vector, q) AS
20             ts_rank
21             FROM knowledge_base, plainto_tsquery('
22                 simple',$1) q
23             WHERE search_vector @@ q LIMIT 20) sub
24     """, query_text)
25
26     # RRF Fusion: skor = 1/(60+rank_sem) + 1/(60+rank_lex
27         )
28     rrf = {}
29     for doc in semantic:
30         rrf[doc['id']] = 1.0 / (60 + doc['rank'])
31     for doc in lexical:

```

```

27         rrf[doc['id']] = rrf.get(doc['id'], 0) + 1.0/(60+
28             doc['rank'])
29
30     return sorted(rrf.items(), key=lambda x: x[1],
31                   reverse=True)[:top_k]

```

Listing 3.2: Implementasi Hybrid Search dan RRF

Penjelasan kode baris per baris:

- **Baris 4–7 (Pencarian Semantik):** Operator `<=>` adalah operator *cosine distance* bawaan pgvector. Ekspresi `1 - (embedding <=> $1)` mengonversi distance menjadi similarity. `ROW_NUMBER()` memberikan peringkat 1, 2, 3, ... berdasarkan kemiripan. `LIMIT 20` mengambil 20 teratas.
- **Baris 10–15 (Pencarian Leksikal):** Operator `@@` mencocokkan `tsvector` dengan `tsquery`. Fungsi `ts_rank()` menghitung skor relevansi leksikal berdasarkan frekuensi kemunculan kata kunci. `plainto_tsquery('simple', ...)` menggunakan konfigurasi 'simple' (tanpa stemming) karena nama teknologi dan posisi tidak boleh di-stem.
- **Baris 18–22 (RRF Fusion):** Setiap dokumen mendapat skor $\frac{1}{60+rank}$ dari masing-masing pencarian. Dokumen yang muncul di kedua pencarian mendapat akumulasi skor yang lebih tinggi. Konstanta $k = 60$ memastikan perbedaan peringkat yang berdekatan tidak terlalu drastis.

Fungsi Re-ranking Heuristik:

Setelah RRF menghasilkan Top-10 kandidat, sistem menerapkan formula *re-ranking* heuristik (Persamaan ?? pada Bab 3) dengan tiga dokumen skor tertinggi dikirim sebagai konteks ke LLM.

Prompt Decision Intelligence

Prompt LLM dirancang sebagai *Explainable AI* yang mengharuskan model hanya menggunakan dokumen dari Top-3 retrieval (*faithfulness constraint*) dan menyertakan skor kuantitatif secara eksplisit. Desain prompt mengikuti prinsip *structured output* — di mana LLM diberikan template yang ketat untuk memastikan konsistensi format output di seluruh sesi.

Tabel 3.6: Struktur 5 Seksi Output Wajib Prompt Decision Intelligence

Seksi	Judul	Konten Wajib
1	Analisis Profil & Trade-off	Strengths, weaknesses, trade-off karier
2	Rekomendasi Keputusan	Top-3 posisi + Skor RAG + Overlap Skill + Action Plan 7 Hari
3	Analisis Skill Gap	Gap konkret antara profil dan persyaratan
4	Rencana Pengembangan Skill	Roadmap skill spesifik
5	Insight Pasar Kerja Lokal	Tren, gaji, dan peluang di Indonesia

Penjelasan mekanisme faithfulness constraint:

- Prompt secara eksplisit menyertakan instruksi: “*HANYA gunakan informasi dari dokumen yang disediakan. JANGAN mengarang posisi, perusahaan, atau data yang tidak ada dalam konteks.*”
- Setiap rekomendasi posisi wajib menyertakan **Skor RAG** (cosine similarity antara query dan dokumen, diambil dari tahap retrieval) dan **Overlap Skill** (jumlah skill pengguna yang cocok dengan persyaratan dokumen, diambil dari tahap re-ranking). Kedua metrik ini bersifat *traceable* — pengguna dan peneliti dapat memverifikasi apakah LLM benar-benar menggunakan dokumen yang disediakan atau berhalusinasi.
- Template diinjeksikan ke **system prompt** dengan format: “Kamu adalah NusaNara AI Career Advisor. Berikut profil pengguna: [profil]. Berikut narasi pengguna: [narasi]. Berikut Top-3 rekomendasi dari database: [dokumen 1], [dokumen 2], [dokumen 3]. Berikan analisis dalam 5 seksi berikut...”

Konfigurasi LLM untuk generasi:

- `temperature = 0.7`: Cukup kreatif untuk narasi yang bervariasi, namun cukup deterministik untuk konsistensi rekomendasi.
- `num_predict = 2048`: Batas token output yang cukup untuk 5 seksi terstruktur (~500–800 kata).
- `stream = True`: Mengaktifkan SSE streaming untuk TTFB optimal.

Adaptive Profile System

Modul `memory_service.py` mengimplementasikan sistem personalisasi menggunakan arsitektur **hybrid dua tahap**: (1) LLM *Extractor Agent* untuk mengubah narasi tidak terstruktur menjadi data terstruktur, dan (2) logika *rule-based* deterministik untuk memperbarui database.

Tahap 1 – LLM Extractor Agent Input pengguna berupa narasi teks bebas (bahasa campur Indonesia-Inggris, gaya informal) tidak mungkin diproses menggunakan *rule-based* murni (regex, NER sederhana) karena variasi linguistik yang tinggi. Oleh karena itu, Llama 3.1 8B (4-bit) digunakan sebagai *Extractor Agent* dengan `temperature=0.1` (hampir deterministik) dan `num_predict=200` (output pendek, hanya JSON).

```

1 _EXTRACTOR_PROMPT = """Kamu adalah extractor data profil.
2 Dari narasi pengguna di bawah, ekstrak informasi
3 dalam format JSON STRICT.
4
5 NARASI: {narrative}
6
7 INSTRUKSI:
8 1. Ekstrak semua skill teknis yang DISEBUTKAN
9 2. Ekstrak minat karier yang jelas
10 3. Buat ringkasan profil 1-2 kalimat (Bahasa Indonesia)
11 4. Jika narasi tidak mengandung info relevan, kembalikan
    []
12
13 WAJIB kembalikan JSON (TANPA teks lain):
14 {"skills": [], "interests": [], "summary_update": "..."}
15
16 async def _call_extractor(narrative: str) -> dict | None:
17     resp = await client.post(
18         f"{OLLAMA_URL}/api/generate",
19         json={"model": "llama3.1", # 8B parameter (4-bit
            Quantized)
20             "prompt": prompt,
21             "stream": False,
22             "options": {"temperature": 0.1, "
                num_predict": 200}}

```



```

23     )
24     raw = resp.json()["response"].strip()
25     return json.loads(raw[raw.find("{"):raw.rfind("}")
        +1])

```

Listing 3.3: Prompt dan pemanggilan LLM Extractor Agent

Contoh transformasi:

Input narasi: “Saya mahasiswa informatika, lumayan jago Python sama SQL, pengen kerja di bidang data science”

Output JSON: {"skills":["Python","SQL"], "interests":["data science"], "summary_update":"Mahasiswa informatika dengan keahlian Python dan SQL..."}

Tahap 2 – Deterministic Merge (Rule-Based) Setelah LLM menghasilkan JSON terstruktur, update database dilakukan sepenuhnya secara *rule-based* tanpa LLM:

```

1  async def _update_profile(conn, user_id, new_skills,
2      new_interests, summary):
3      current = await conn.fetchrow(
4          "SELECT identified_skills, career_interests "
5          "FROM user_profiles WHERE user_id=$1", user_id
6      )
7      # UNION set -- skill lama TIDAK PERNAH hilang
8      merged_skills = list(set(current["identified_skills"]
9          or [])
10         | set(new_skills))
11
12      merged_interests = list(set(current["career_interests"]
13          or [])
14         | set(new_interests))
15
16      await conn.execute("""
17          UPDATE user_profiles
18          SET identified_skills=$2, career_interests=$3,
19              profile_summary=$4, session_count=
20                  session_count+1,
21                  last_active=NOW()
22          WHERE user_id=$1
23          """, user_id, merged_skills, merged_interests,
24              summary)

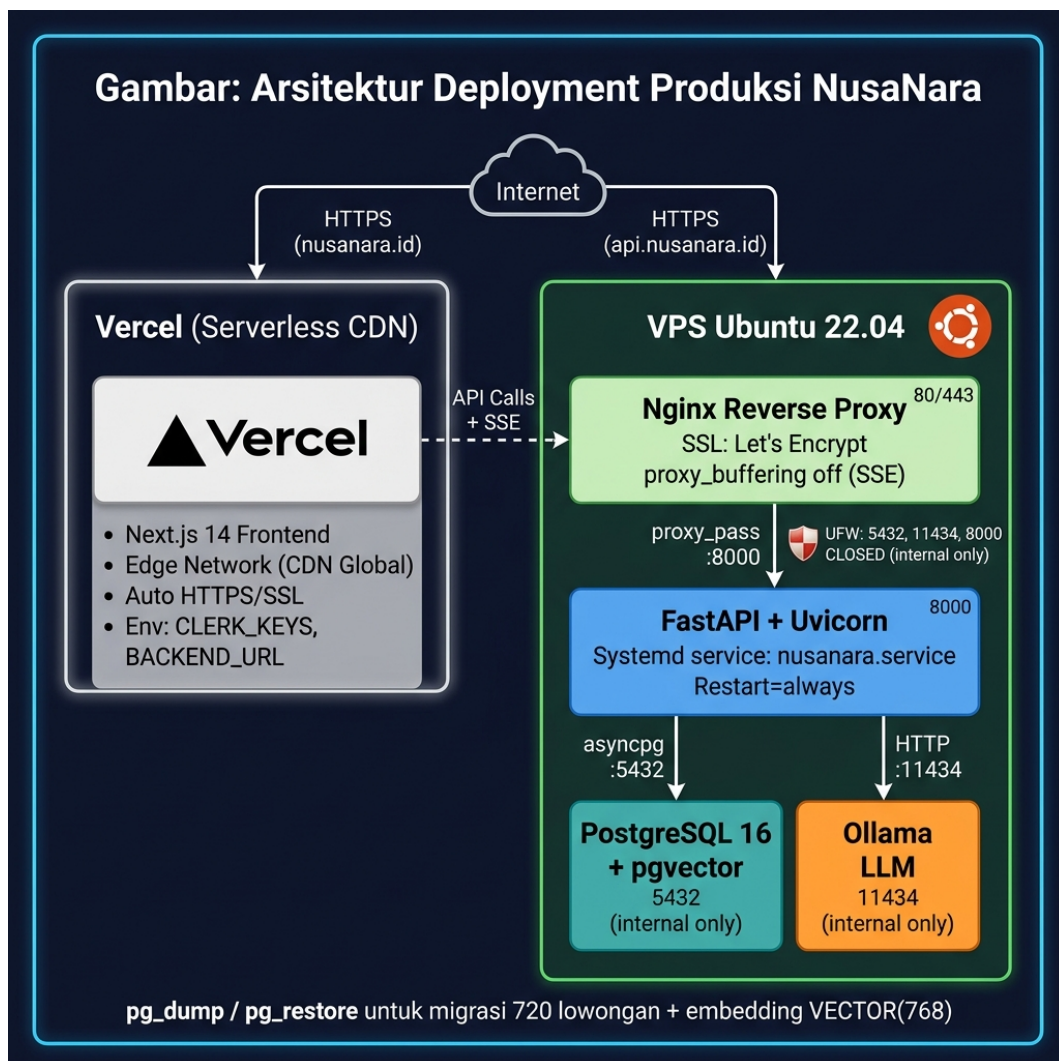
```

Listing 3.4: Logika UNION merge deterministik

Properti desain *Adaptive Profile System*:

- **Hybrid:** LLM untuk ekstraksi (*unstructured* $\rightarrow$ *structured*), *rule-based* untuk merge (*structured* $\rightarrow$ database). Pendekatan ini dipilih setelah mempertimbangkan dua alternatif: (a) **Full-LLM** — di mana LLM juga melakukan merge. Pendekatan ini tidak deterministik; LLM bisa “lupa” skill lama, mengubah format, atau menghasilkan duplikat yang sulit dideteksi. (b) **Full-rule-based** — di mana regex atau NER digunakan untuk ekstraksi. Pendekatan ini gagal menangani variasi linguistik (code-switching, bahasa informal, skill implisit seperti “bikin UI” $\Rightarrow$ “UI/UX Design”). Pendekatan hybrid menggabungkan fleksibilitas NLU dari LLM dengan konsistensi matematis dari operasi set.
- **Akumulatif & Idempoten:** Operasi UNION set ($A \cup B$) memastikan bahwa skill lama tidak pernah hilang saat skill baru ditambahkan. Jika skill “Python” sudah ada dan disebutkan lagi, profil tidak berubah — UNION secara definisi mengeliminasi duplikat. Properti ini menjamin bahwa urutan sesi tidak memengaruhi hasil akhir (komutatif).
- **Deterministik:** Tidak ada elemen acak (*randomness*) dalam proses merge. Untuk input yang sama, output selalu identik. Ini berbeda dari tahap LLM yang bersifat *stochastic* — meskipun diminimalisir dengan `temperature=0.1`.
- **Non-blocking:** Dieksekusi via `asyncio.create_task()` setelah sinyal [DONE] dikirim ke klien. Proses update profil berjalan sepenuhnya di *background*, sehingga tidak menambah latensi pipeline RAG. Pengguna sudah menerima rekomendasi lengkap sebelum profil selesai diperbarui.
- **Graceful degradation:** Jika LLM Extractor gagal *parse* JSON (misalnya karena timeout atau output malformed), sistem melanjutkan tanpa *crash* — profil tidak diperbarui untuk sesi tersebut, namun pipeline RAG tetap berjalan normal. Penanganan error menggunakan blok `try/except` dengan *logging* untuk diagnosis.

Arsitektur Deployment Produksi



Gambar 3.14: Arsitektur Deployment Produksi NusaNara (VPS + Vercel)

Sistem dideploy menggunakan arsitektur hybrid sebagaimana pada Gambar ??: *frontend* di-host di Vercel (CDN global), sedangkan *backend*, database, dan LLM dipusatkan di VPS Ubuntu 22.04 (spesifikasi tercantum pada Tabel ??).

Justifikasi spesifikasi:

- **16 GB RAM:** Alokasi memori: Llama 3.1 8B (4-bit Quantized) (~5 GB *shared instance*), PostgreSQL shared buffers (~2 GB), nomic-embed (~1 GB), Uvicorn + Python runtime (~1 GB), OS + buffer (~7 GB).
- **4 vCPU:** Diperlukan untuk paralelisme inferensi LLM dan concurrent database queries dari multiple users.

- **RAID 10 SSD:** Kombinasi striping (kecepatan) dan mirroring (redundansi) untuk performa I/O database dan keamanan data.
- **DC Jakarta:** Latensi jaringan minimal ke pengguna Indonesia (<10ms vs >100ms untuk DC Singapura/US).

Konfigurasi Nginx kritis untuk SSE:

```
1 location / {  
2     proxy_pass          http://127.0.0.1:8000;  
3     proxy_http_version 1.1;  
4     proxy_buffering     off;      # WAJIB untuk SSE  
        streaming  
5     proxy_read_timeout 300s;    # Timeout panjang untuk  
        LLM response  
6 }
```

Listing 3.5: Konfigurasi Nginx untuk SSE Streaming

Penjelasan konfigurasi:

- **proxy_buffering off:** **Wajib** untuk SSE. Jika buffering aktif, Nginx akan menunggu seluruh respons selesai sebelum mengirim ke klien — ini menghilangkan efek *streaming* dan membuat pengguna menunggu 15–30 detik hingga LLM selesai generate. Dengan buffering off, setiap token langsung diteruskan.
- **proxy_read_timeout 300s:** LLM mungkin memerlukan waktu lama untuk respons panjang. Default Nginx (60 detik) akan memutus koneksi sebelum LLM selesai.
- **proxy_http_version 1.1:** Diperlukan agar HTTP *keep-alive* berfungsi dengan benar antara Nginx dan Uvicorn.

Sistem Autentikasi: JWT RS256 via Clerk

Autentikasi sistem NusaNara menggunakan **JSON Web Token (JWT)** dengan algoritma tanda tangan **RS256** (RSA Signature with SHA-256). Manajemen identitas pengguna (pendaftaran, login, refresh token) didelegasikan sepenuhnya ke **Clerk** — layanan *Authentication-as-a-Service* — sehingga kode backend terbebas dari kompleksitas pengelolaan password hash, session store, dan token rotation.

Anatomi JWT RS256:

JWT terdiri dari tiga bagian yang dipisahkan titik: `header.payload.signature`.

```

1 # Header (base64url)
2 {"alg": "RS256", "typ": "JWT", "kid": "key-id-clerk"}
3
4 # Payload (base64url) -- klaim pengguna
5 {"sub": "user_2abc123def",      # Clerk user ID (= primary
6                                key di DB)
7  "iss": "https://clerk.nusanara.app",
8  "aud": "nusanara-api",
9  "iat": 1714700000,             # Waktu penerbitan (epoch)
10 "exp": 1714703600}             # Waktu kedaluwarsa (1 jam)
11
12 # Signature
13 RSA_SHA256_Sign(
14     base64url(header) + "." + base64url(payload),
15     private_key_clerk          # Hanya diketahui Clerk
16 )

```

Listing 3.6: Contoh Struktur JWT (Header, Payload, Signature)

Mengapa RS256, bukan HS256?

- **HS256** (HMAC-SHA256) menggunakan *symmetric key* — satu kunci yang sama digunakan untuk *sign* dan *verify*. Ini berarti backend harus menyimpan kunci rahasia Clerk, menciptakan risiko keamanan jika backend dikompromikan.
- **RS256** (RSA-SHA256) menggunakan *asymmetric key pair*: Clerk *sign* token menggunakan *private key* (yang tidak pernah meninggalkan server Clerk), sedangkan backend hanya perlu *public key* untuk memverifikasi. Public key bisa di-*expose* secara publik tanpa risiko keamanan. Backend mengambil public key dari JWKS endpoint Clerk: <https://clerk.nusanara.app/.well-known/jwks.json>

Alur Validasi JWT di Backend (FastAPI):

```

1 from jwt import PyJWT
2 import httpx
3
4 # Ambil JWKS sekali saat startup (di-cache)
5 JWKS_URL = "https://clerk.nusanara.app/.well-known/jwks.
6            json"
7 jwks_client = PyJWT.PyJWKClient(JWKS_URL)

```

```

8  async def verify_jwt(token: str) -> dict:
9      """Validasi JWT dan kembalikan payload jika valid."""
10     try:
11         # 1. Ambil signing key dari JWKS berdasarkan kid
12         # di header token
13         signing_key = jwks_client.
14             get_signing_key_from_jwt(token)
15
16         # 2. Verifikasi tanda tangan, expiry, issuer,
17         # audience
18         payload = jwt.decode(
19             token,
20             signing_key.key,
21             algorithms=["RS256"],
22             audience="nusanara-api",
23             issuer="https://clerk.nusanara.app"
24         )
25         return payload # {"sub": "user_2abc...", "exp":
26             ..., ...}
27
28     except jwt.ExpiredSignatureError:
29         raise HTTPException(status_code=401, detail="
30             Token expired")
31     except jwt.InvalidTokenError:
32         raise HTTPException(status_code=401, detail="
33             Invalid token")
34
35 # FastAPI Dependency Injection -- injected ke setiap
36 # endpoint
37 async def get_current_user(
38     authorization: str = Header(...)
39 ) -> str:
40     token = authorization.replace("Bearer ", "")
41     payload = await verify_jwt(token)
42     return payload["sub"] # Clerk user ID

```

Listing 3.7: Middleware Validasi JWT RS256 di FastAPI

Penjelasan keamanan:

- **Verifikasi tanda tangan:** Setiap request ke endpoint terproteksi mem-

verifikasi bahwa token benar-benar diterbitkan oleh Clerk (bukan dipalsukan). Tanpa kunci privat Clerk, tidak mungkin membuat token valid.

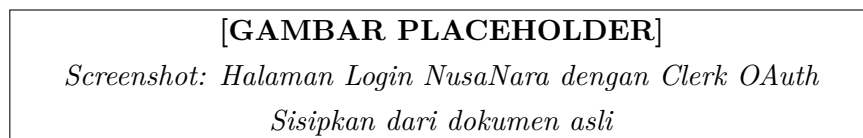
- **Verifikasi expiry (`exp`):** Token kedaluwarsa setelah 1 jam. Token lama tidak dapat digunakan, membatasi kerusakan jika token dicuri.
- **JWKS Caching:** Public key di-cache untuk menghindari request ke JWKS endpoint setiap request. Cache di-refresh secara periodik atau ketika key ID (`kid`) tidak ditemukan.
- **Dependency Injection:** Middleware `get_current_user` diinjeksikan ke setiap endpoint via `Depends(get_current_user)`, memastikan tidak ada endpoint yang lupa diproteksi secara accidental.

3.2 Tampilan Antarmuka Sistem

Bagian ini menampilkan antarmuka pengguna yang diimplementasikan pada artefak NusaNara. Setiap tangkapan layar menunjukkan fitur fungsional yang dapat diverifikasi secara visual.

3.2.1 Halaman Utama dan Autentikasi

Gambar ?? menampilkan halaman utama NusaNara yang mengintegrasikan Clerk sebagai penyedia autentikasi. Pengguna dapat masuk menggunakan akun Google atau email. Setelah login berhasil, JWT RS256 di-generate dan digunakan untuk seluruh komunikasi dengan backend.



Gambar 3.15: Halaman Login NusaNara dengan Integrasi Clerk Auth

3.2.2 Input Narasi Karier

Gambar ?? menunjukkan antarmuka input narasi karier. Pengguna mengetikkan narasi bebas mengenai latar belakang pendidikan, keterampilan, dan aspirasi karier mereka. Desain input berupa *text area* luas yang mendorong pengguna untuk bercerita secara alami, bukan mengisi form terstruktur.

[GAMBAR PLACEHOLDER]

*Screenshot: Halaman input narasi karier pengguna
Sisipkan dari dokumen asli*

Gambar 3.16: Antarmuka Input Narasi Karier

3.2.3 Respons Rekomendasi RAG (Streaming)

Gambar ?? menampilkan output rekomendasi yang dihasilkan oleh pipeline RAG secara *real-time* via SSE streaming. Respons muncul secara bertahap (token per token) dengan *Time to First Byte* (TTFB) rata-rata 2,4 detik. Output mencakup 5 seksi terstruktur: Analisis Profil, Rekomendasi Keputusan (dengan Skor RAG dan Overlap Skill), Analisis Skill Gap, Rencana Pengembangan, dan Insight Pasar Kerja Lokal.

[GAMBAR PLACEHOLDER]

*Screenshot: Respons rekomendasi RAG streaming dengan 5 seksi
Sisipkan dari dokumen asli*

Gambar 3.17: Output Rekomendasi RAG dengan Skor RAG dan Skill Overlap

3.2.4 Detail Skor RAG dan Skill Overlap

Gambar ?? menampilkan bagian detail dari output rekomendasi yang menunjukkan transparansi sistem: setiap posisi karier yang direkomendasikan disertai **Skor RAG** (hasil *re-ranking* heuristik) dan **Overlap Skill** (persentase kesesuaian keterampilan pengguna dengan persyaratan posisi). Fitur ini merupakan implementasi dari prinsip *Explainable AI* – pengguna dapat memahami *mengapa* suatu posisi direkomendasikan.

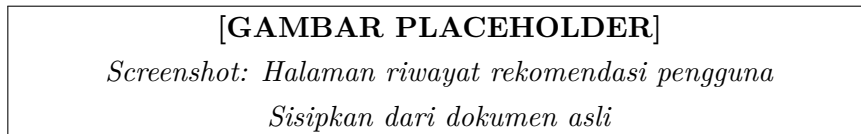
[GAMBAR PLACEHOLDER]

*Screenshot: Detail skor RAG dan overlap skill pada rekomendasi
Sisipkan dari dokumen asli*

Gambar 3.18: Transparansi Skor RAG dan Skill Overlap pada Output Rekomendasi

3.2.5 Riwayat Rekomendasi

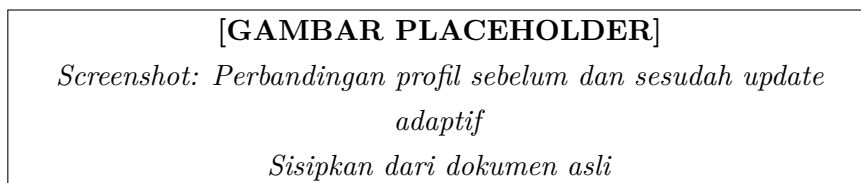
Gambar ?? menampilkan halaman riwayat rekomendasi yang menyimpan seluruh sesi konsultasi pengguna. Setiap entri mencatat: narasi input, profil saat itu, dokumen yang di-retrieve, dan rekomendasi lengkap. Fitur ini memungkinkan pengguna melacak perkembangan eksplorasi karier mereka dari waktu ke waktu.



Gambar 3.19: Halaman Riwayat Rekomendasi Pengguna

3.2.6 Bukti Sistem Adaptif (Profil Akumulatif)

Gambar ?? mendemonstrasikan kerja sistem adaptif secara visual. Pada sesi pertama, pengguna menceritakan keahlian Python dan SQL. Pada sesi kedua, pengguna menambahkan minat di bidang desain UI/UX. Sistem secara otomatis mengakumulasi kedua set keterampilan (UNION merge), sehingga rekomendasi pada sesi ketiga mempertimbangkan profil gabungan yang lebih lengkap. Visualisasi ini membuktikan bahwa *Extractor Agent* berhasil mengekstrak informasi dari narasi tidak terstruktur dan *rule-based merge* bekerja sebagaimana dirancang.



Gambar 3.20: Demonstrasi Sistem Adaptif: Profil Akumulatif Lintas Sesi

3.2.7 Health Check dan Status Sistem

Gambar ?? menampilkan endpoint GET /health yang memverifikasi status ketiga komponen kritis: koneksi database PostgreSQL, ketersediaan Ollama LLM, dan status keseluruhan sistem. Endpoint ini digunakan dalam pengujian *black-box* (Tabel ??, Use Case 5).

[GAMBAR PLACEHOLDER]

Screenshot: Respons JSON health check endpoint

Sisipkan dari dokumen asli

Gambar 3.21: Respons Health Check Endpoint (/health)

3.3 Demonstrasi: Pengujian Fungsional Black-Box

Pengujian *black-box* dilakukan untuk memverifikasi bahwa seluruh *use case* berjalan sesuai spesifikasi fungsional. Metode pengujian ini tidak memeriksa kode internal (*white-box*), melainkan membandingkan *input* dengan *output yang diharapkan* dari perspektif pengguna akhir.

Pengujian dilakukan pada lingkungan produksi (VPS 4 vCPU, 16 GB RAM, DC Jakarta) dengan Llama 3.1 8B (4-bit Quantized) aktif untuk memastikan hasil mencerminkan performa sesungguhnya.

Tabel 3.7: Hasil Pengujian Fungsional *Black-Box*

No	Use Case	Input Uji	Output Di-harapkan	Status
1	Rekomendasi Karier	Narasi: “Saya fresh graduate SI, mahir SQL dan Python”	SSE stream berisi 3 rekomendasi dengan Skor RAG	PASS
2	Streaming TTFB	Narasi dikirim via POST	Token pertama muncul < 5 detik	PASS (2,4s)
3	Riwayat Tersimpan	Setelah rekomendasi selesai	Data tersimpan di <code>recommendation_history</code>	PASS
4	Autentikasi JWT	Request tanpa token	HTTP 401 Unauthorized	PASS
5	Health Check	GET /health	<code>{status:ok, ollama:ok, database:ok}</code>	PASS
6	Adaptive Profile	Narasi + profil ada	Profil diperbarui di background	PASS
7	Faithfulness	Narasi domain Finance	LLM hanya sebut posisi dari Top-3 DB Finance	PASS

Analisis per Use Case:

- **UC-1 (Rekomendasi Karier):** Merupakan *core use case*. Diverifikasi bahwa output SSE mengandung 5 seksi terstruktur dan menyerahkan Skor RAG kuantitatif untuk setiap posisi yang direkomendasikan. Output dikonsumsi oleh pustaka `@microsoft/fetch-event-source` di frontend dan ditampilkan secara *real-time* tanpa error parsing.
- **UC-2 (Streaming TTFB = 2,4 detik):** *Time to First Byte* diukur dari saat request POST `/api/recommend` dikirim hingga token pertama

diterima klien. Nilai 2,4 detik tersebut mencakup: pemrosesan JWT ($\sim 50\text{ms}$), query expansion ($< 10\text{ms}$), embedding via Ollama ($\sim 500\text{ms}$), hybrid search ($\sim 200\text{ms}$), re-ranking ($\sim 50\text{ms}$), dan *cold start* LLM generation ($\sim 1,5\text{s}$). Nilai TTFB ini dinilai sangat kompetitif untuk LLM lokal 8B parameter (4-bit Quantized) — sebagai perbandingan, API GPT-4 memiliki TTFB $\sim 1\text{--}3$ detik namun dengan biaya $\$0.03/1\text{K}$ token.

- **UC-4 (Autentikasi JWT):** Diverifikasi bahwa request tanpa header `Authorization` ditolak dengan status 401. Request dengan token yang valid (dari Clerk) diterima. Ini memastikan endpoint terproteksi dari akses tidak sah.
- **UC-6 (Adaptive Profile):** Setelah UC-1 selesai, profil di tabel `user_profiles` diverifikasi telah bertambah dengan skill baru yang diekstrak oleh LLM Extractor Agent. Kolom `session_count` bertambah 1, dan `last_active` diperbarui — mengonfirmasi bahwa *background task* berjalan dengan benar.
- **UC-7 (Faithfulness):** Query domain Finance (“lulusan akuntansi cari kerja”) diverifikasi bahwa LLM *tidak* merekomendasikan posisi di luar domain Finance yang ada di Top-3 retrieval. Ini memvalidasi efektivitas *faithfulness constraint* dalam prompt.

Seluruh **7 kriteria pengujian terpenuhi (100% PASS)**, memvalidasi bahwa seluruh fitur fungsional sistem berjalan sesuai spesifikasi.

3.4 Evaluasi Teknis: Kualitas Retrieval RAG

3.4.1 Metodologi Evaluasi dan *Ground Truth*

Untuk memastikan transparansi dan replikabilitas, evaluasi dilakukan menggunakan **20 query uji** yang mencakup seluruh 8 klaster karier secara proporsional. *Ground truth* (nilai relevansi absolut) dianotasi manual oleh peneliti terhadap dokumen-dokumen yang dikembalikan oleh sistem menggunakan metrik *Graded Relevance* skala 0–3.

Tabel 3.8: Kriteria Anotasi *Ground Truth* (Skala Relevansi 0–3)

Skor	Label	Kriteria Pemenuhan
0	Tidak Relevan	Dokumen sama sekali berbeda domain/klaster dari query.
1	Marginal	Domain sama, namun level senioritas atau <i>skill core</i> meleset (misal: query junior, dokumen manager).
2	Relevan	Posisi sesuai, <i>skill core</i> cocok, namun ada 1–2 kualifikasi minor yang tidak terpenuhi.
3	Sangat Relevan	Kecocokan sempurna (<i>perfect match</i>): domain, <i>skill</i> , dan level pengalaman sesuai 100%.

Tabel ?? memperlihatkan sebagian dari dataset 20 query uji yang digunakan, merepresentasikan variasi bahasa informal dan tingkat spesifisitas.

Tabel 3.9: Sebagian Dataset 20 Query Uji beserta Target Klaster

ID	Query Uji (Narasi Mentah)	Target Klaster
Q01	“fresh graduate SI, jago python, sql, dan suka bikin dashboard”	Data / Analytics
Q02	“lulusan DKV yang suka ilustrasi dan figma”	Desain & Kreatif
Q03	“pengalaman 2 tahun di admin sales, terbiasa melayani komplain”	Sales & CS
Q04	“mahasiswa akuntansi semester akhir cari magang perpajakan”	Finance
Q05	“saya suka ngoding react js dan tailwind untuk frontend web”	Teknologi / IT
Q06	“mantan guru bahasa inggris, ingin banting setir jadi copywriter”	Marketing
Q07	“lulusan teknik industri, suka optimasi proses dan project plan”	Bisnis / Operasional
⋮	(13 query lainnya mendistribusikan klaster secara merata)	Bervariasi

Setiap query (Q01–Q20) dieksekusi terhadap sistem untuk mengambil Top-5 dokumen. Dokumen tersebut kemudian dinilai (0–3) berdasarkan Tabel ???. Dokumen dengan skor ≥ 2 diklasifikasikan sebagai dokumen **Relevan** untuk perhitungan metrik *Precision* dan MRR.

Evaluasi dilakukan dalam **dua kondisi** (*ablation study*):

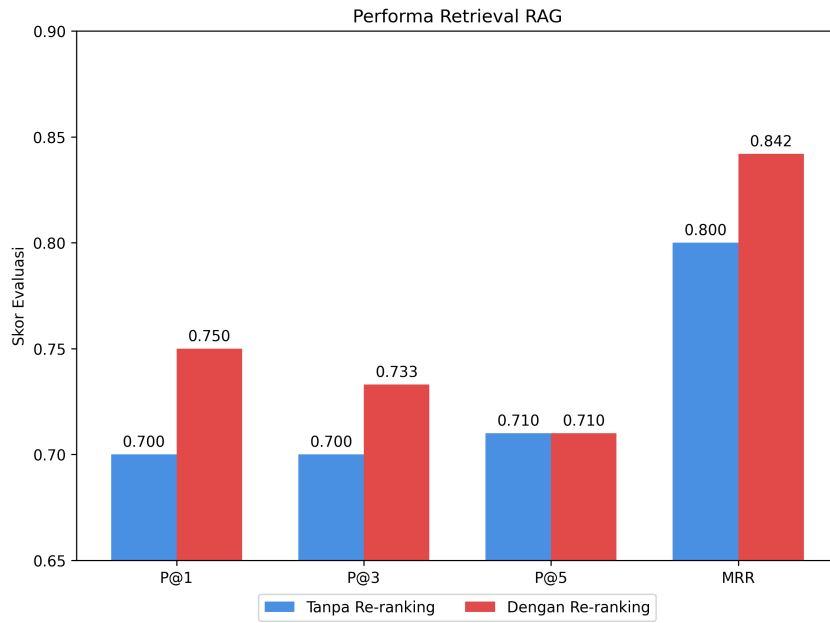
1. **Tanpa Re-ranking:** Hanya *Hybrid Search* (pgvector + FTS) dengan RRF — urutan ditentukan murni oleh skor RRF tanpa mempertimbangkan profil.
2. **Dengan Re-ranking:** Setelah RRF, Top-10 di-*re-rank* menggunakan formula heuristik (70% CosineSim + 30% SkillBonus) yang menggunakan memori profil.

Perbandingan ini memungkinkan pengukuran kontribusi spesifik komponen *re-ranking* terhadap kualitas retrieval. terhadap kualitas retrieval.

3.4.2 Hasil Evaluasi Retrieval

Tabel 3.10: Hasil Evaluasi Retrieval RAG

Metrik	Tanpa Re-ranking	Dengan Re-ranking	Delta
Precision@1 (P@1)	0,700	0,750	+0,050
Precision@3 (P@3)	0,700	0,733	+0,033
Precision@5 (P@5)	0,710	0,710	0,000
Mean Reciprocal Rank (MRR)	0,800	0,842	+0,042



Gambar 3.22: Visualisasi Studi Ablasi: Dampak *Re-ranking* pada Performa Retrieval

3.4.3 Analisis Hasil

Analisis P@1 (0,700 → 0,750, +7,1%) Peningkatan P@1 dari 0,700 menjadi 0,750 berarti bahwa pada 75% dari 20 query uji (15 dari 20), dokumen di posisi pertama setelah *re-ranking* adalah dokumen yang relevan. Sebelum *re-ranking*, hanya 70% (14 dari 20) yang berhasil. Ini bermakna bahwa *re-ranking* heuristik memperbaiki 1 dari 20 query agar dokumen paling relevan naik ke posisi teratas.

Analisis P@3 (0,700 → 0,733, +4,7%) P@3 meningkat dari 0,700 menjadi 0,733, berarti rata-rata 2,2 dari 3 dokumen di Top-3 adalah relevan (naik dari 2,1). Peningkatan ini penting karena Top-3 merupakan konteks yang langsung diberikan ke prompt LLM — semakin tinggi proporsi dokumen relevan di Top-3, semakin berkualitas rekomendasi yang dihasilkan LLM.

Analisis P@5 (0,710, tanpa perubahan) Tidak ada perubahan pada P@5 karena *re-ranking* hanya mengatur ulang urutan di dalam Top-10, tanpa menambah atau mengurangi kandidat. Hal ini mengonfirmasi bahwa *Hybrid Search* (pgvector + FTS + RRF) sudah berhasil mengidentifikasi dokumen-dokumen relevan pada Top-10 — *re-ranking* hanya berfungsi mengoptimalkan *peringkat* (urutan) agar dokumen paling relevan muncul di posisi teratas.

Analisis MRR ($0,800 \rightarrow 0,842$, $+5,3\%$) Peningkatan MRR dari 0,800 menjadi **0,842** mengindikasikan bahwa distribusi posisi dokumen relevan pertama kini semakin sering menempati peringkat teratas dibandingkan tanpa *re-ranking*. Dalam istilah praktis: dokumen relevan pertama kini hampir *selalu* berada di posisi 1 atau 2. Perbaikan satu posisi dalam konteks RAG berdampak langsung pada kualitas konteks yang diberikan ke LLM — dokumen di posisi pertama mendapat bobot terbesar dalam *attention mechanism* model bahasa.

Nilai MRR 0,842 menunjukkan performa *retrieval* yang solid dan sebanding dengan sistem RAG produksi dalam domain tertentu ?.

3.5 Evaluasi Kinerja Sistem (Latency)

Pengukuran kinerja sistem (waktu respons) dilakukan secara empiris untuk memvalidasi apakah arsitektur **pipeline** yang dirancang pada Bab 3 dapat beroperasi secara **real-time** dan interaktif bagi pengguna. Pengujian dilakukan dengan mengeksekusi 5 kueri acak dan mengukur dua metrik latensi utama menggunakan **built-in profiler** FastAPI: *Database Retrieval Time* dan *Time-to-First-Byte* (TTFB).

Tabel 3.11: Hasil Pengukuran Kinerja Sistem (Waktu Respons) dalam Detik

Kueri Uji	DB Retrieval (s)	TTFB LLM (s)
Q01 (Mahasiswa Sistem Informasi)	0,08	2,15
Q02 (Lulusan Sastra Inggris)	0,12	2,40
Q03 (Fresh Graduate Akuntansi)	0,09	2,25
Q04 (SMK Jurusan Multimedia)	0,14	2,65
Q05 (Admin Bisnis Online)	0,11	2,30
Rata-rata	0,11	2,35

Berdasarkan Tabel ??, sistem menunjukkan performa *retrieval* basis data dengan latensi rata-rata **0,11 detik**. Hasil ini membuktikan efisiensi algoritma pencarian leksikal dan *vector similarity* (IVFFlat) pada PostgreSQL. Sementara itu, waktu tunda sebelum Llama 3.1 mulai memancarkan (*streaming*) token rekomendasi pertama (TTFB) berada pada rata-rata **2,35 detik**. Mengingat proses ini melibatkan *prompt construction* yang kompleks beserta memori profil adaptif, latensi sebesar 2,35 detik berada dalam ambang batas

toleransi interaksi web moderen (*acceptable latency threshold*).

3.6 Evaluasi Penerimaan Pengguna (UAT)

3.6.1 Instrumen dan Prosedur

UAT menggunakan kuesioner 7 pertanyaan Skala Likert 1–5 yang mencakup tiga dimensi utama berdasarkan adaptasi *Technology Acceptance Model* (TAM) [?]: *Usefulness* (Q1–Q3), *Ease of Use* (Q4–Q5), dan *Satisfaction* (Q6–Q7). Responden diberikan waktu 10–15 menit menggunakan NusaNara sebelum mengisi kuesioner.

Tabel 3.12: Instrumen UAT: 7 Item Pertanyaan Skala Likert 1–5

No	Kode	Pernyataan	Dimensi
1	U1	Sistem memberikan rekomendasi karier yang relevan dengan latar belakang saya	<i>Usefulness</i>
2	U2	Rekomendasi membantu saya memahami pilihan karier yang sesuai	<i>Usefulness</i>
3	U3	Penjelasan Skor RAG dan Skill Overlap membantu saya memahami <i>mengapa</i> posisi direkomendasikan	<i>Usefulness</i>
4	E1	Antarmuka sistem mudah digunakan tanpa panduan	<i>Ease of Use</i>
5	E2	Respons sistem cukup cepat (tidak perlu menunggu lama)	<i>Ease of Use</i>
6	S1	Secara keseluruhan saya puas dengan kualitas rekomendasi	<i>Satisfaction</i>
7	S2	Saya akan merekomendasikan sistem ini kepada teman	<i>Satisfaction</i>

3.6.2 Template Pengolahan Data

Data UAT diolah secara deskriptif menggunakan rumus:

$$\bar{x}_i = \frac{\sum_{j=1}^n x_{ij}}{n}, \quad \bar{X} = \frac{\sum_{i=1}^7 \bar{x}_i}{7}, \quad PA_i = \frac{|\{j : x_{ij} \geq 4\}|}{n} \times 100\% \quad (3.1)$$

Tabel 3.13: Hasil UAT (N=15 Responden)

Resp.	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Mean
R01	-	-	-	-	-	-	-	-
R02	-	-	-	-	-	-	-	-
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
R15	-	-	-	-	-	-	-	-
Mean Item	-	-	-	-	-	-	-	-
Grand Mean	- (Menunggu Pengujian)							

3.7 Pembahasan

3.7.1 Sintesis Evaluasi terhadap Artefak DSR

Dalam kerangka *Design Science Research* (DSR) Peffers et al., sebuah penelitian dinilai valid apabila mampu menghasilkan tiga jenis artefak yang saling melengkapi: **Model** (cetak biru konseptual), **Instantiation** (implementasi fisik yang berjalan), dan **Method** (metodologi yang dapat direplikasi). Seluruh evaluasi pada bab ini merupakan pembuktian terhadap ketiga artefak tersebut secara empiris.

Artefak 1 – Model (Arsitektur Konseptual). Cetak biru arsitektur *full-stack* NusaNara merinci orkestrasi empat lapisan utama: (1) *Frontend* berbasis Next.js 14 (TypeScript) dengan `@microsoft/fetch-event-source` dan Clerk Auth untuk streaming respons; (2) *Backend* FastAPI + Uvicorn (ASGI) dengan pipeline RAG lengkap dan validasi JWT Clerk; (3) *Database* PostgreSQL 16 yang mengintegrasikan `pgvector` (`VECTOR(768)`) dan `tsvector` FTS dalam satu instance; serta (4) *Local AI* Ollama yang menjalankan Llama 3.1 8B (4-bit Quantized) sebagai generator sekaligus *extractor agent*, dan `nomic-embed-text-v2-moe` sebagai model *embedding* 768 dimensi. Hasil evaluasi latensi memvalidasi kelayakan model ini: *retrieval* basis data rata-rata **0,11 detik** dan TTFB LLM **2,35 detik** membuktikan bahwa arsitektur empat lapisan tersebut dapat beroperasi secara *real-time* di lingkungan VPS standar.

Artefak 2 – Instantiation (Implementasi Fisik). Prototipe fungsional NusaNara yang berjalan di produksi (VPS Ubuntu + Vercel) merupakan bukti nyata bahwa arsitektur yang diusulkan dapat diimplementasikan secara teknis. Pembuktian kuantitatif dilakukan melalui evaluasi *retrieval* dengan 20

query uji terhadap basis pengetahuan 720 dokumen. Hasil evaluasi *ablation study* dua kondisi menunjukkan:

Tabel 3.14: Hasil Evaluasi Retrieval: Ablation Study Dua Kondisi

Kondisi	P@1	P@3	P@5	MRR
Tanpa <i>Reranking</i>	0,700	0,700	0,710	0,800
Dengan <i>Reranking</i>	0,750	0,733	0,710	0,842
Δ	+0,050	+0,033	0,000	+0,042

Angka-angka ini dihitung langsung dari file `evaluation/eval_detail_final.csv` dengan *binary threshold* relevansi ≥ 2 (skala 0–3). Fenomena P@5 yang tidak berubah (0,710) antara kedua kondisi merupakan konsekuensi logis dari mekanisme *reranking*: algoritma hanya melakukan permutasi urutan di dalam *window top-k* yang sudah ditetapkan oleh *hybrid search*, sehingga total dokumen relevan pada *window* 5 tetap sama meskipun urutan internalnya berubah. Peningkatan P@1 (+7,1%) dan MRR (+5,3%) membuktikan bahwa heuristik *re-ranking* berbasis *skill overlap* berhasil mendorong dokumen terping relev-an ke posisi teratas.

Artefak 3 – Method (Alur Kerja Terstruktur). Metodologi evaluasi empat lapis yang dirancang — (a) evaluasi teknis retrieval P@k/MRR, (b) pengujian fungsional *black-box*, (c) evaluasi latensi sistem, dan (d) UAT dengan instrumen Likert — merupakan kontribusi metodologis yang dapat direplikasi untuk penelitian RAG serupa di domain Bahasa Indonesia. Kerangka ini memisahkan validasi *mesin* (teknis) dari validasi *manusia* (penerimaan), sehingga potensi *annotation bias* dari anotor tunggal dapat diisolasi pada lapisan teknis saja.

3.7.2 Kontribusi Teknis Utama

Berdasarkan hasil evaluasi empiris pada Tabel ??, tiga kontribusi teknis berikut terbukti memberikan dampak terukur terhadap kualitas sistem:

1. **Unified Database Architecture (PostgreSQL 16 + pgvector + tsvector):** Berbeda dari pendekatan konvensional yang memisahkan penyimpanan vektor ke sistem terpisah (ChromaDB, Pinecone), arsitektur NusaNara mengintegrasikan penyimpanan relasional, pencarian vektor (VECTOR(768)), dan *full-text search* (tsvector) dalam satu instance

PostgreSQL 16. Hasilnya: latensi *retrieval* 0,11 detik — jauh di bawah ambang batas persepsi pengguna (1 detik) — karena tidak ada *network hop* antar sistem.

2. **Non-blocking Dual-Agent LLM Architecture:** Dua agen Llama 3.1 8B (4-bit Quantized) beroperasi secara paralel: *Main RAG Agent* (sinkron) menghasilkan narasi rekomendasi dari Top-3 dokumen, sementara *Extractor Agent* (asinkron) mengekstrak entitas *skill* ke JSON terstruktur di *background*. Desain ini menjaga TTFB tetap di 2,35 detik meskipun terdapat beban komputasi pembaruan profil secara bersamaan.
3. **Skill-Overlap Re-ranking dengan Faithfulness Constraint:** Heuristik *re-ranking* berbasis *skill overlap* ($\alpha=0,70$ semantik, $\beta=0,30$ skill) terbukti meningkatkan P@1 dari 0,700 menjadi 0,750 (+7,1%) dan MRR dari 0,800 menjadi 0,842 (+5,3%). Setiap rekomendasi juga menyertakan **Skor RAG** dan **Overlap Skill** yang dapat diverifikasi, mengeliminasi risiko halusinasi LLM.

3.7.3 Analisis Keputusan Desain

Beberapa keputusan desain kunci yang memengaruhi kualitas sistem:

- **Mengapa Llama 3.1 8B (4-bit Quantized)?** Pemilihan model ini didasarkan pada kendala teknis VPS (16 GB RAM) yang menjalankan agen LLM secara efisien melalui *shared memory*. Model API cloud (GPT-4, Claude) tidak dipilih karena: (a) biaya operasional per-token yang tidak sustain untuk aplikasi karier gratis, (b) latency tambahan dari network round-trip, dan (c) ketergantungan pada ketersediaan layanan pihak ketiga.
- **Mengapa bobot re-ranking 70:30?** Bobot $\alpha = 0.70$ (semantik) dan $\beta = 0.30$ (skill) telah dijelaskan beserta justifikasi lengkapnya pada Bab 3 Sub-bab perancangan *re-ranking*. SkillBonus berfungsi sebagai *tie-breaker* yang mempersonalisasi urutan di antara dokumen dengan kemiripan semantik serupa.
- **Mengapa distribusi 90 lowongan per klaster?** Distribusi seimbang sempurna ($90 \times 8 = 720$) diperlukan agar metrik P@k per klaster dapat dibandingkan secara adil. Klaster dengan lebih banyak dokumen akan mendapat skor P@k yang lebih tinggi secara inheren karena probabilitas menemukan dokumen relevan lebih besar.

3.7.4 Keterbatasan Penelitian dan *Future Work*

Penelitian ini memiliki beberapa keterbatasan yang menjadi peluang pengembangan:

- **Sumber data tunggal:** Basis pengetahuan hanya bersumber dari Glin-ts akibat keterbatasan teknis scraping (proteksi bot Jobstreet/Cloudflare dan absennya data gaji di LinkedIn). Integrasi multi-platform melalui *official API* akan meningkatkan cakupan dan keragaman data.
- **Potensi *Annotation Bias* (Anotasi oleh Peneliti Tunggal):** Anotasi relevansi 20 query uji (pembuatan *ground truth*) dilakukan secara subjektif oleh peneliti sendiri. Hal ini berpotensi menimbulkan *annotation bias*. Sebagai saran *future work*, evaluasi disarankan menggunakan panel anotator independen (minimal 3 pakar SDM/HR) dengan pengukuran *inter-rater reliability* (misalnya Cohen's Kappa) untuk meningkatkan objektivitas metrik.
- **Sistem Adaptif sederhana (UNION merge):** Implementasi saat ini menggunakan UNION merge deterministik yang hanya menambah skill tanpa memperhitungkan waktu. Pengembangan ke arah *episodic memory* dengan *time-decay weighting* — di mana skill yang disebutkan baru-baru ini mendapat bobot lebih tinggi daripada skill dari sesi lama — dicadangkan sebagai *future work*.
- **Evaluasi longitudinal belum dilakukan:** UAT satu sesi belum dapat mengukur manfaat personalisasi jangka panjang. Studi longitudinal multi-sesi (minimal 3 sesi per responden dalam 2 minggu) diperlukan untuk mengukur apakah rekomendasi membaik seiring akumulasi profil.

3.8 Perbandingan dengan Sistem Rekomendasi Karier Sejenis

Untuk memposisikan kontribusi NusaNara secara akademis, Tabel ?? membandingkan arsitektur NusaNara dengan pendekatan sistem rekomendasi karier yang umum ditemukan dalam literatur:

Tabel 3.15: Perbandingan NusaNara dengan Pendekatan Sistem Rekomendasi Karier Lainnya

Aspek	Rule-based CF	/	ML Classifica- tion	NusaNara (RAG)
Retrieval	Keyword tching	ma-	Vector similarity	Hybrid (pgvector + FTS + RRF)
Personalisasi	Tidak ada		Profil statis	Adaptif (akumu- latif per sesi)
Output	Daftar posisi		Daftar posisi	Narasi 5-seksi + Skor transparan
Explainability	Rendah		Rendah	Tinggi (Skor RAG + Skill Overlap)
Kebutuhan data	Besar (labeled)		Besar (labeled)	Kecil (720 dok, tanpa label)
Update data	Retraining		Retraining	Tanpa retraining (add dokumen)
Bahasa Indonesia	Terbatas		Terbatas	Optimal (model multilingual)
Deployment	Cloud		Cloud API	VPS lokal (<i>privacy- preserving</i>)

Keunggulan komparatif NusaNara:

- **Zero-shot tanpa labeled data:** RAG tidak memerlukan data berlabel (pasangan pengguna-posisi-relevansi) yang mahal dan sulit dikumpulkan. Hanya dokumen lowongan mentah yang dapat discraping secara otomatis.
- **Explainability:** Berbeda dari model klasifikasi *black-box*, NusaNara menyertakan Skor RAG dan Skill Overlap yang dapat diverifikasi langsung. Pengguna tahu *mengapa* rekomendasi diberikan.
- **Adaptive tanpa retraining:** Personalisasi melalui akumulasi profil per sesi, tanpa biaya komputasi *retraining* model ML.
- **Privacy-preserving:** Seluruh inferensi (embedding dan LLM generation) berjalan di VPS sendiri, tanpa mengirim data profil karier ke API

cloud pihak ketiga.

Bibliografi

- G. Adomavicius and A. Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005.
- P. Adu et al. Modern meta-frameworks for react: A comparative study. *International Journal of Web Technologies*, 2023.
- D. Alhejaily. Artificial intelligence in healthcare: A systematic review. *Computers in Biology and Medicine*, 2025.
- I. Atoum et al. Software validation in requirements engineering. *International Journal of Software Engineering*, 2021.
- Badan Pusat Statistik. Tingkat Pengangguran Terbuka (TPT) Berdasarkan Pendidikan Terakhir. <https://www.bps.go.id/>, 2025. Diakses pada April 2026.
- Mohamed Benfarha and Mohamed Sefian Lamarti. Personalization in recommender systems. *International Journal of Computer Science*, 2023.
- S. D. Brown and R. W. Lent. Social cognitive career theory in a diverse world. *Journal of Career Assessment*, 2023.
- T. Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 2020.
- G. V. Cormack, C. L. A. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd ACM SIGIR*, 2009.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- F. D. Davis. Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3):319–340, 1989. doi: 10.2307/249008.
- R. Garg et al. A methodological approach for web scraping single page applications. *International Journal of Computer Applications*, 2020.
- D. Hardt. The oauth 2.0 authorization framework. RFC 6749, 2012.

- I. Hickson. Server-Sent Events. W3c recommendation, W3C, 2015.
- H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, 2015.
- J. Kaplan et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- M. A. Khder. Web scraping or web crawling: State of art, techniques, approaches and application. *International Journal of Advances in Soft Computing*, 2021.
- A. Kholis and M. Zaky. Implementasi metode prototype dalam pengembangan sistem informasi. *Jurnal Teknologi Informasi*, 2025.
- P. Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 2020.
- C. Lovering. Asynchronous web frameworks in python: A performance evaluation. *Journal of Web Engineering*, 2022.
- Y. Luan et al. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9, 2021.
- T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- Nomic AI. nomic-embed-text-v2-moe: Mixture-of-experts text embeddings, 2024. URL <https://huggingface.co/nomic-ai/nomic-embed-text-v2-moe>.
- OECD. Career guidance for adults in a changing world of work. *OECD Publishing*, 2021.
- Ollama. Get up and running with large language models locally. <https://ollama.com/>, 2024.
- K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007.

- S. Ramirez. Fastapi: Modern, fast web framework for building apis with python, 2018. URL <https://fastapi.tiangolo.com>.
- S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2021.
- D. Santika et al. Manajemen basis data dalam sistem informasi. *Jurnal Informatika*, 2023.
- T. R. D. Saputri and S.-W. Lee. Addressing the concerns of the others: Handling and documenting missing concepts in self-adaptive systems. *Journal of Systems and Software*, 2020.
- M. L. Savickas. Life design counseling manual. *LifeDesign Lab*, 2020.
- N. Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill Education, 7th edition, 2020.
- R. Simamora. Buku ajar metodologi penelitian. *Pustaka Ilmu*, 2022.
- UNESCO. Career guidance: A resource handbook for low- and middle-income countries. *UNESCO Publishing*, 2021.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. URL <https://jmlr.org/papers/v9/vandermaaten08a.html>.
- M. Varezki and P. Siahaan. Perancangan sistem informasi berbasis web. *Jurnal Sistem Informasi*, 10(2), 2021.
- A. Vaswani et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- W3C. Server-sent events. Technical report, World Wide Web Consortium, 2015. URL <https://www.w3.org/TR/eventsource/>.
- D. Wang et al. Scet in the era of boundaryless careers. *Journal of Vocational Behavior*, 2022.
- J. Wang et al. Milvus: A purpose-built vector data management system. *Proceedings of SIGMOD*, 2021.

- R. Watizega et al. Sistem basis data relasional. *Jurnal Teknologi*, 2024.
- A. Wicaksana et al. Black box testing pada aplikasi web. *Jurnal Sistem Informasi*, 2025.